

Network Computing

(058172)



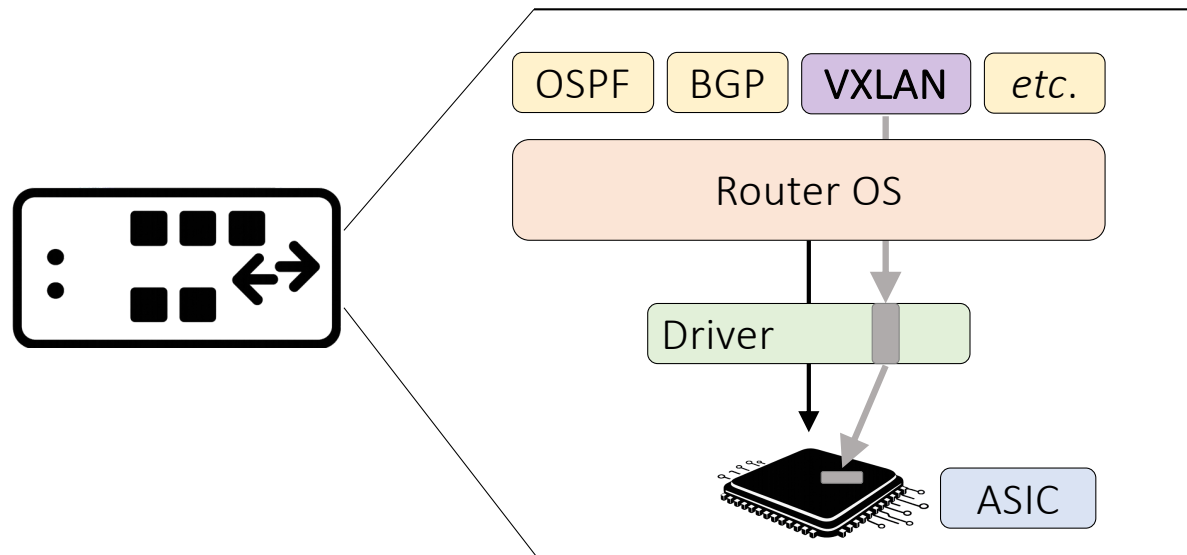
POLITECNICO
MILANO 1863

Programmable Switches

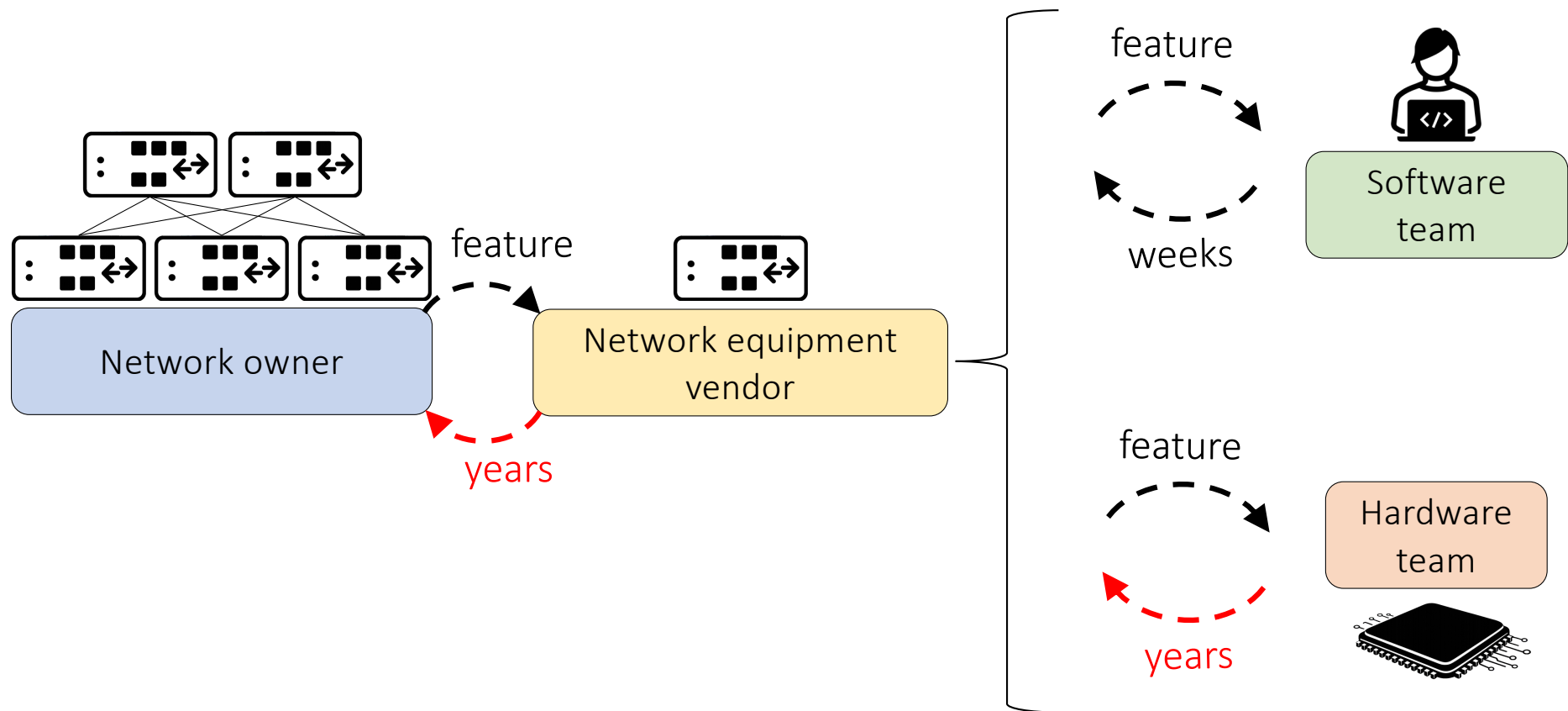
Content

- Introduction on data plane innovation
- Protocol Independent Switch Architecture
- P4 programming language

Recap from the previous lecture (1)



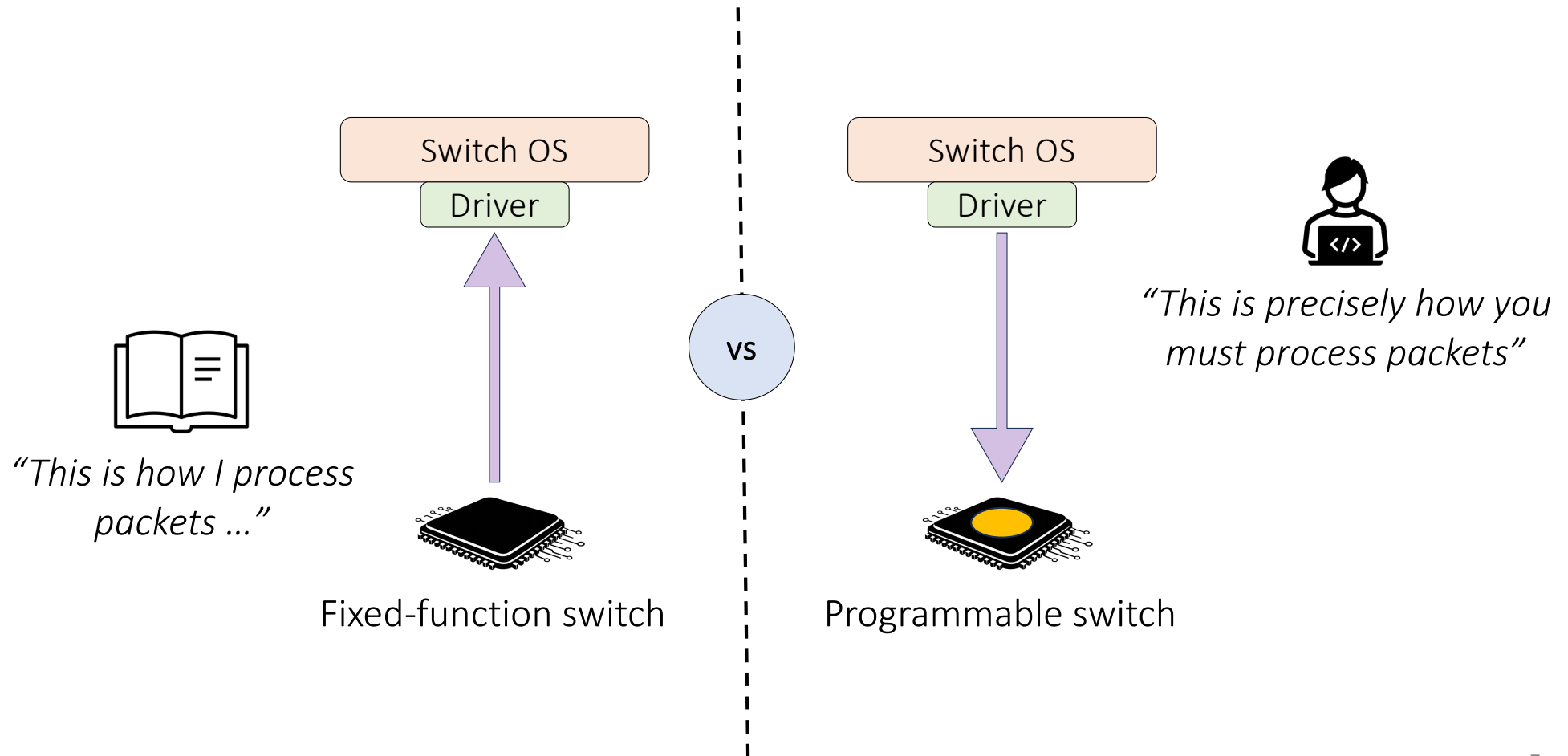
Recap from the previous lecture (2)



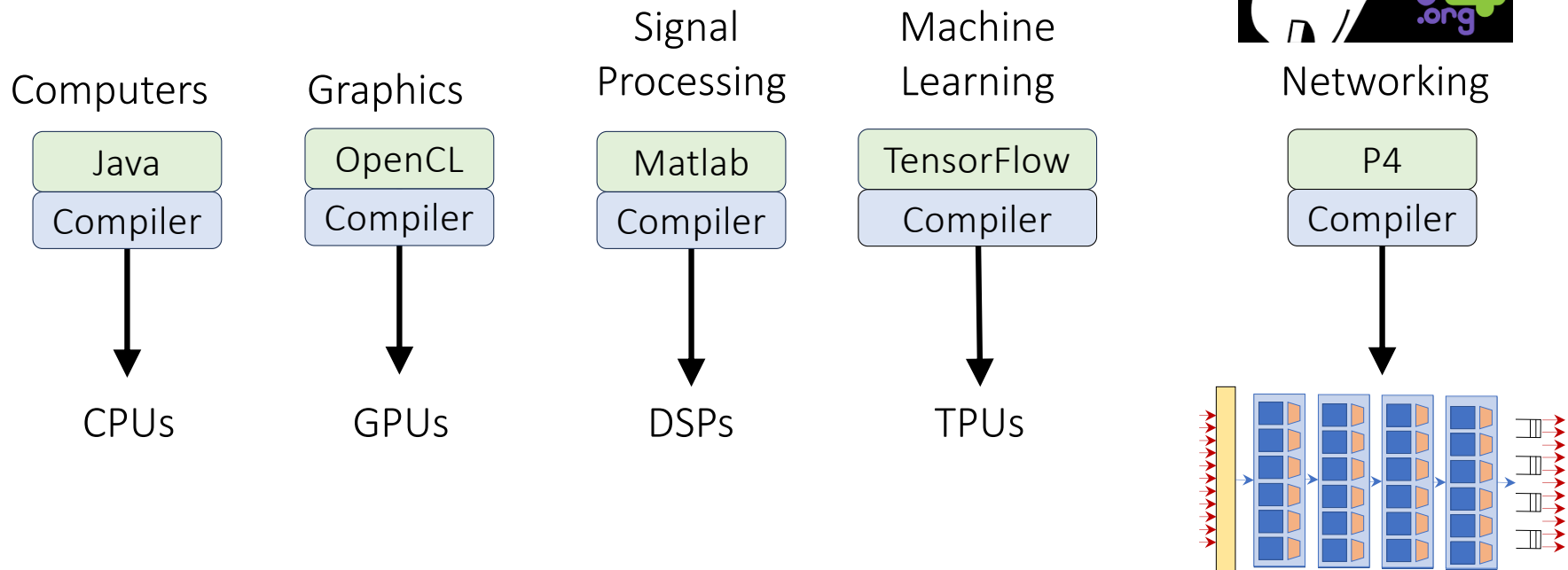
Recap from the previous lecture (3)

- Equipment vendor **can't** just send you a software upgrade
- New forwarding features take **years** to develop
- By then, you've figured out a kludge to work around it
- Your network gets more complicated
- Eventually, when the upgrade is available, it either
 - No longer solves your problem, or
 - You need a fork-lift upgrade, at huge expense

Recap from the previous lecture (4)



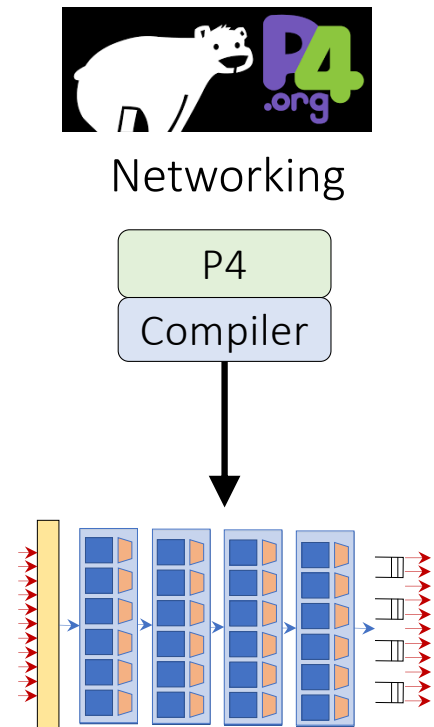
Recap from the previous lecture (5)



Recap from the previous lecture (5)

An attempt to bring to networking what RISC brought to computing.

The idea is finding the right set of primitives to help the processing of packets in high throughput and low latency



Why now?

- This seems like a cool research project or academic dream
- Three important aspects to consider:
 - Chip speed we can now make programmable switch chips as fast as fixed ones

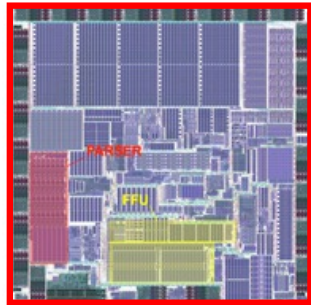
Why now?

- This seems like a cool research project or academic dream
- Three important aspects to consider:
 - Chip speed we can now make programmable switch chips as fast as fixed ones
 - Chip complexity there are now too many protocols to correctly hard-code in silicon (hence the years for new features)

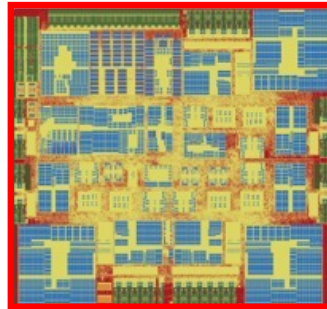
Why now?

- This seems like a cool research project or academic dream
- Three important aspects to consider:
 - **Chip speed** we can now make programmable switch chips as fast as fixed ones
 - **Chip complexity** there are now too many protocols to correctly hard-code in silicon (hence the years for new features)
 - **Chip technology** the difference in chip area and power between programmable and fixed-function is going away

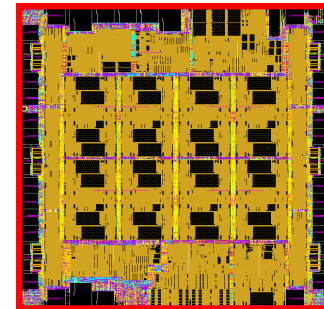
Serial I/O: about 30% of the switch chip



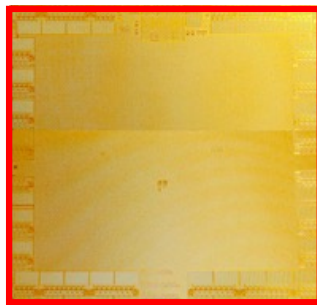
Intel Alta (2011)



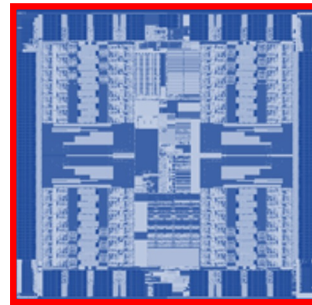
Cisco (2011)



Ericsson Spider (2011)

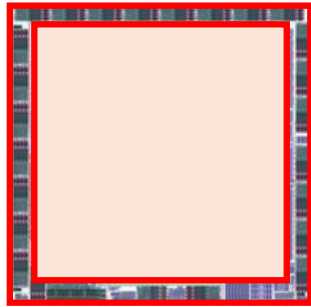


Broadcom
Tomahawk (2014)

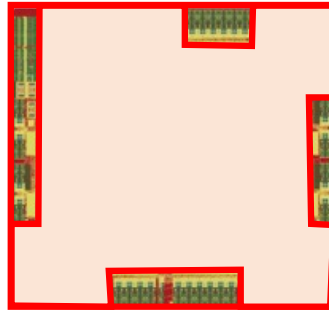


Barefoot
Tofino (2016)

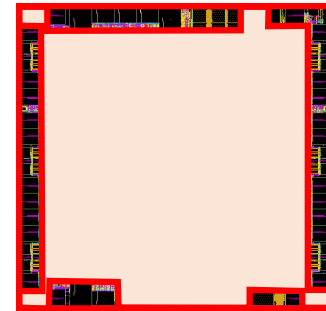
Serial I/O: about 30% of the switch chip



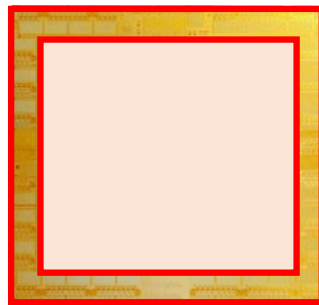
Intel Alta (2011)



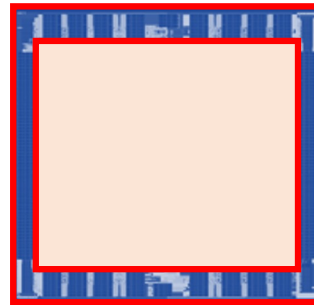
Cisco (2011)



Ericsson Spider (2011)

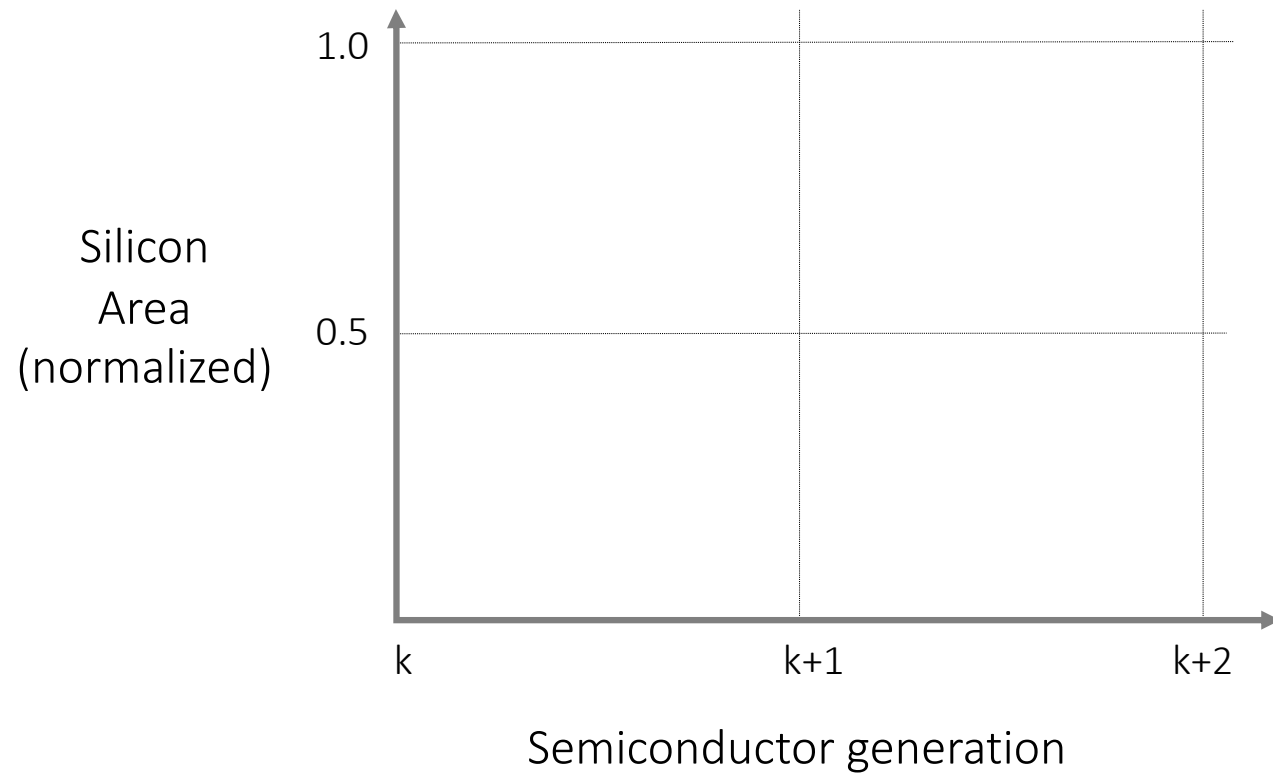


Broadcom
Tomahawk (2014)



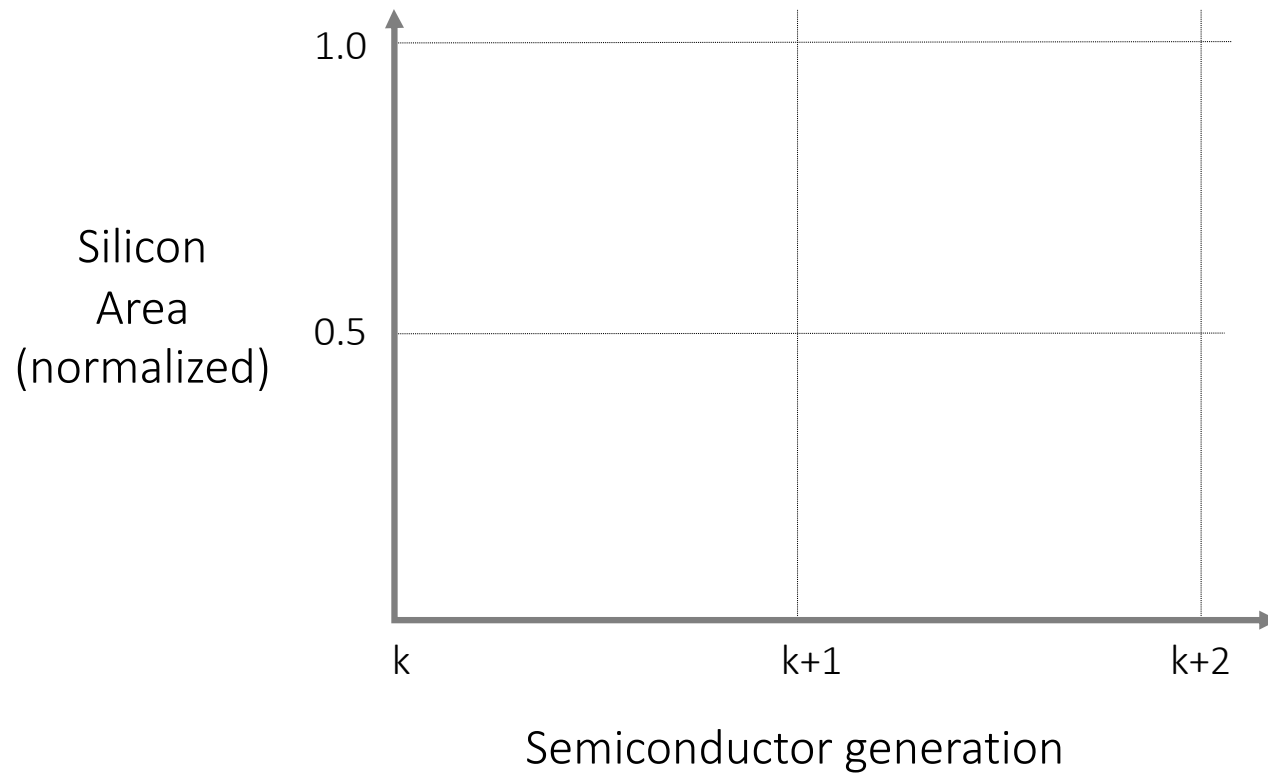
Barefoot
Tofino (2016)

Switch chip area: serial I/O



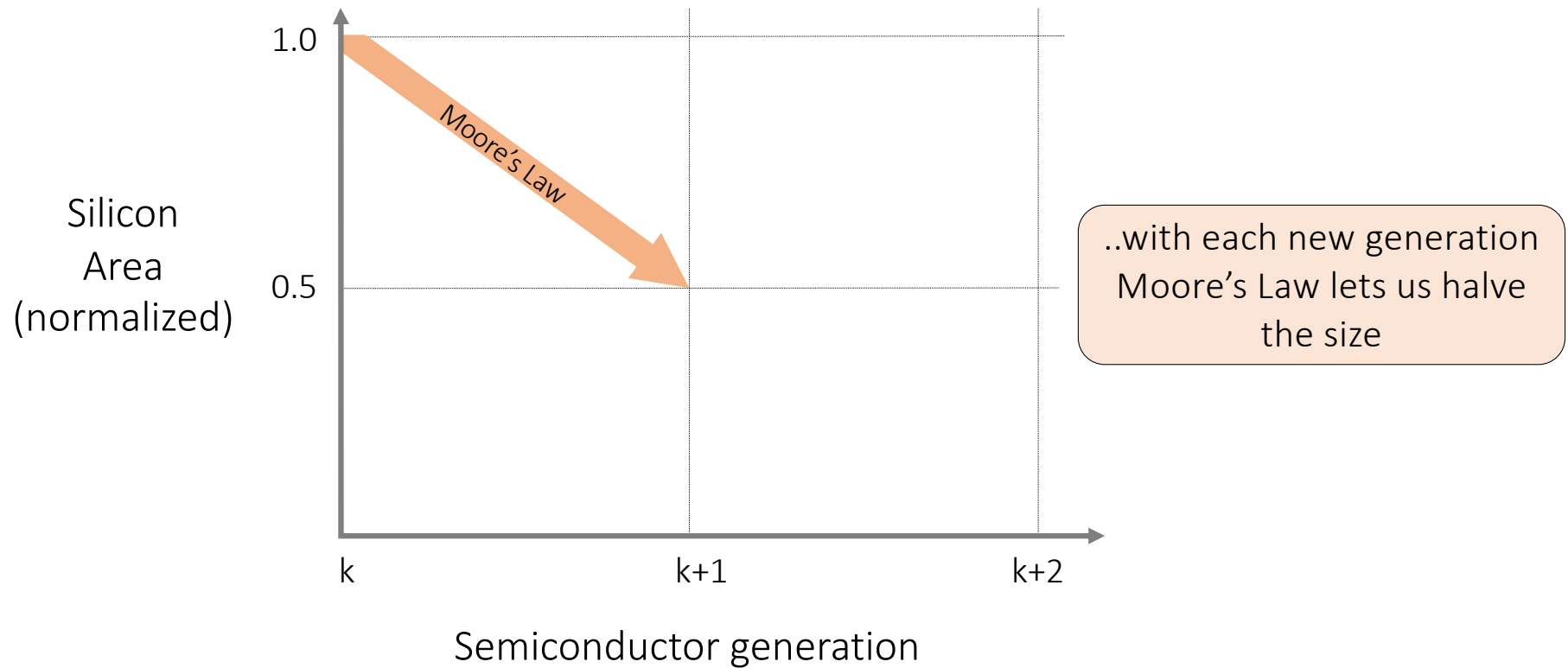
Switch chip area: serial I/O

Moore's law is the observation that the number of transistors in a dense integrated circuit doubles about every two years

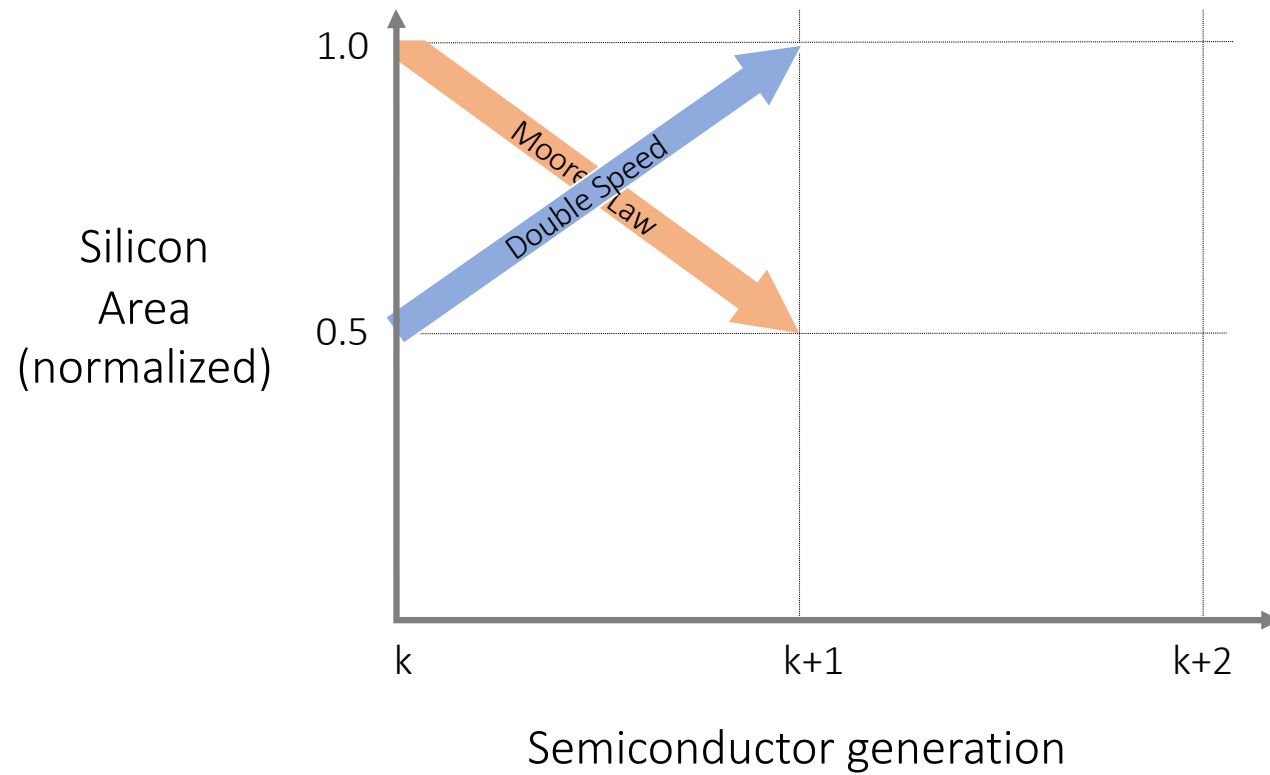


Switch chip area: serial I/O

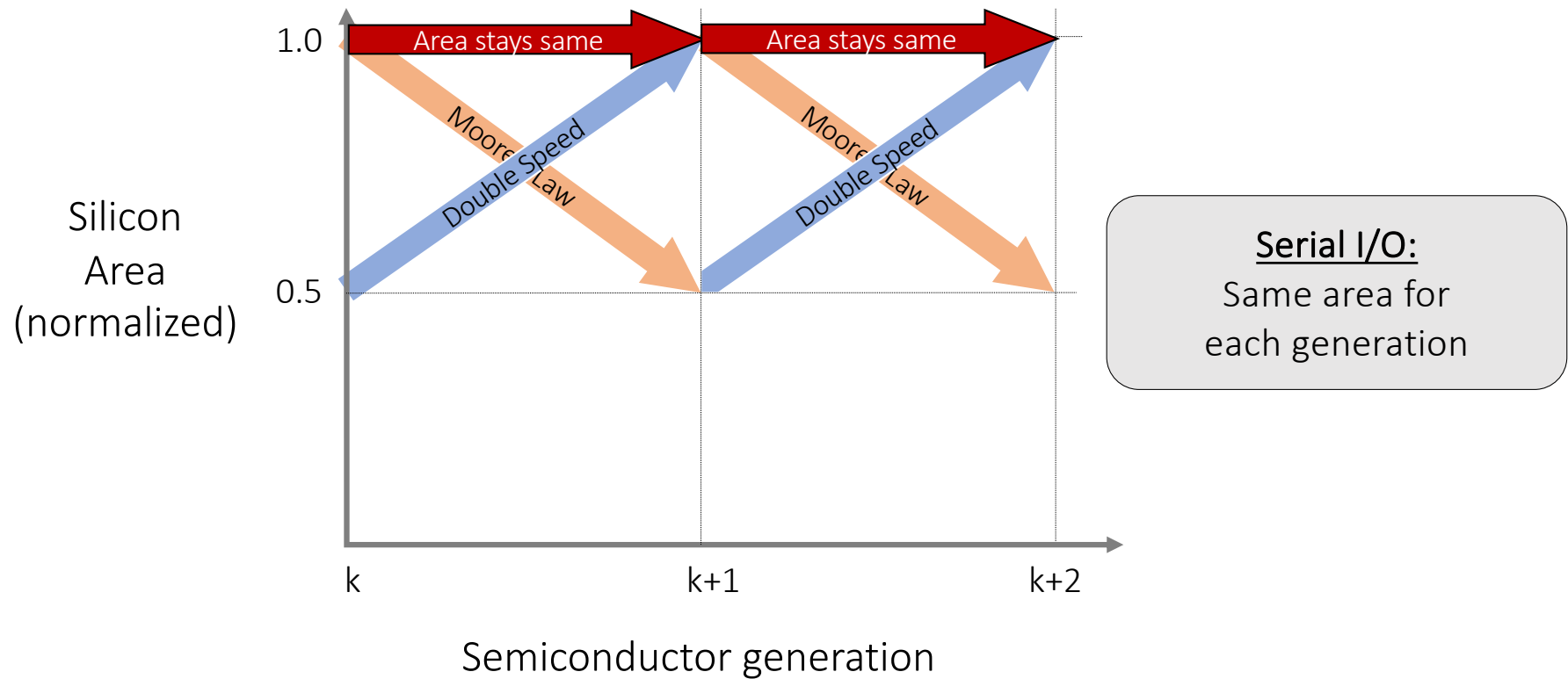
Moore's law is the observation that the number of transistors in a dense integrated circuit doubles about every two years



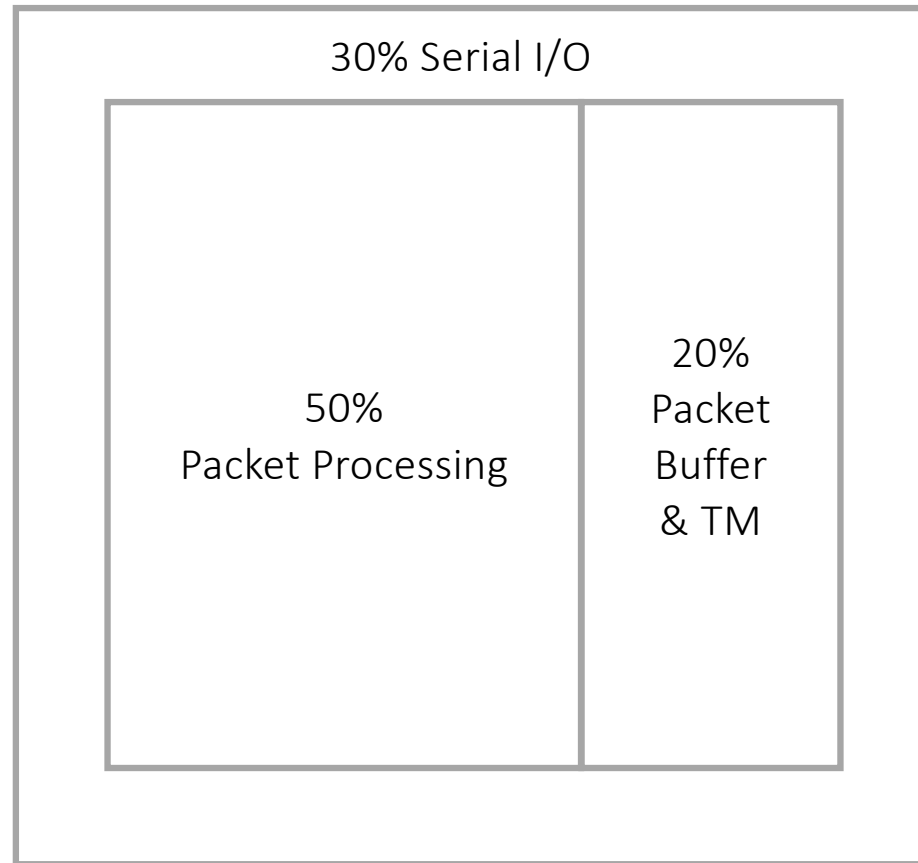
Switch chip area: serial I/O



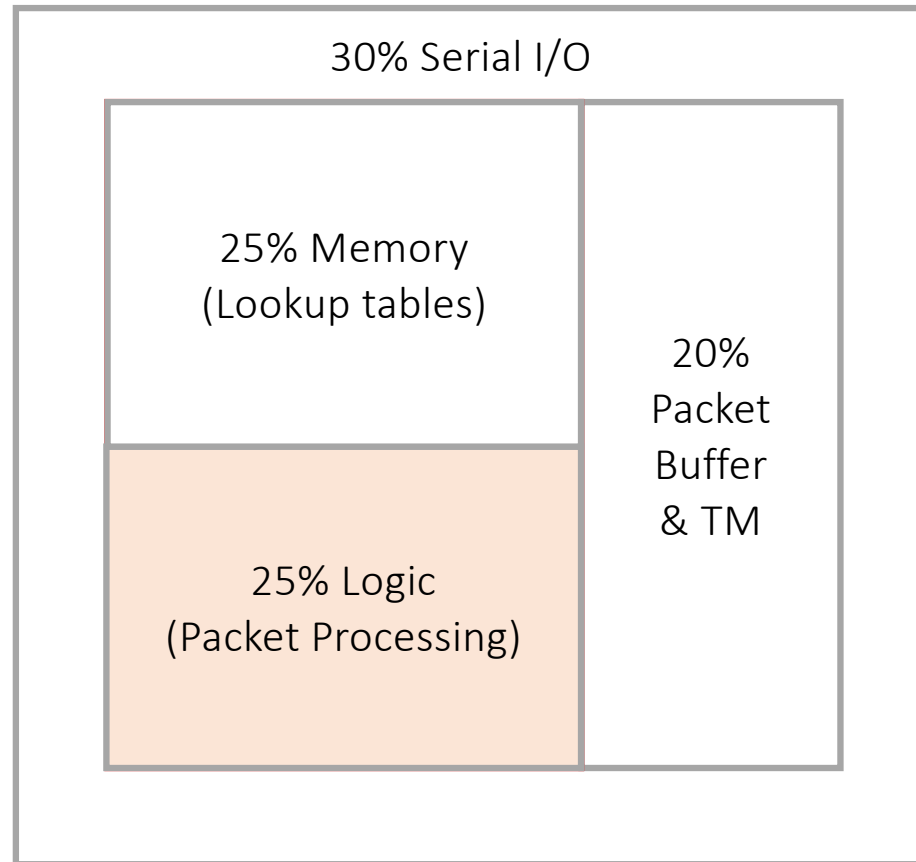
Switch chip area: serial I/O



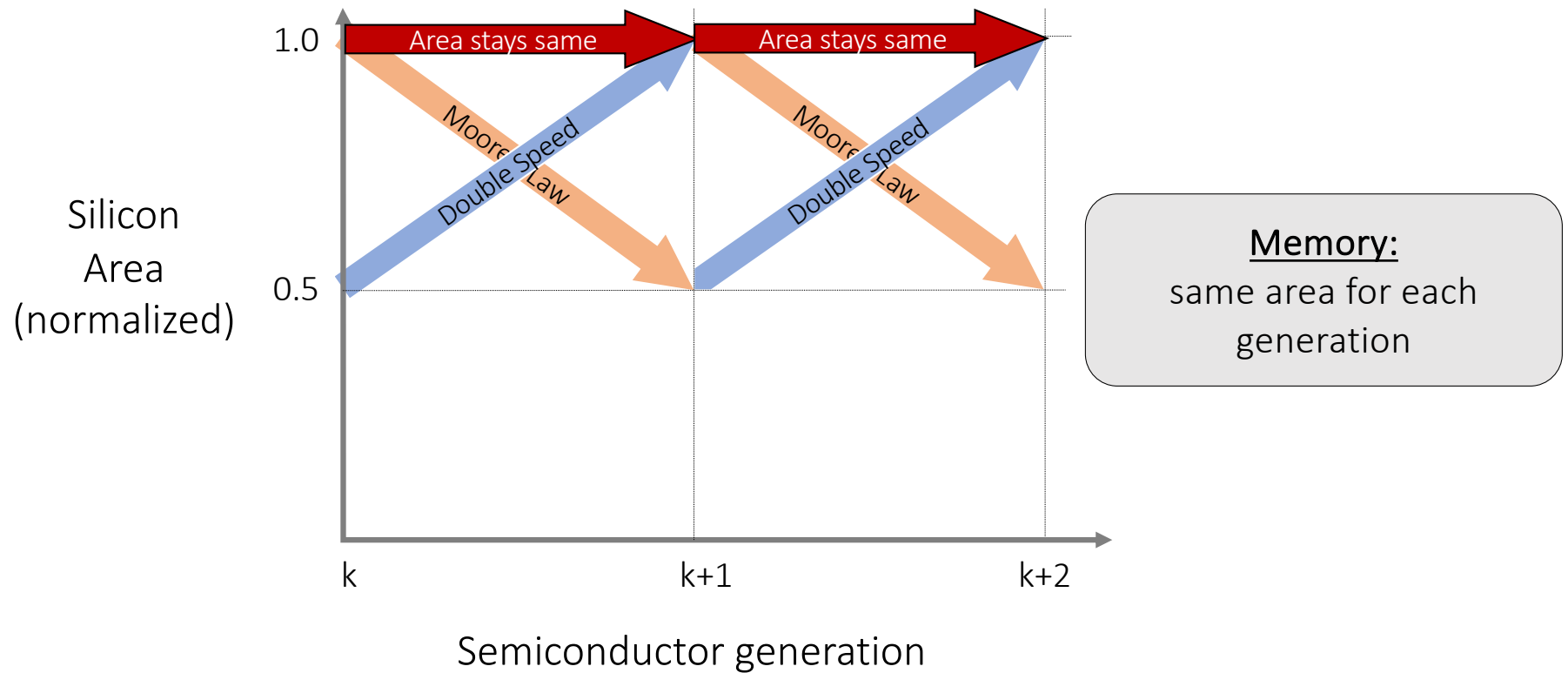
Chip area



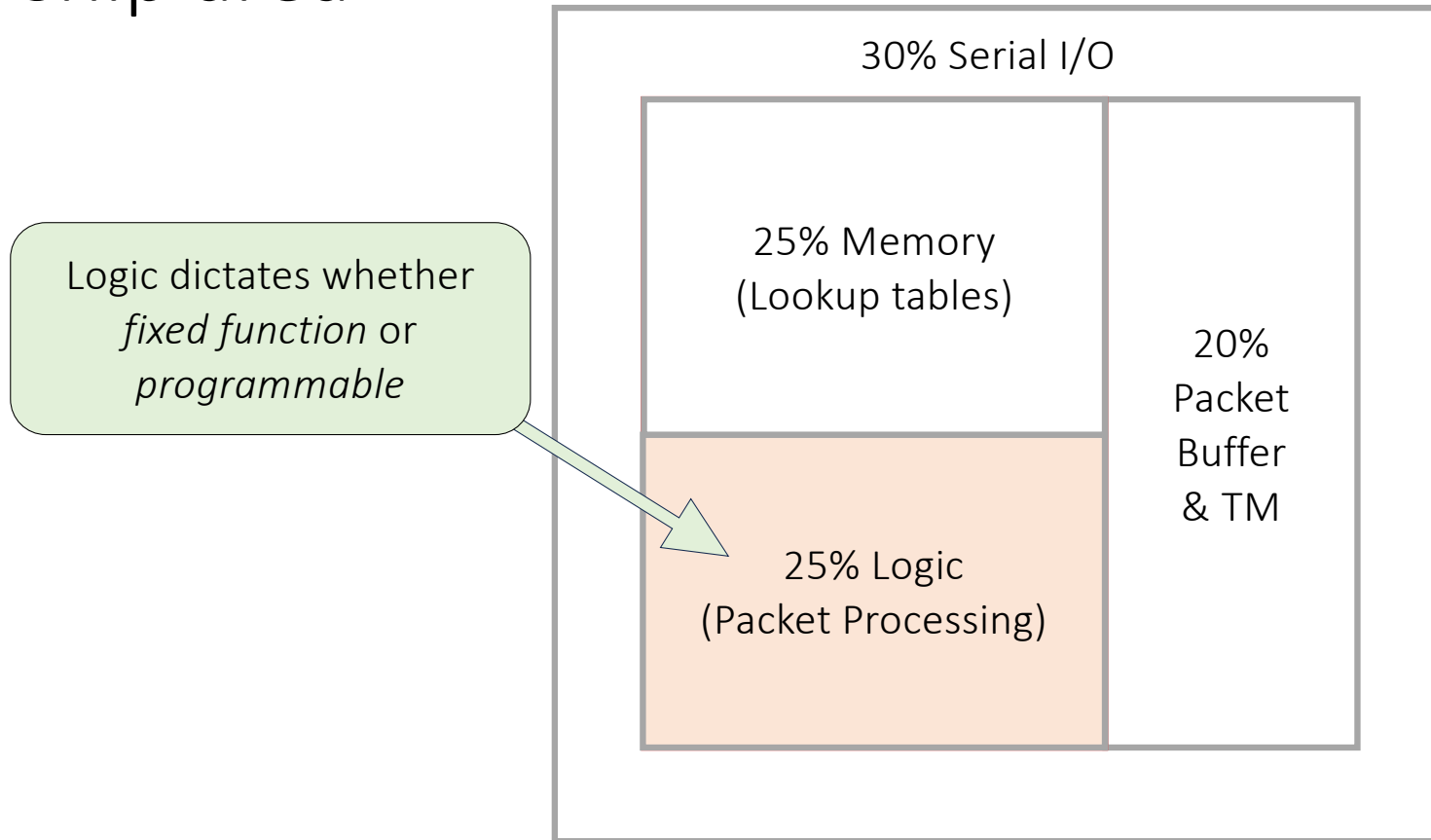
Chip area



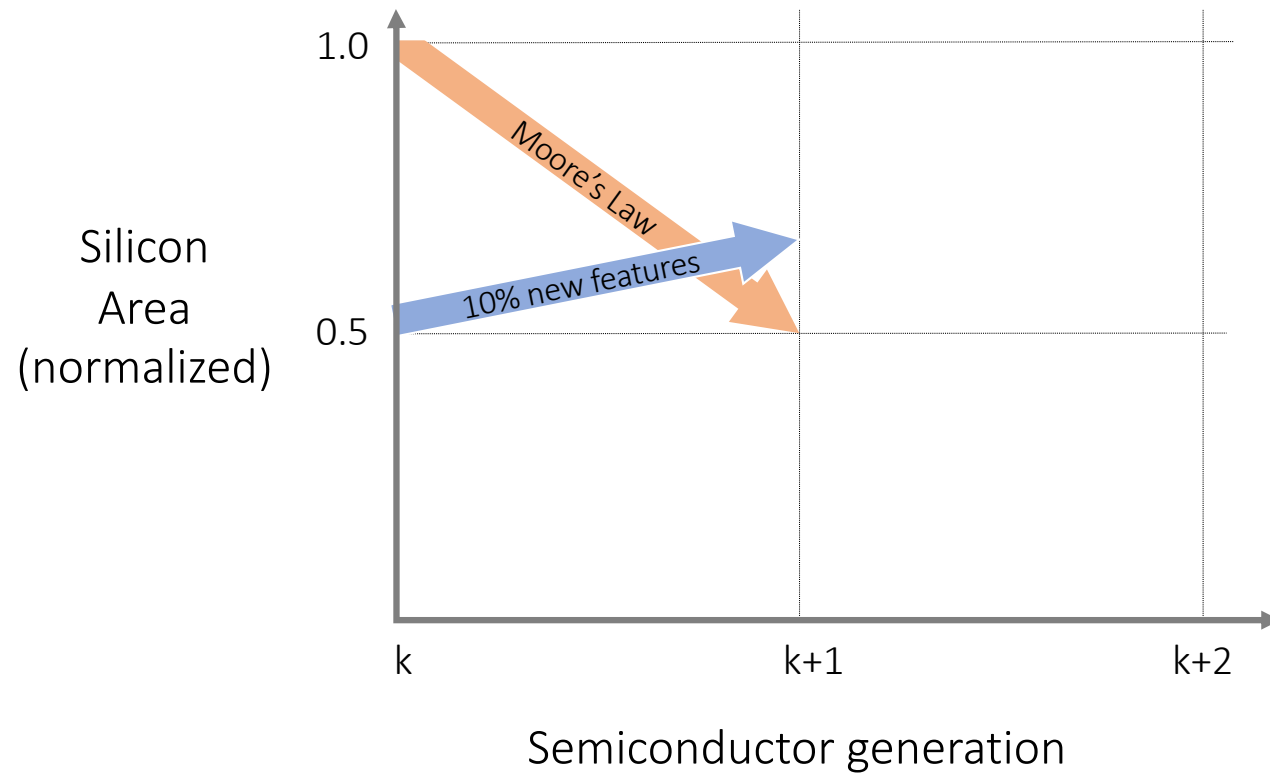
Switch chip area: memory (tables and buffer)



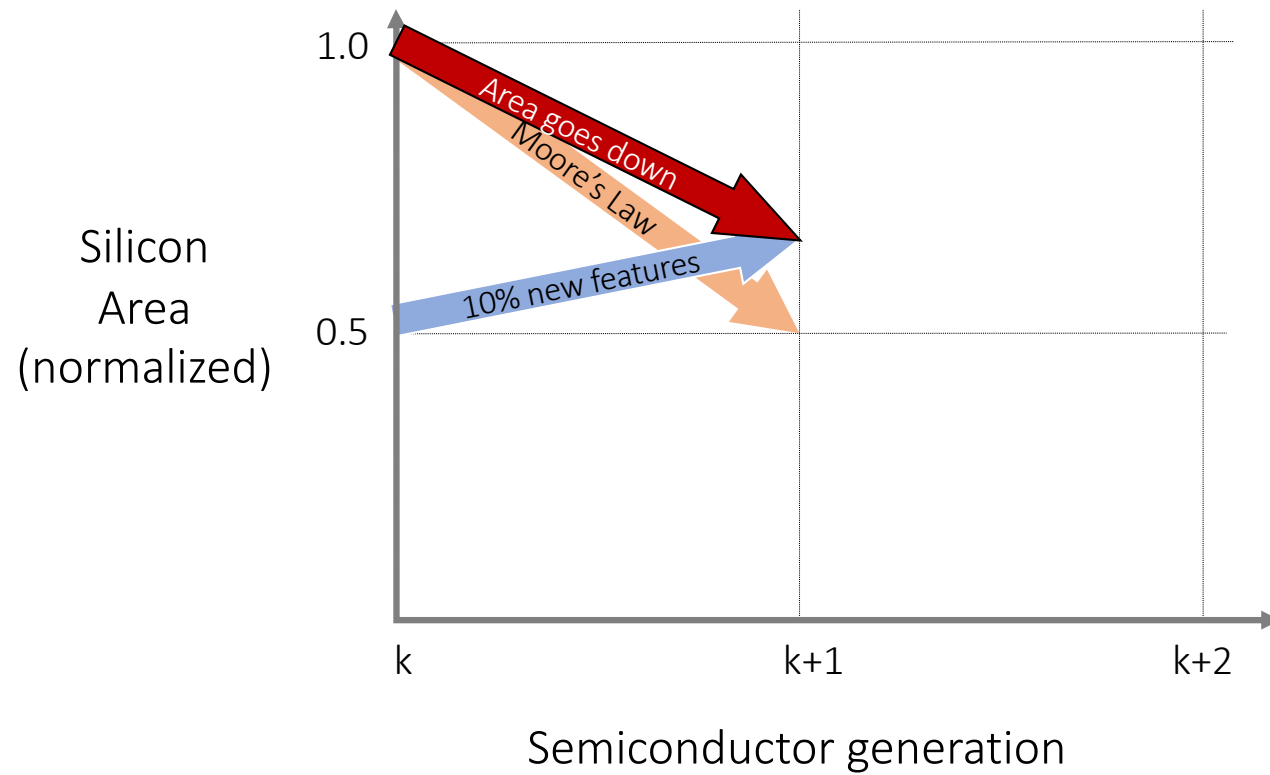
Chip area



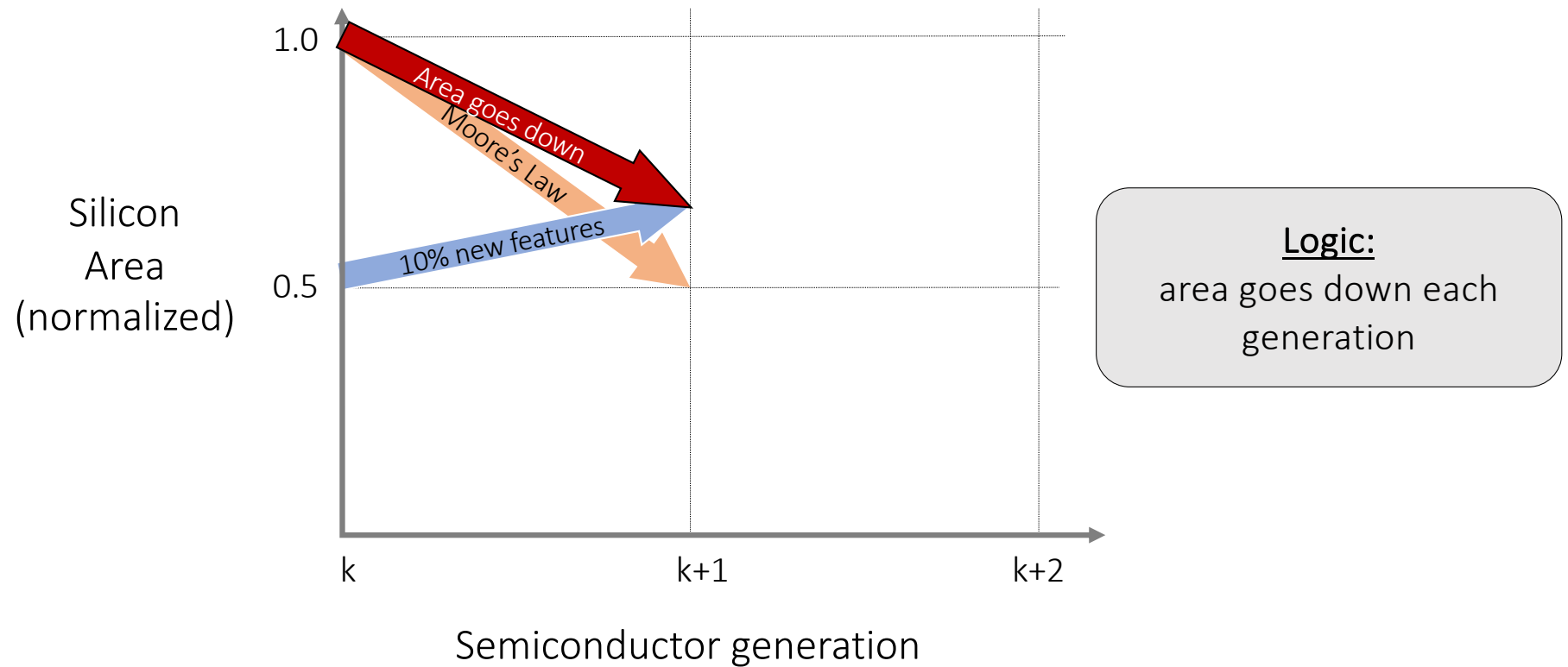
Switch chip area: packet processing logic



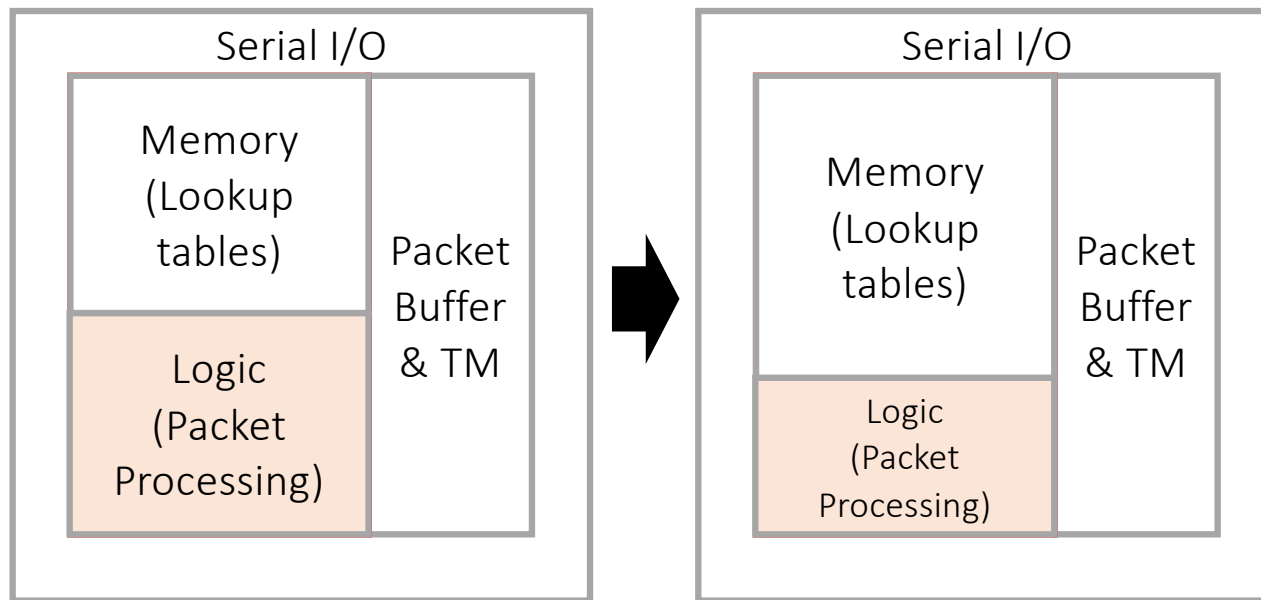
Switch chip area: packet processing logic



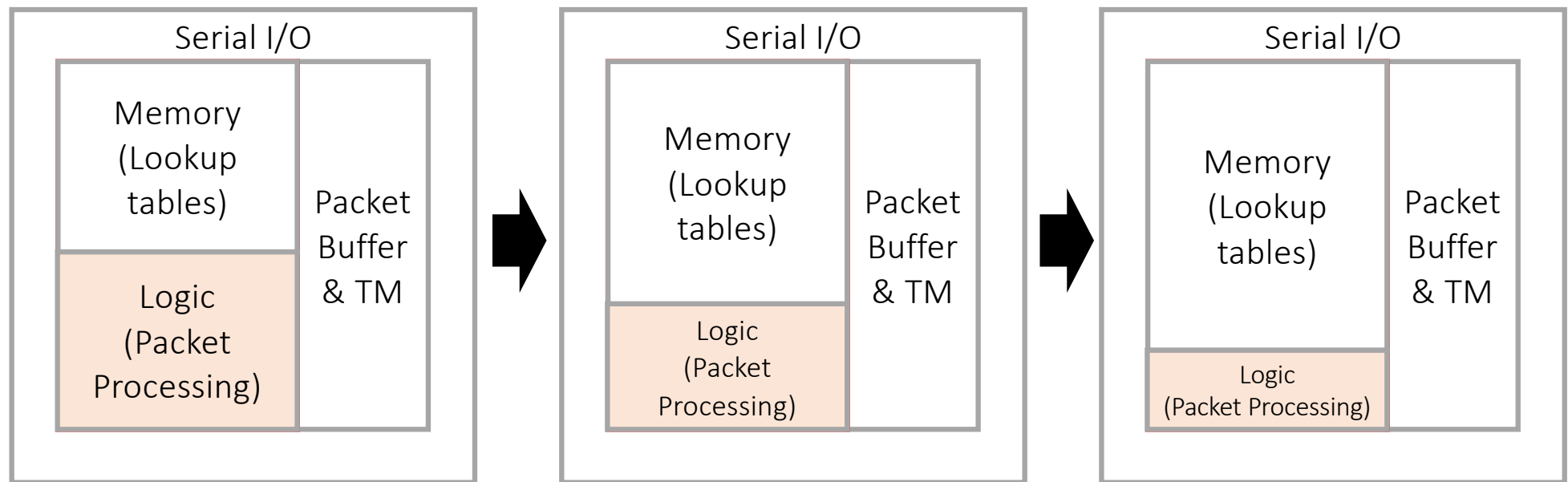
Switch chip area: packet processing logic



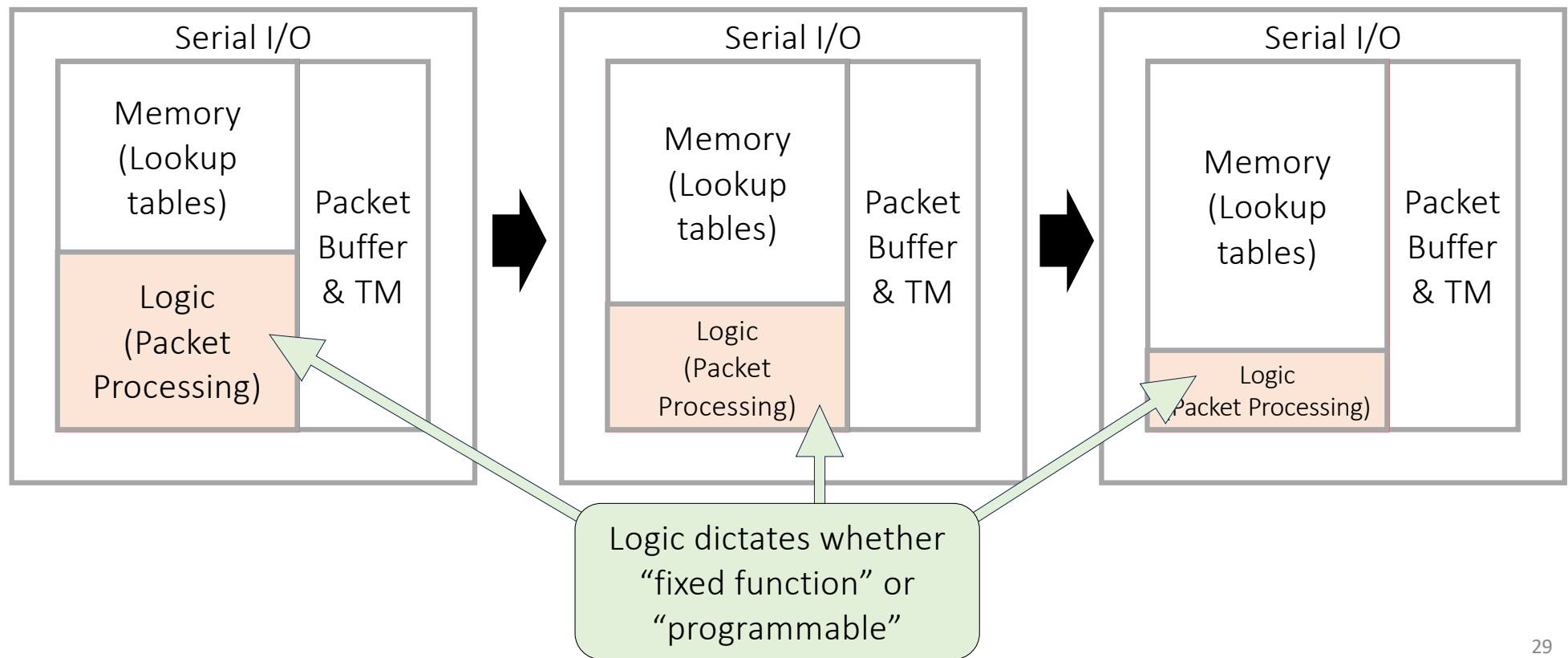
Chip area



Chip area



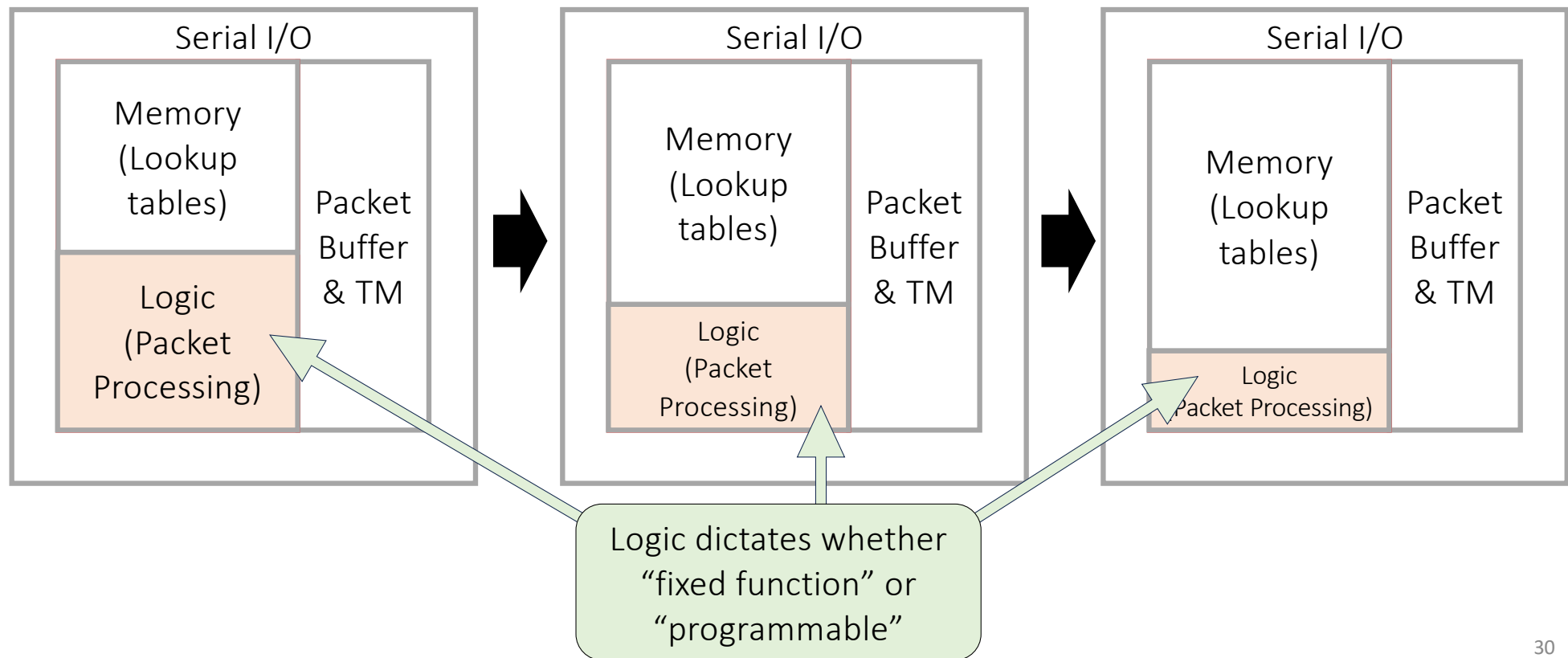
Chip area



Chip area

Takeaway:

The logic is becoming smaller. The bit that distinguishes a fixed function switch from a programmable is going away in terms of area, power and costs!



2017: Programmable Switches start to gain traction



AT&T is the first telecommunications provider to announce using programmable switches from Barefoot Networks in its network.

Barefoot Networks last summer was the first company to announce a switch operating 6.5 Tb/s. It roughly doubled the previous speed record. The company's [Tofino ICs](#) were also the first programmable switches announced. Tofino [silicon](#) began sampling only in December.



Barefoot Networks scored deals with a trio of China-based technology [companies](#) to deploy its Tofino programmable switch.

The [companies](#) include [Alibaba Group](#), [Baidu](#), and Tencent Holding, which are set to deploy Tofino switches running [open source P4](#) language, targeted at programmable forwarding plane technology. Financial terms of the deals were not released.

2018: A 12.8 Tbps programmable switch

sdxcentral SD-WAN SDN NFV CONTAINERS 5G SECURITY EDGE MULTI-CLOUD OPE

News

Barefoot Networks Debuts Tofino 2, Using 7nm Technology

**Ali Longwell**
December 4, 2018 6:24 PM

Share this article:
     



Barefoot Networks released the second generation of its P4-programmable Tofino Ethernet switch ASIC family, called Tofino 2. The new switch uses 7 nanometer (nm) technology to process packets at 12.8 Tb/s to better support hyperscale data centers and emerging network architectures and application workloads.

The company claims that Tofino 2 delivers twice the performance of its predecessor. It still leverages the open source programming language, P4, so that it can be deployed in a multitude of ways. Specifically, Tofino 2 can be deployed from a top-of-rack switch, a service provider router, or as an appliance. It delivers 32x400GE on a single chip for hyperscale data centers that require 400GE.

2018: A 12.8 Tbps programmable switch



..to put this in perspective



21 JANUARY 2022

Data traffic at DE-CIX Internet Exchanges jumps to over 38 exabytes

DE-CIX in Frankfurt is one of the largest Internet Exchanges in the world, with over 29 exabytes of data traffic in 2021, a peak data throughput of nearly 11 terabits per second (Tbps), and over 1100 connected networks. DE-CIX currently operates a total of 32 Internet Exchanges worldwide, either alone or in various cooperation and partnership models.

Barefoot Networks released the second generation of its P4-programmable Tofino Ethernet switch ASIC family, called Tofino 2. The new switch uses 7 nanometer (nm) technology to process packets at 12.8 Tb/s to better support hyperscale data centers and emerging network architectures and application workloads.

The company claims that Tofino 2 delivers twice the performance of its predecessor. It still leverages the open source programming language, P4, so that it can be deployed in a multitude of ways. Specifically, Tofino 2 can be deployed from a top-of-rack switch, a service provider router, or as an appliance. It delivers 32x400GE on a single chip for hyperscale data centers that require 400GE.

2019: Barefoot sold to Intel



2023: Intel halts Tofino development 😊

HARDWARE

Time to save: Intel stopped development of network switches and ended the support program for chip designers on RISC-V

© January 27, 2023 🗣️ Dylan Harris ➡️ Add Comment

For Intel, the past fourth quarter was one of the worst, if not in history, then certainly in years — revenue plummeted 32% and the company lost \$700 million. Unsurprisingly, Intel has announced new cost-cutting measures. Discontinued investments in new products in the Network Switching Division. Intel is also ending support for the RISC-V Pathfinder program.



Image source: Intel

...but nothing is lost!

ARISTA

7170B Series Programmable Data Center Switches

Data Sheet

Using Trio – Juniper Networks’ Programmable Chipset – for Emerging In-Network Applications

Mingran Yang[†], Alex Baban*, Valery Kugel*, Jeff Libby*, Scott Mackie*,

Swamy Sadashivaiah Renu Kananda*, Chang-Hong Wu*, Manya Ghobadi[†]

[†]Massachusetts Institute of Technology *Juniper Networks

NEWS

Netberg launches Aurora 721 and Aurora 421 programmable switches

Taoyuan city, Taiwan, 24th of June 2024.

Netberg announced the new [Aurora 721](#) 100G and [Aurora 421](#) 10G switches, which feature programmable pipelines powered by Broadcom StrataXGS® Trident3 Ethernet switch chips.

6.4T PROGRAMMABLE DATA CENTER SWITCH

DCS800

Wedge 100BF-32X
32 x 100G QSFP28 switch ports with Tofino 32D.
Programmable Tofino switches for data centers



High-Speed Excellence

APS6132Q

Features

- ✓ Intel Tofino 3.2 Tb/s network ASIC
- ✓ 8-core Intel Xeon D-1700 CPU
- ✓ 32x 100Gb/s ports
- ✓ Supports 10/25/40/50/100Gb/s data rates
- ✓ OpenBMC Management
- ✓ PTP Time Synchronization (optional)

Asterfusion releases 400G P4 Programmable Switch by Intel Tofino 2

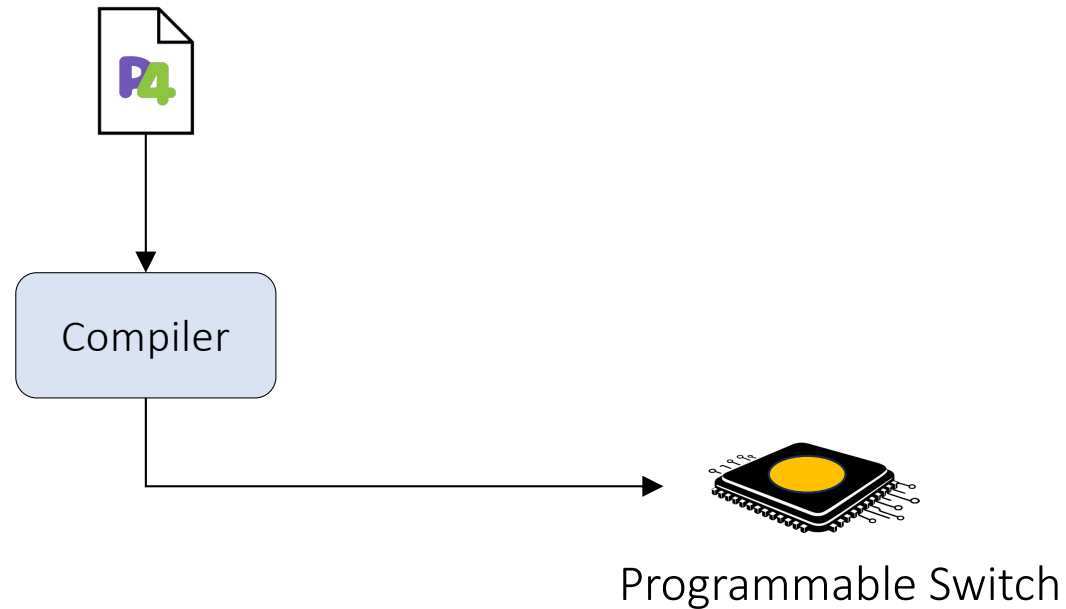


Asterfusion data technologies · [Follow](#)

3 min read · Nov 8, 2023

What data plane programmability has brought

- The ability to programmatically decide which features to enable on your switch



What data plane programmability has brought

- The ability to programmatically decide which features to enable on your switch



IPv4 and IPv6 routing

- Unicast Routing
 - Routed Ports & SVI
 - VRF
- Unicast RPF
 - Strict and Loose
- Multicast
 - PIM-SM/DM & PIM-Bidir

Ethernet switching

- VLAN Flooding
- MAC Learning & Aging
- STP state
- VLAN Translation

Load balancing

- LAG
- ECMP & WCMP
- Resilient Hashing
- Flowlet Switching

Fast Failover

- LAG & ECMP

Tunneling

- IPv4 and IPv6 Routing & Switching
 - IP-in-IP (6in4, 4in4)
 - VXLAN, NVGRE, GENEVE & GRE
 - Segment Routing, ILA

MPLS

- LER and LSR
- IPv4/v6 routing (L3VPN)
- L2 switching (EoMPLS, VPLS)
- MPLS over UDP/GRE

ACL

- MAC ACL, IPv4/v6 ACL, RACL
- QoS ACL, System ACL, PBR
- Port Range lookups in ACLs

QoS

- QoS Classification & marking
- Drop profiles/WRED
- RoCE v2 & FCoE
- CoPP (Control plane policing)

NAT and L4 Load Balancing

Security Features

- Storm Control, IP Source Guard

Monitoring & Telemetry

- Ingress Mirroring and Egress Mirroring
- Negative Mirroring
- Sflow
- INT

Counters

- Route Table Entry Counters
- VLAN/Bridge Domain Counters
- Port/Interface Counters

Protocol Offload

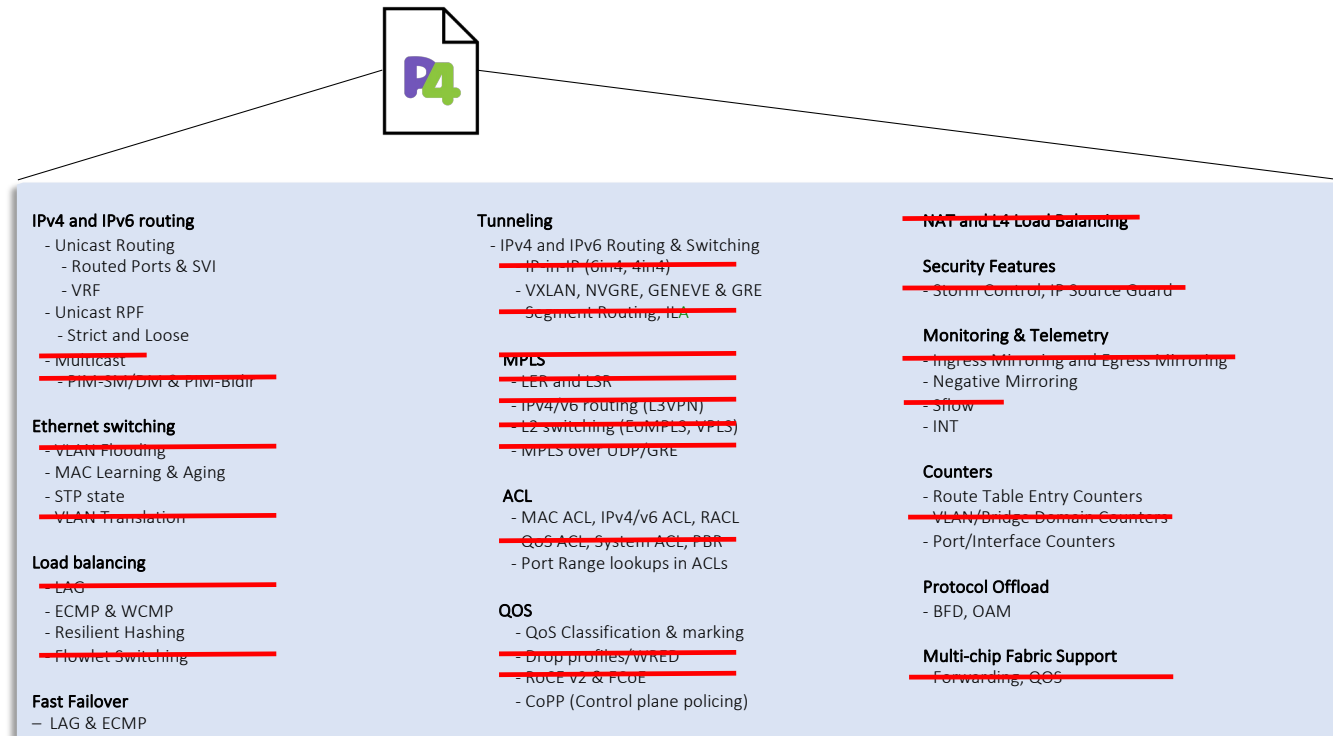
- BFD, OAM

Multi-chip Fabric Support

- Forwarding, QOS

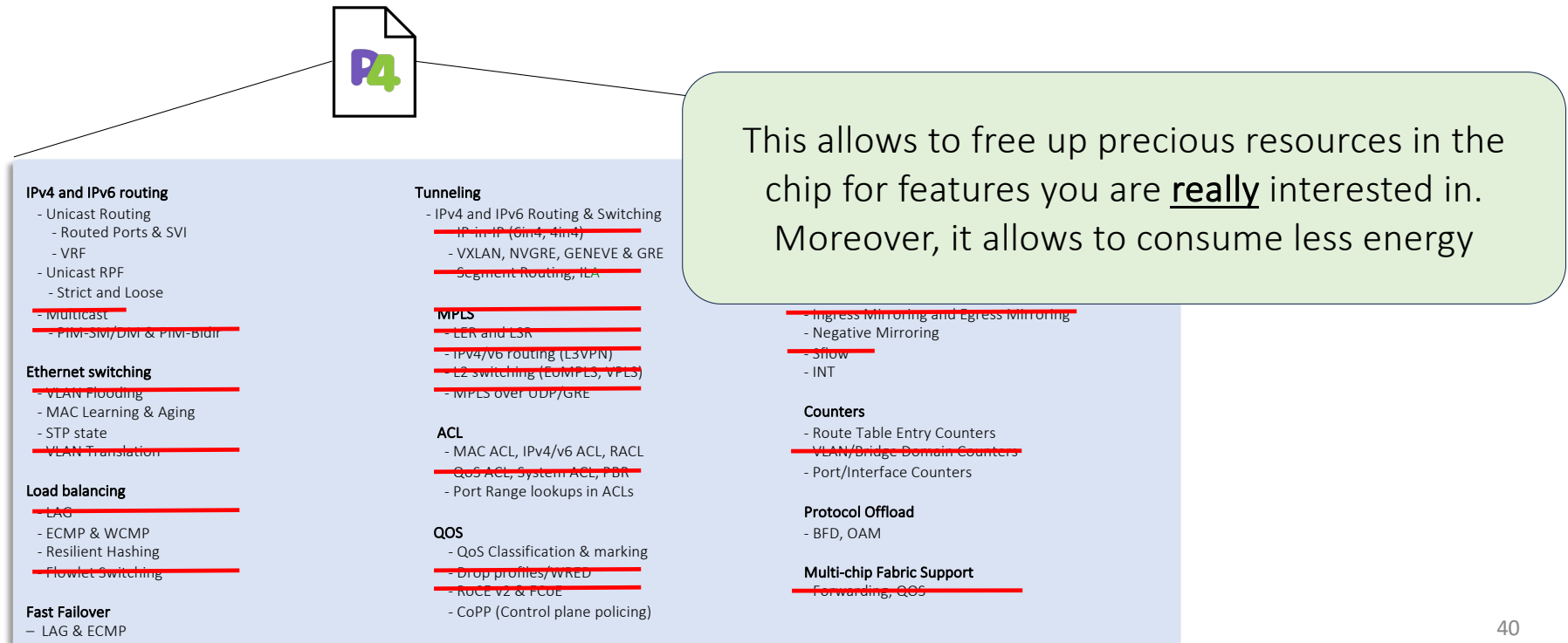
What data plane programmability has brought

- The ability to programmatically decide which features to enable on your switch



What data plane programmability has brought

- The ability to programmatically decide which features to enable on your switch



What data plane programmability has brought

- The ability to enhance the data plane with new features
 - New encapsulations and tunnels
 - New ways to tag packets for special treatments
 - New approaches to network monitoring
 - New way to route packets into the network
 - New mechanisms for congestion control

What data plane programmability has brought

- The ability to enhance the data plane with new features
 - New encapsulations and tunnels
 - New ways to tag packets for special treatments
 - New approaches to network monitoring
 - New way to route packets into the network
 - New mechanisms for congestion control

This will be matter for next lectures



Before we continue..

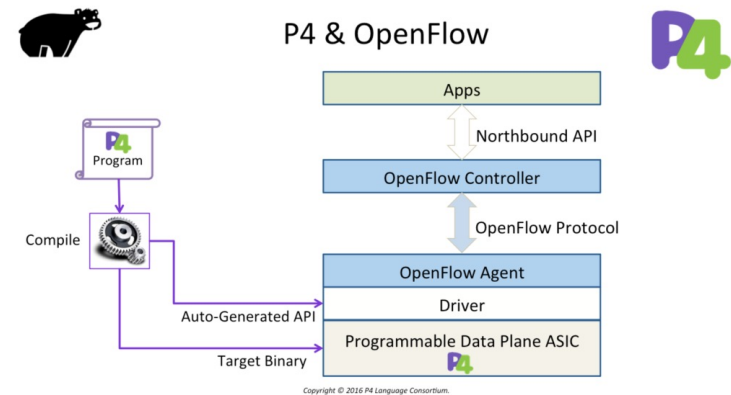
- Is an OpenFlow switch a programmable switch?

P4 vs OpenFlow

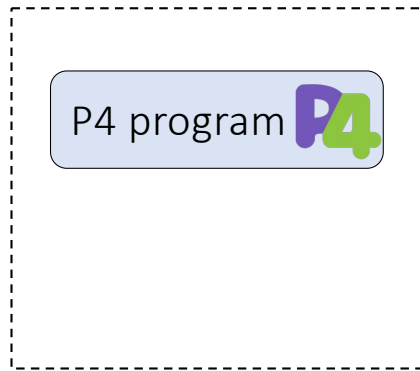
- P4 is not OpenFlow 2.0
- P4 addresses a much more general problem
- P4 addresses the *need to program* the data plane
- With P4 you can implement an OpenFlow switch for instance

P4 vs OpenFlow

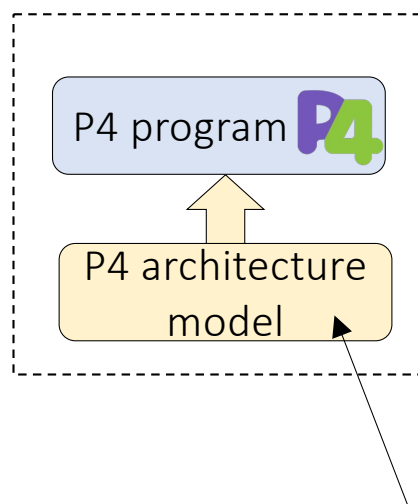
- P4 is not OpenFlow 2.0
- P4 addresses a much more general problem
- P4 addresses the *need to program* the data plane
- With P4 you can implement an OpenFlow switch (*openflow.p4*)
(<https://github.com/p4lang/p4ofagent/blob/master/p4src/openflow.p4>)



The workflow

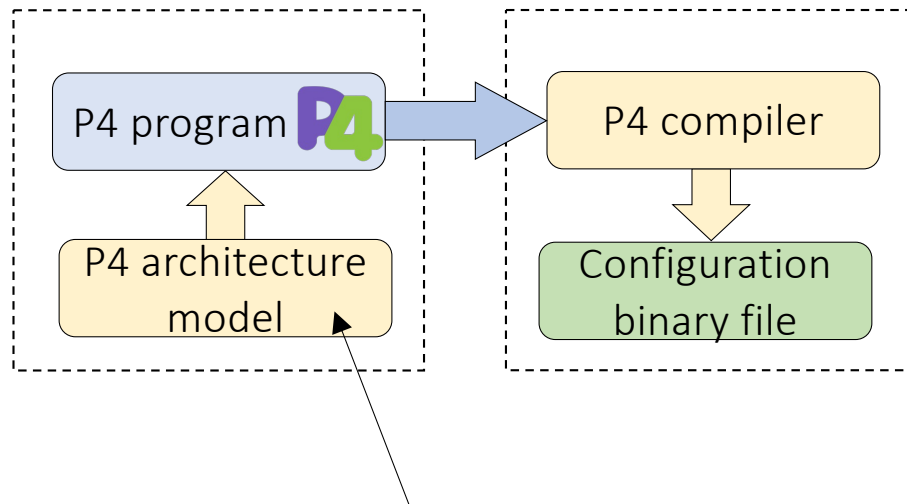


The workflow



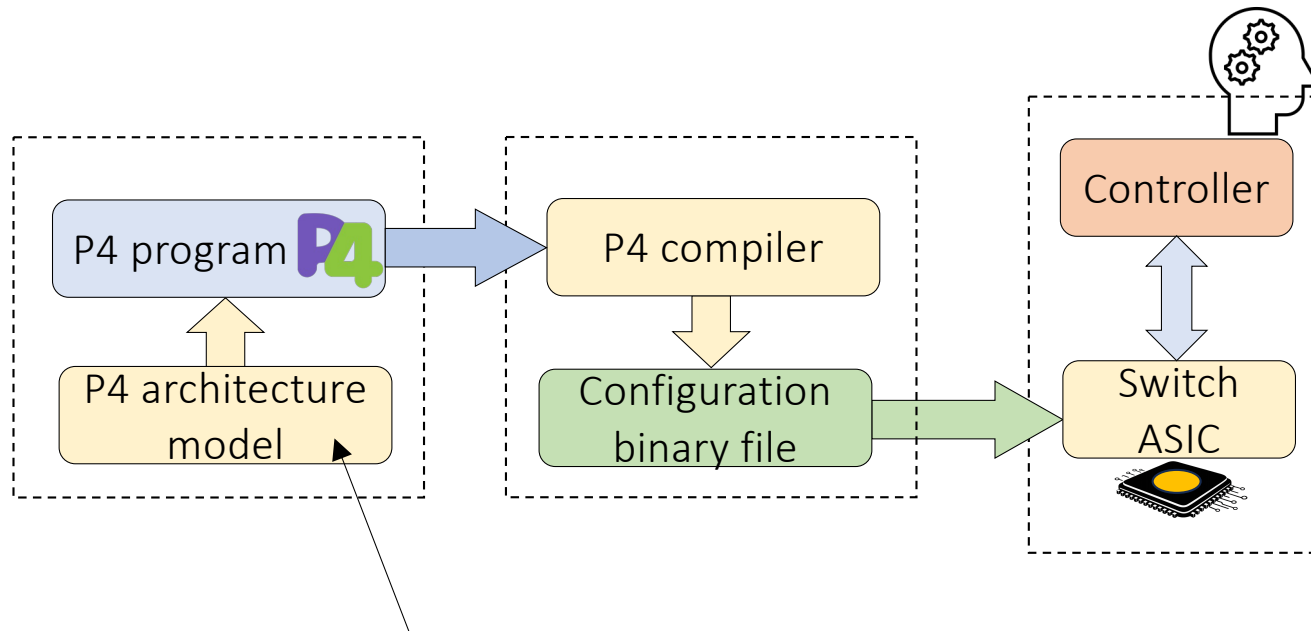
It provides a logical view of the underlying processing so to insulate programmers from the hardware details. This allows P4 programs to compile to different targets (hardware devices)

The workflow



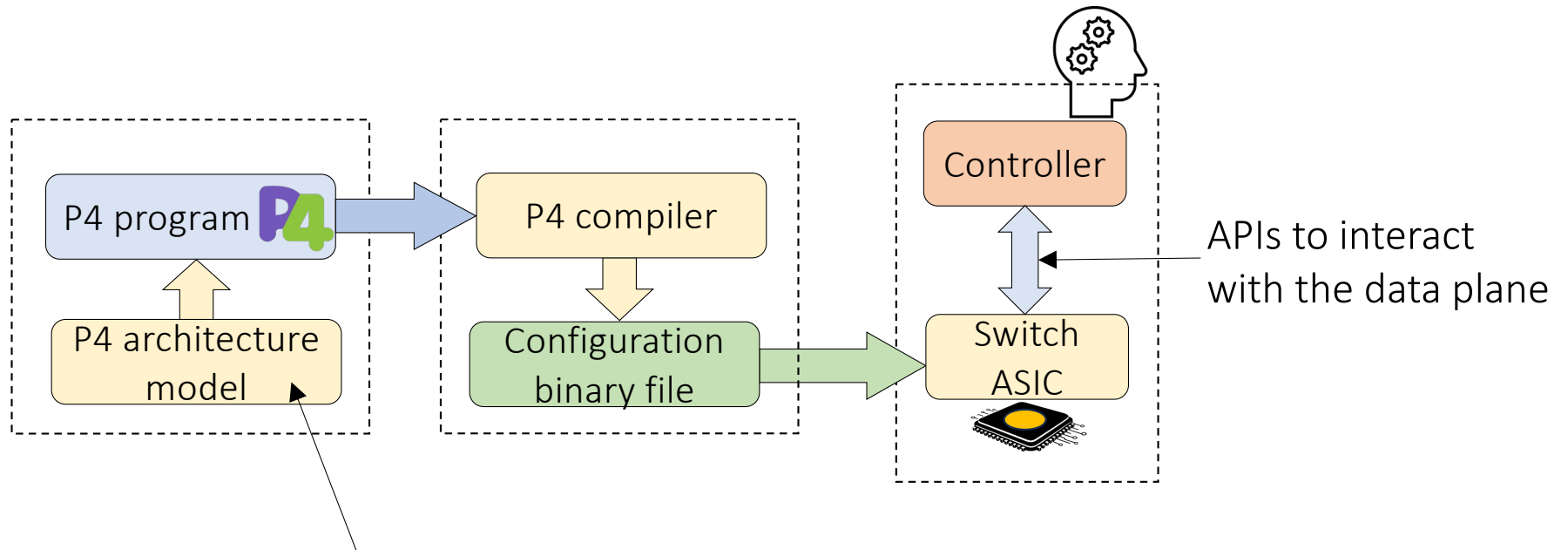
It provides a logical view of the underlying processing so to insulate programmers from the hardware details. This allows P4 programs to compile to different targets (hardware devices)

The workflow



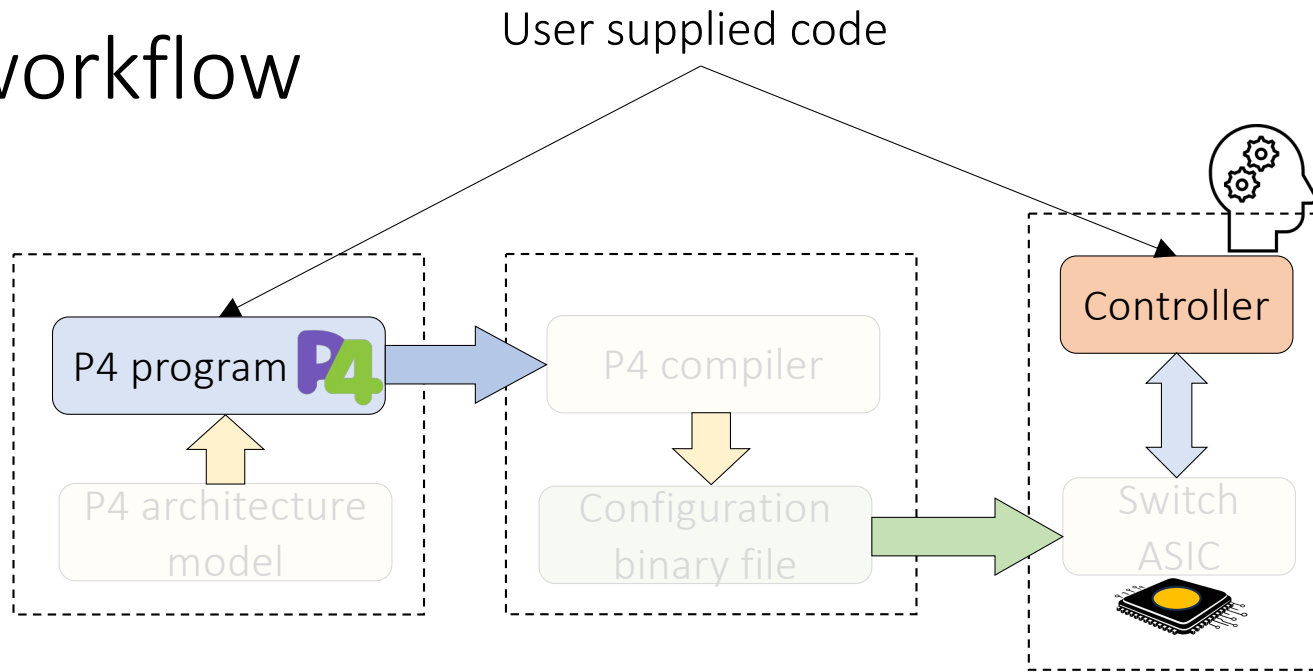
It provides a logical view of the underlying processing so to insulate programmers from the hardware details. This allows P4 programs to compile to different targets (hardware devices)

The workflow

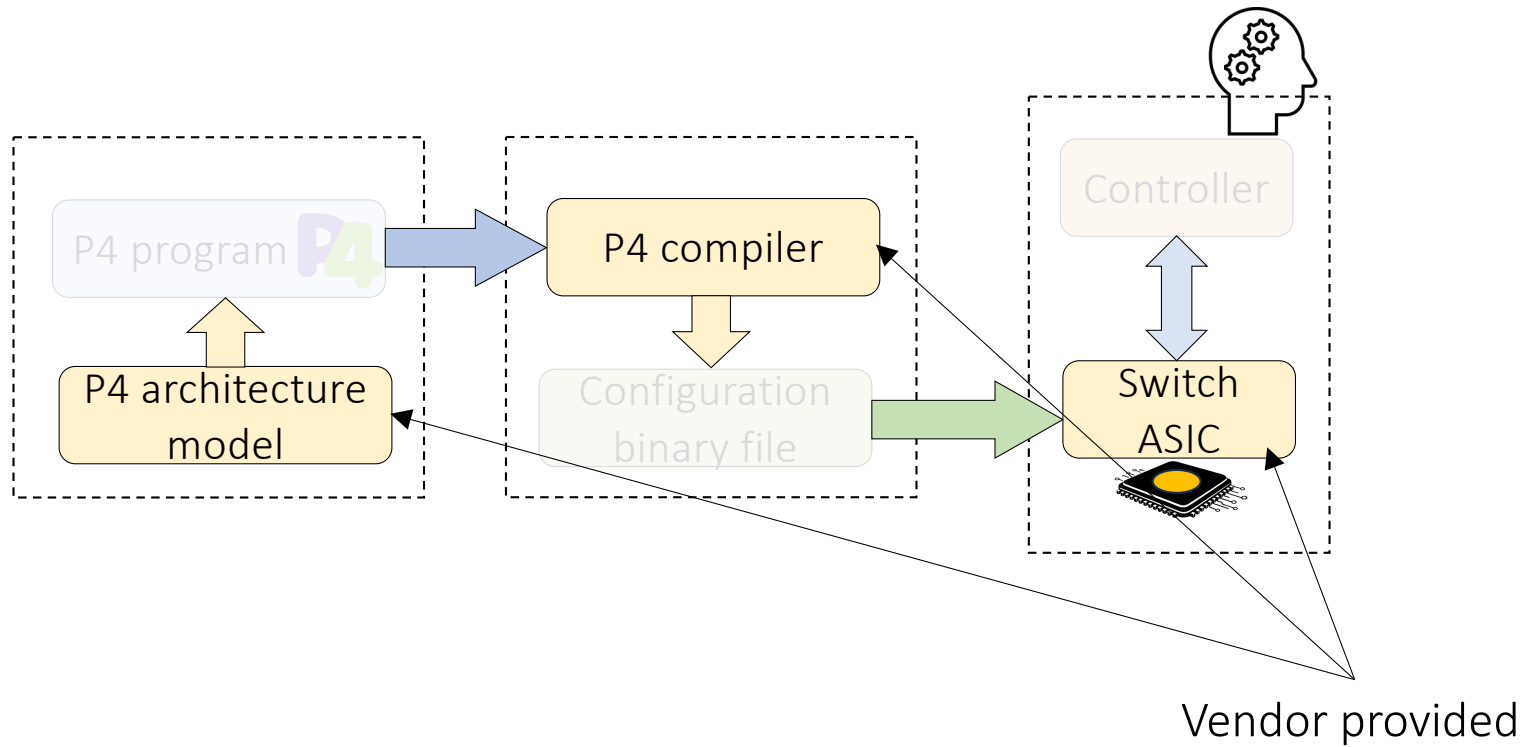


It provides a logical view of the underlying processing so to insulate programmers from the hardware details. This allows P4 programs to compile to different targets (hardware devices)

The workflow

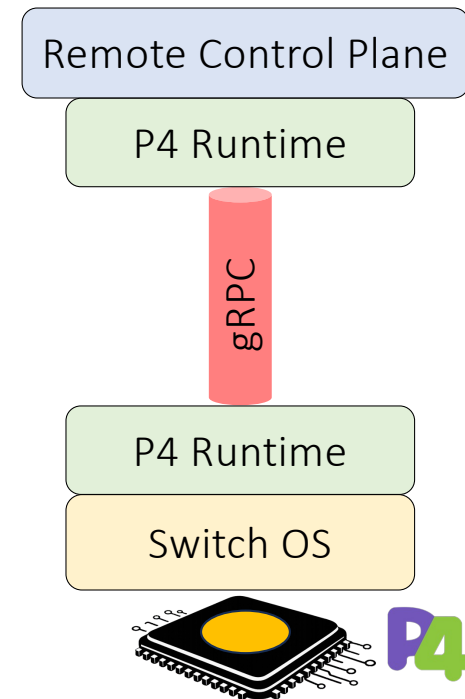


The workflow



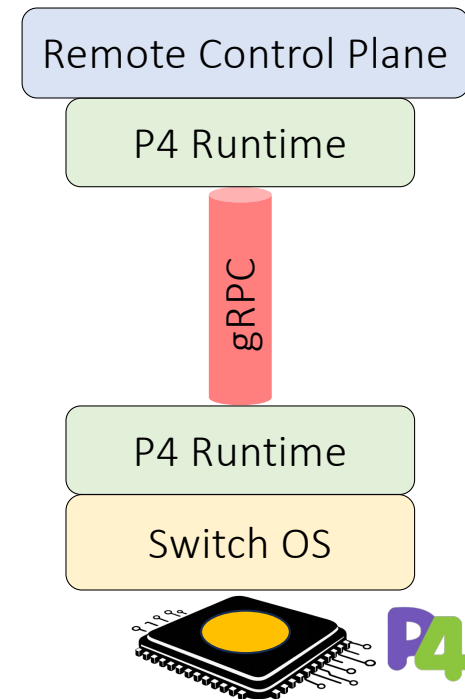
P4runtime

- An open source project to remotely control P4 switches
- OpenFlow was target-independent but not protocol-independent
- P4runtime is both target-independent and protocol-independent (portable across vendor; if you change the P4 program the API does not change)



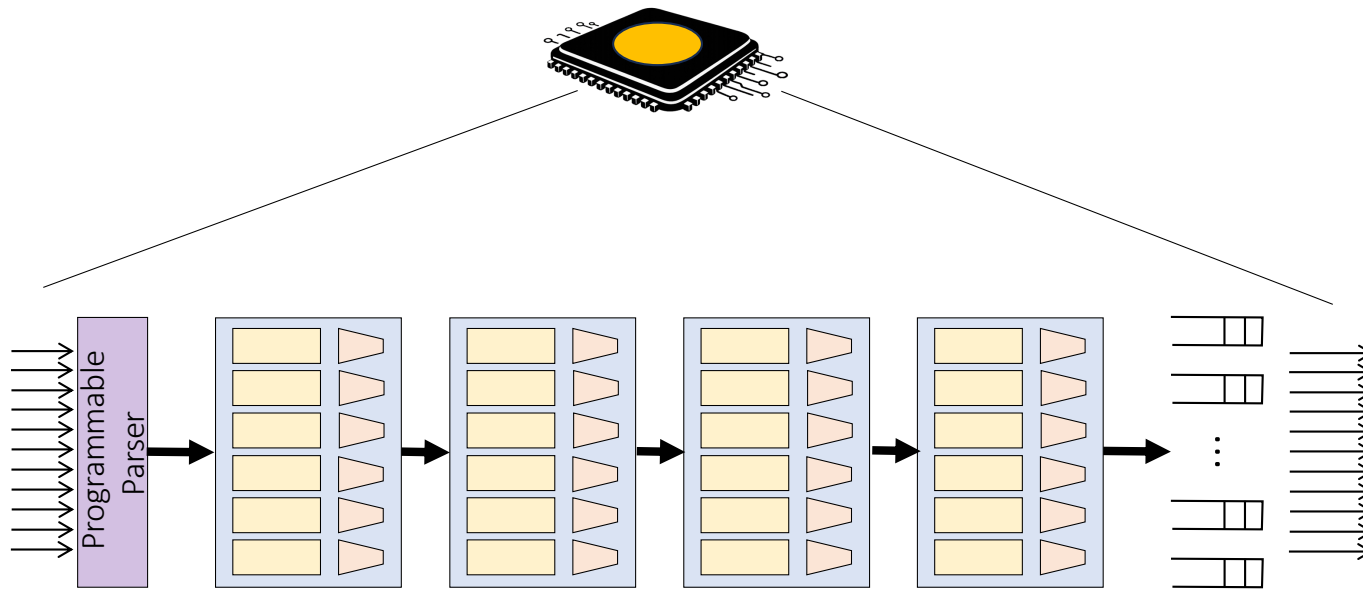
P4runtime

- An open source project to remotely control P4 switches
- OpenFlow was target-independent but not protocol-independent
- P4runtime is both target-independent and protocol-independent (portable across vendor; if you change the P4 program the API does not change)

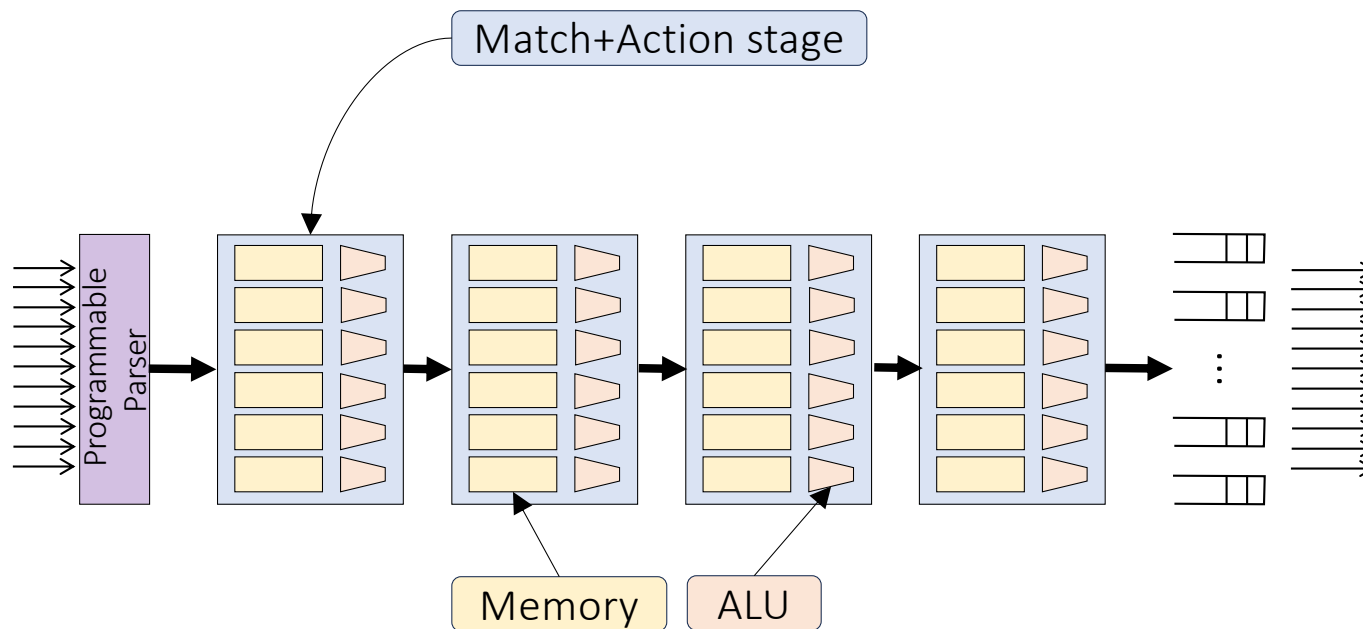


Here we will focus more on the architecture of a programmable switch and the P4 language

Protocol Independent Switch Architecture

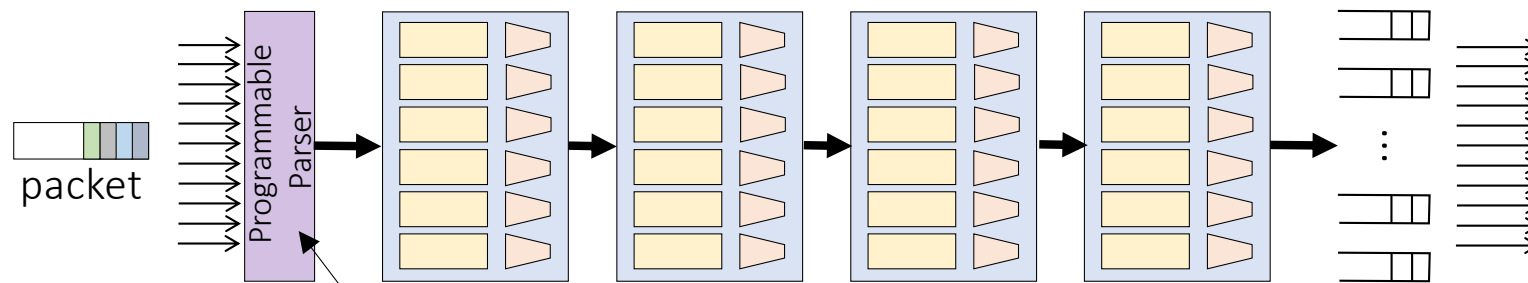


Protocol Independent Switch Architecture



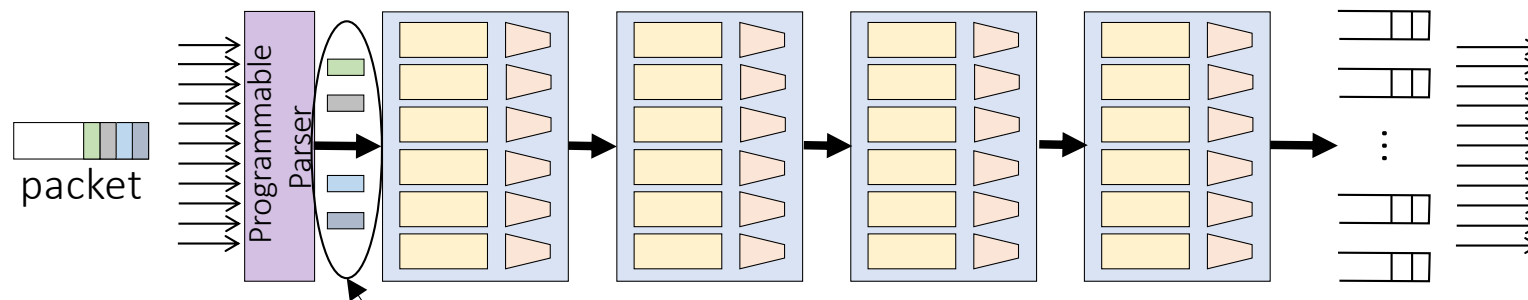
There is an identical set of pipeline stages. Each of them is designed to perform a set of match+action operations that can be done in parallel

Protocol Independent Switch Architecture



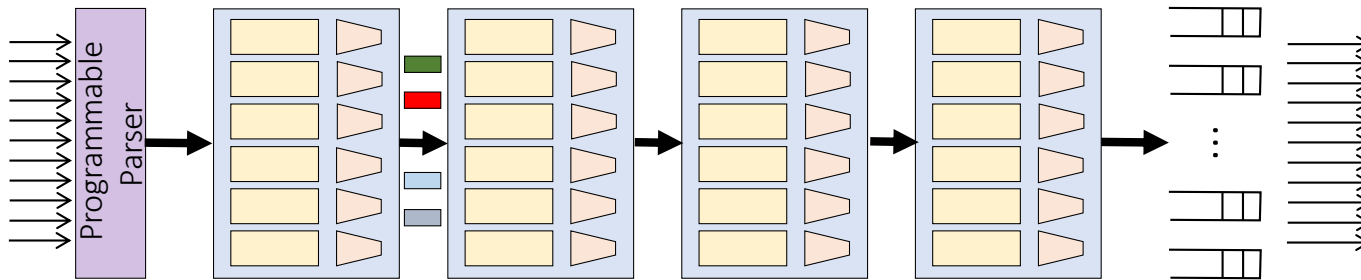
Extracts the headers from the incoming packet

Protocol Independent Switch Architecture



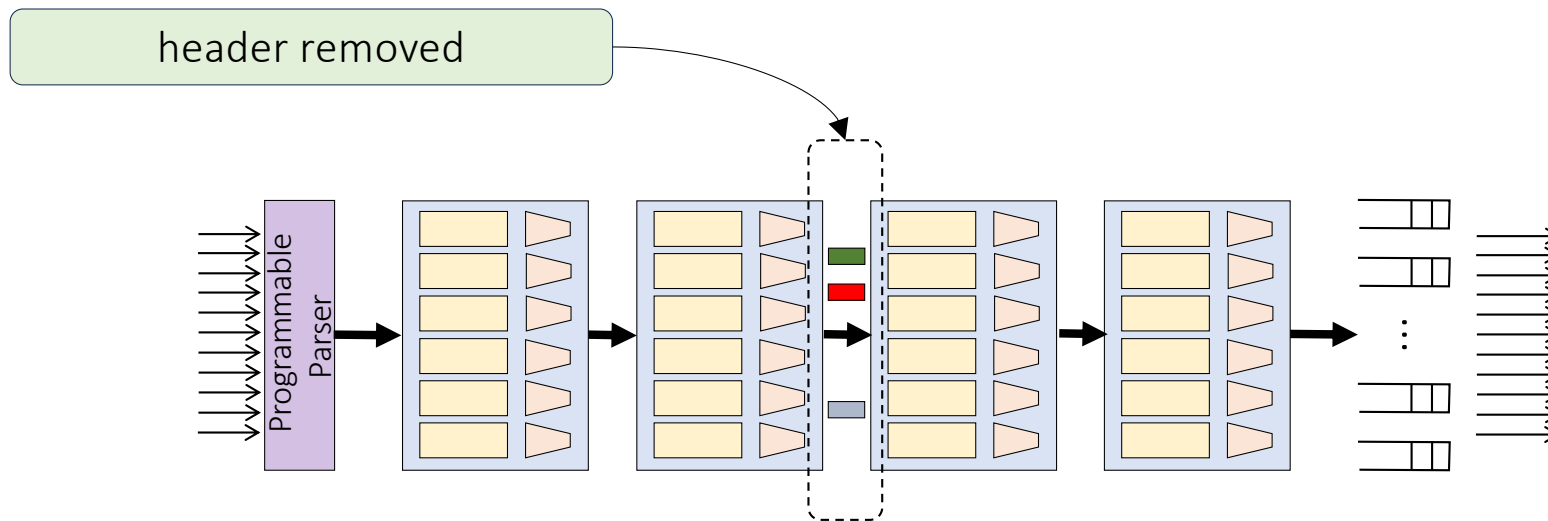
PHV (Packet Header Vector) is used as container that host the match keys for the Match+Action units

Protocol Independent Switch Architecture



Based on match entries you perform some actions
that can change, add, remove packet headers

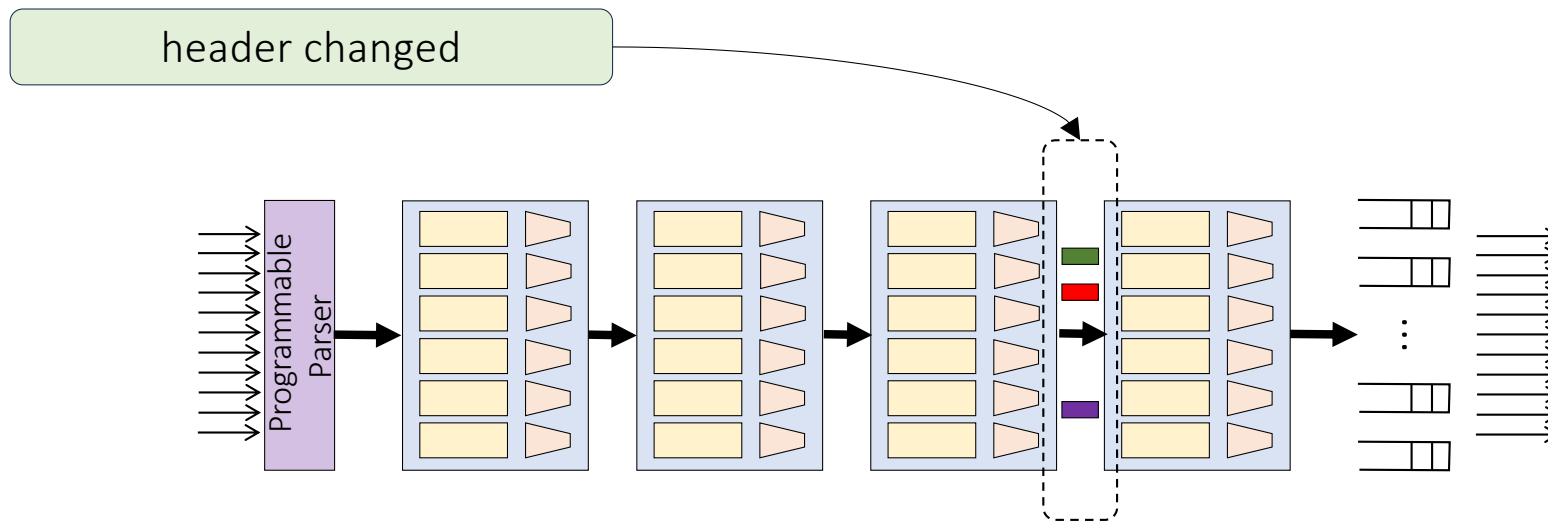
Protocol Independent Switch Architecture



Based on match entries you perform some actions that can change, add, remove packet headers

...the same thing repeats at every stage

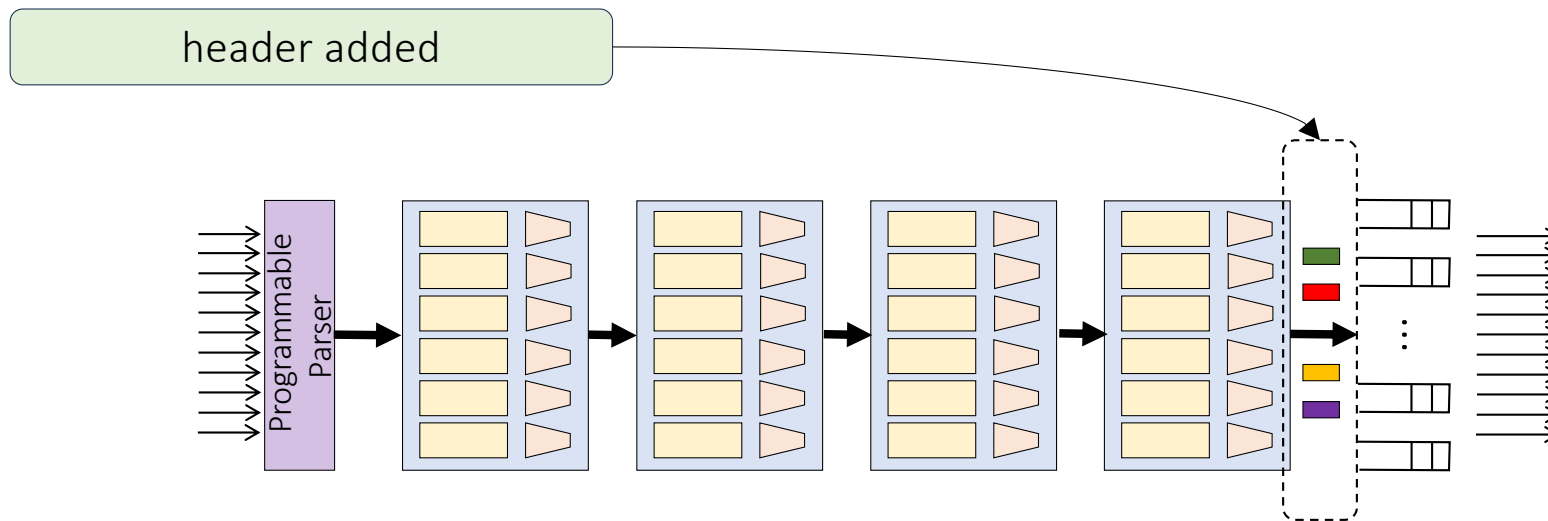
Protocol Independent Switch Architecture



Based on match entries you perform some actions that can change, add, remove packet headers

...the same thing repeats at every stage

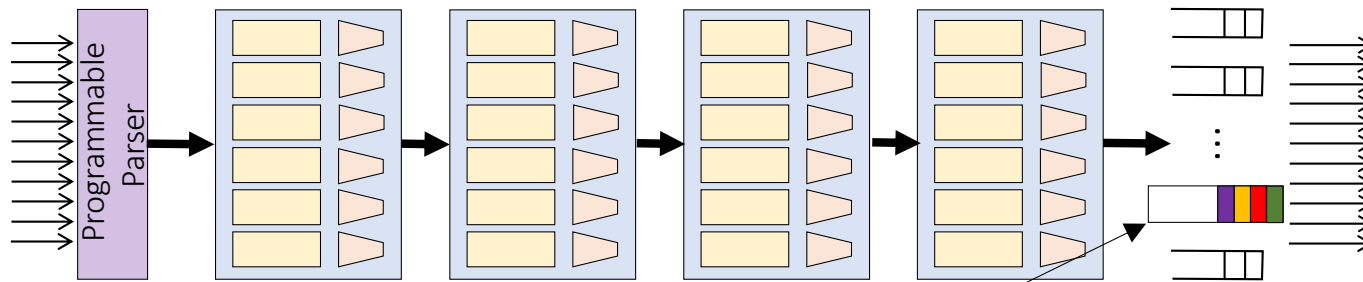
Protocol Independent Switch Architecture



Based on match entries you perform some actions that can change, add, remove packet headers

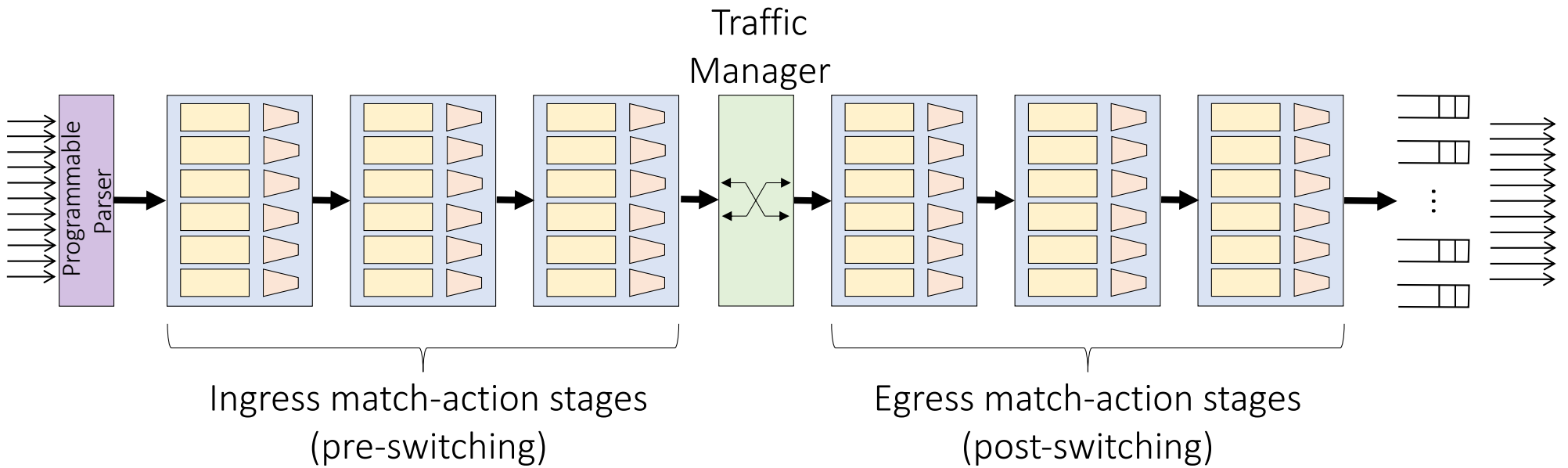
...the same thing repeats at every stage

Protocol Independent Switch Architecture

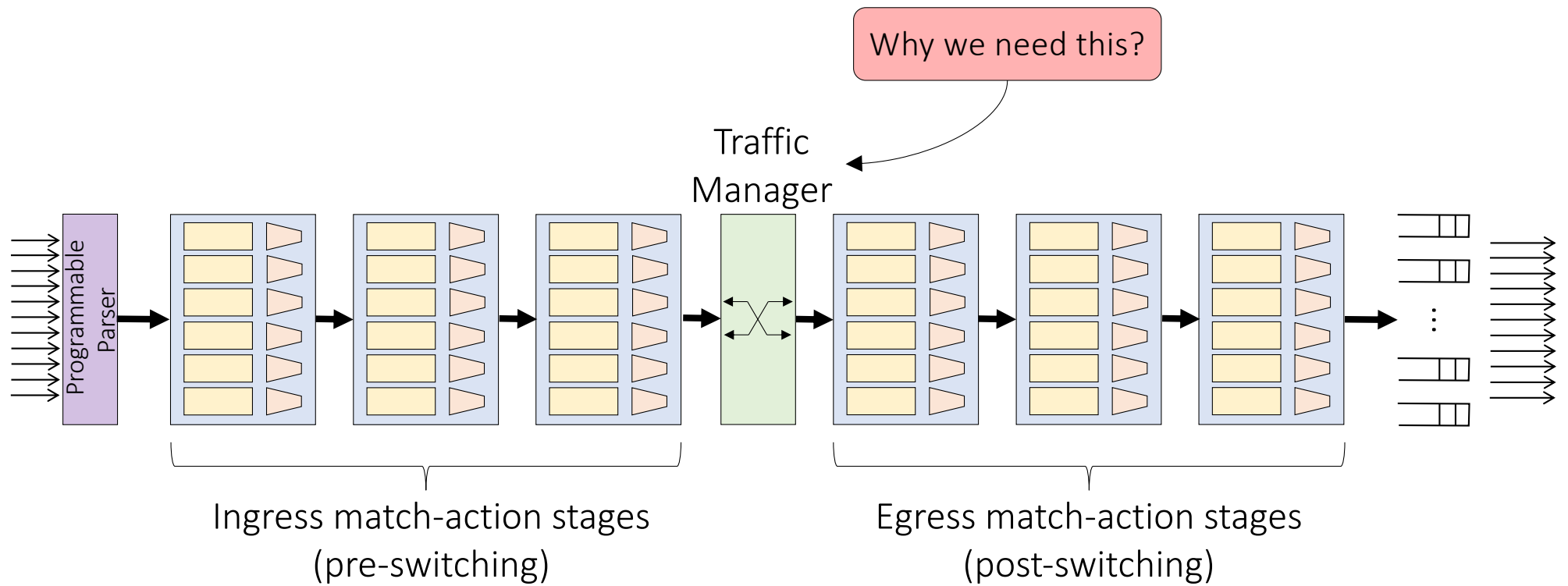


Finally, the packet gets reassembled in the *deparser*

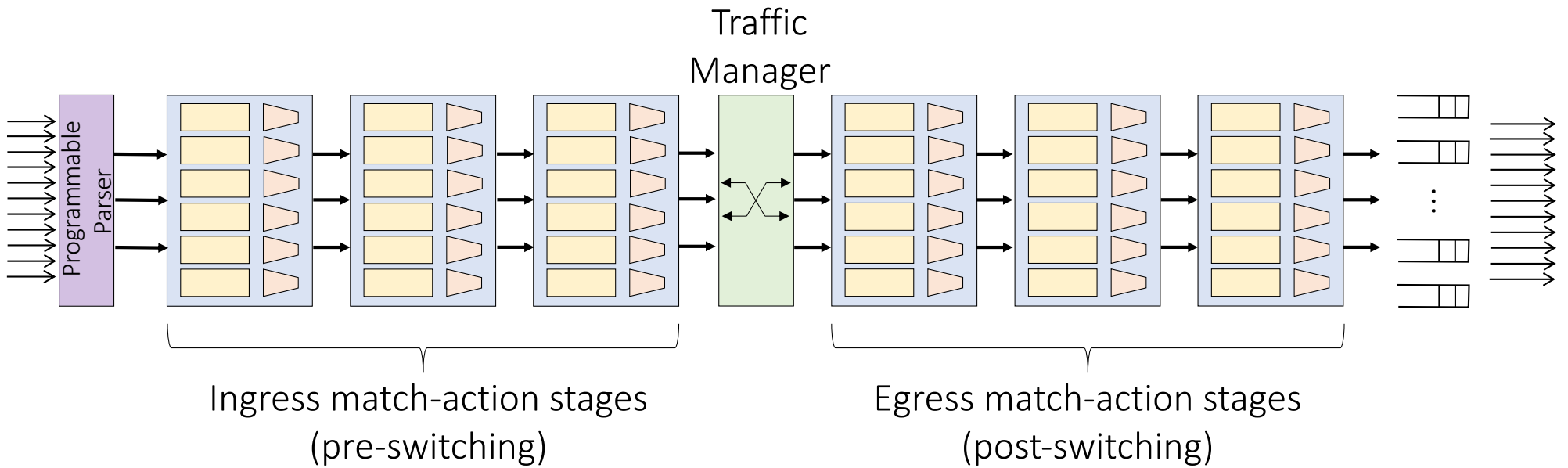
A more complete picture



A more complete picture

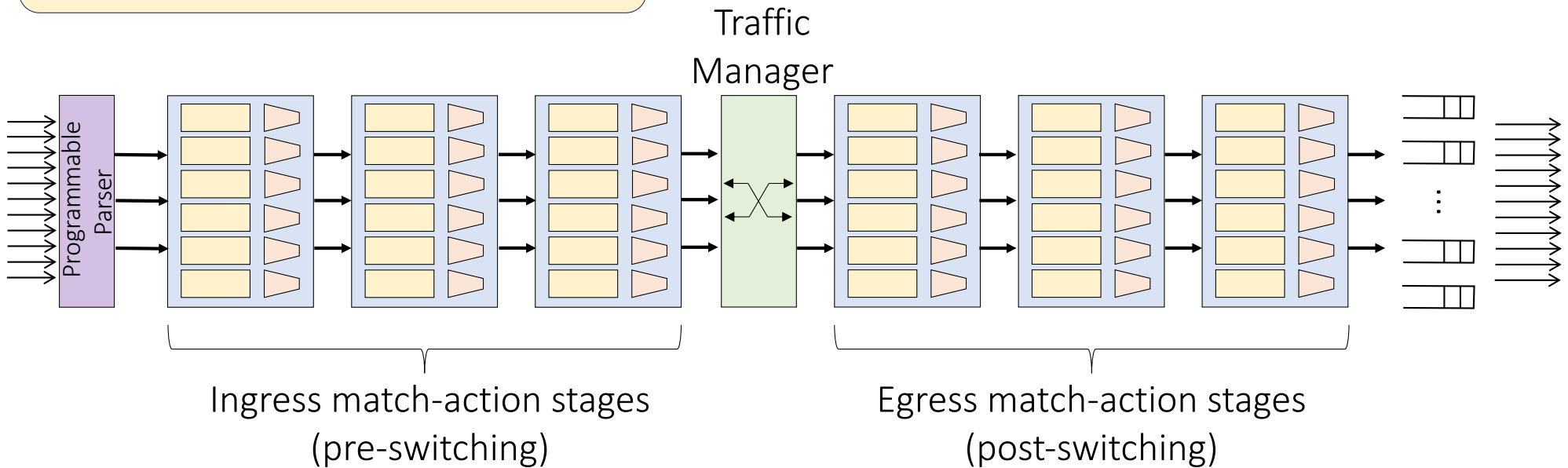


A more complete picture



A more complete picture

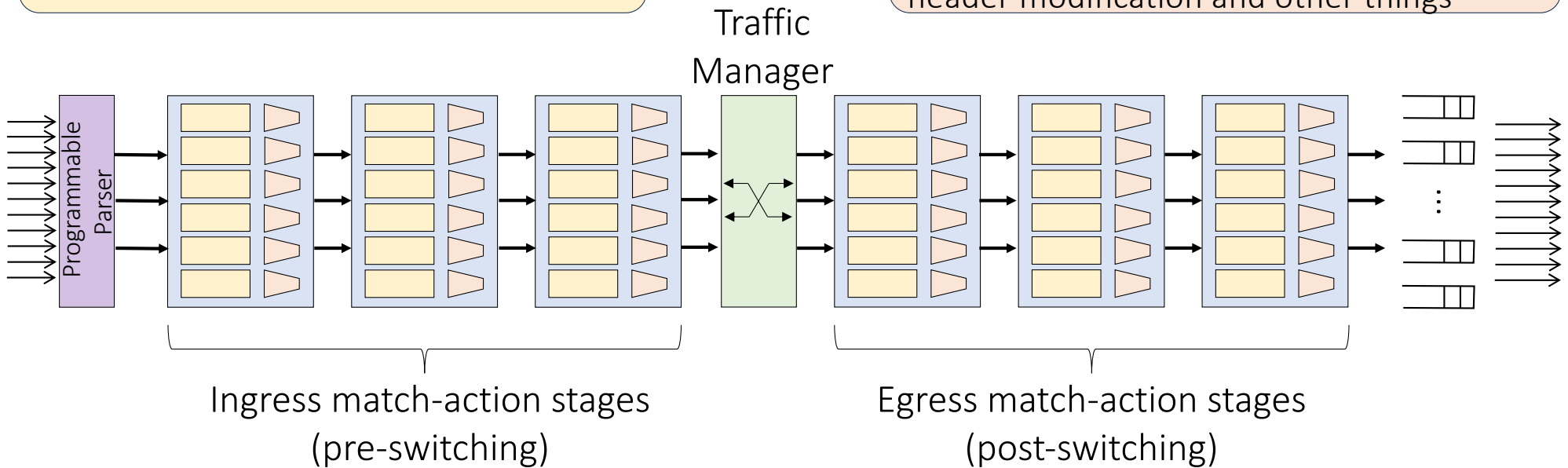
Match Logic: a mix of SRAM, TCAM for lookup tables and other things



A more complete picture

Match Logic: a mix of SRAM, TCAM for lookup tables and other things

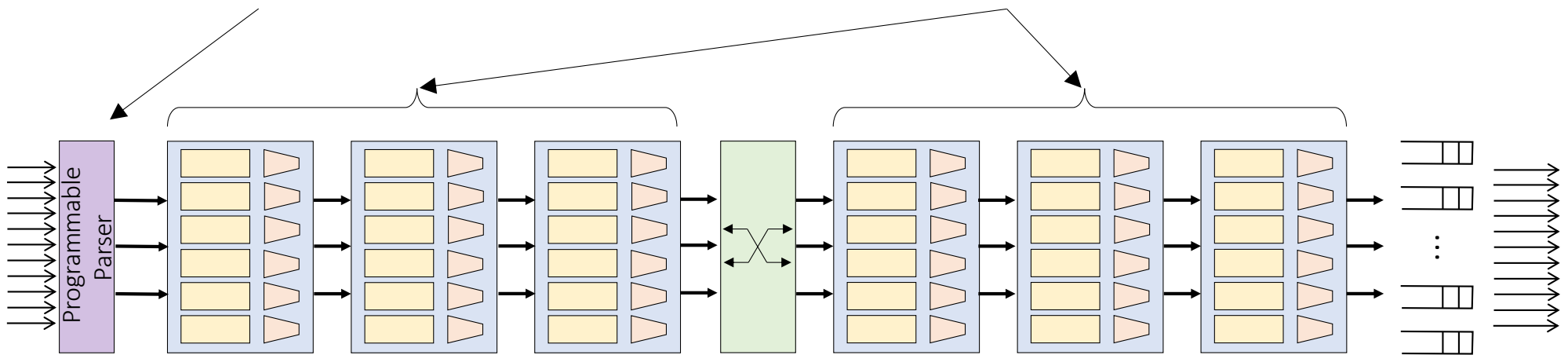
Action Logic: Arithmetic Logic Units (ALU) for boolean and arithmetic operations + header modification and other things



A more complete picture

Programmer declares which
headers are recognized

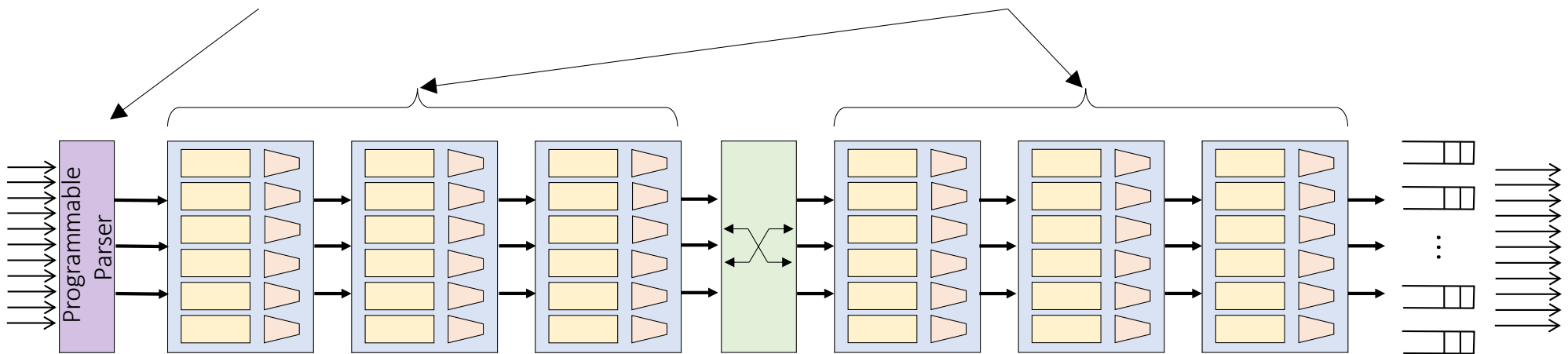
Programmer declares what
tables are needed and how packets are processed



A more complete picture

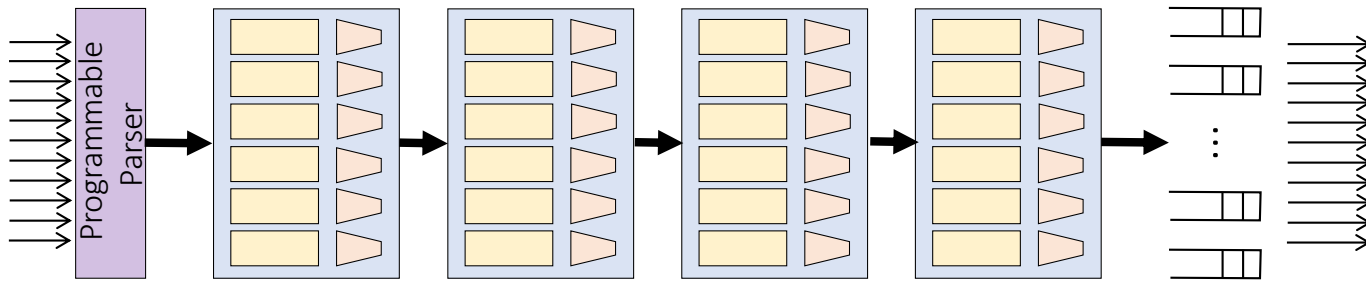
Programmer declares which
headers are recognized

Programmer declares what
tables are needed and how packets are processed

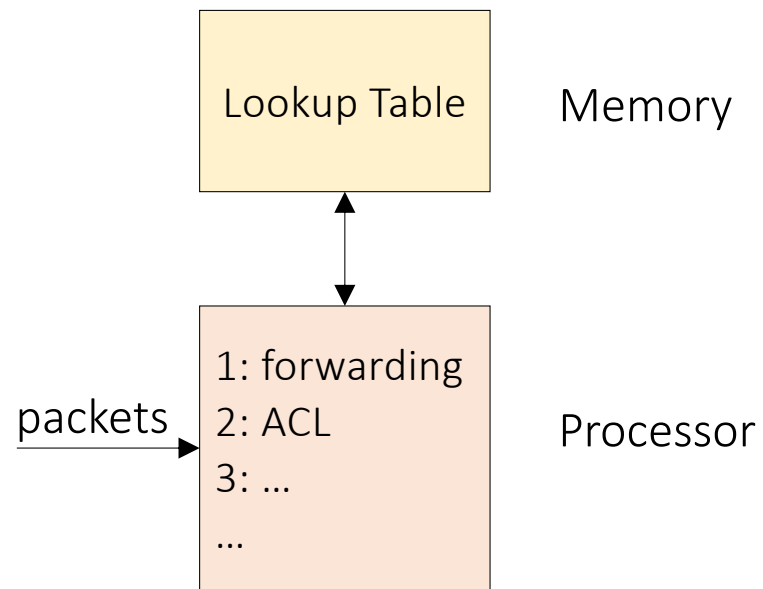


The PISA chip knows nothing at the beginning. Cannot parse packets nor process them. It is the programmer that decide.

Why multiple stages?

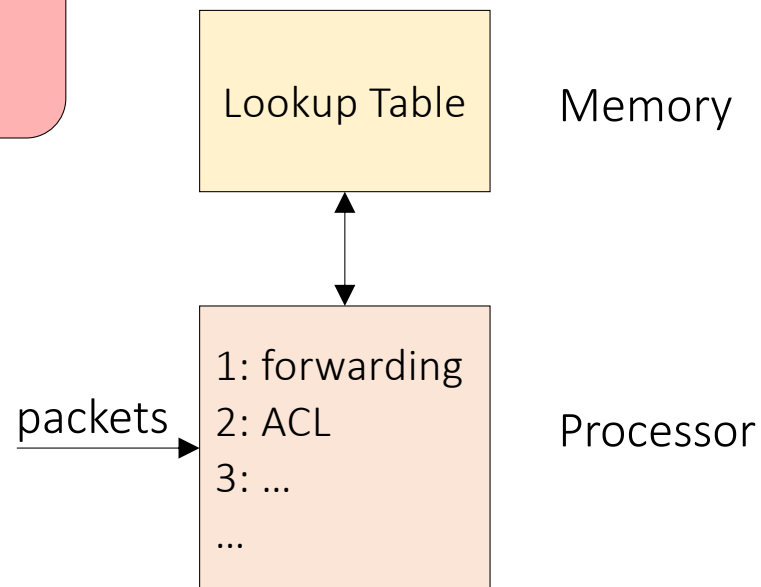


If we had a single processor..



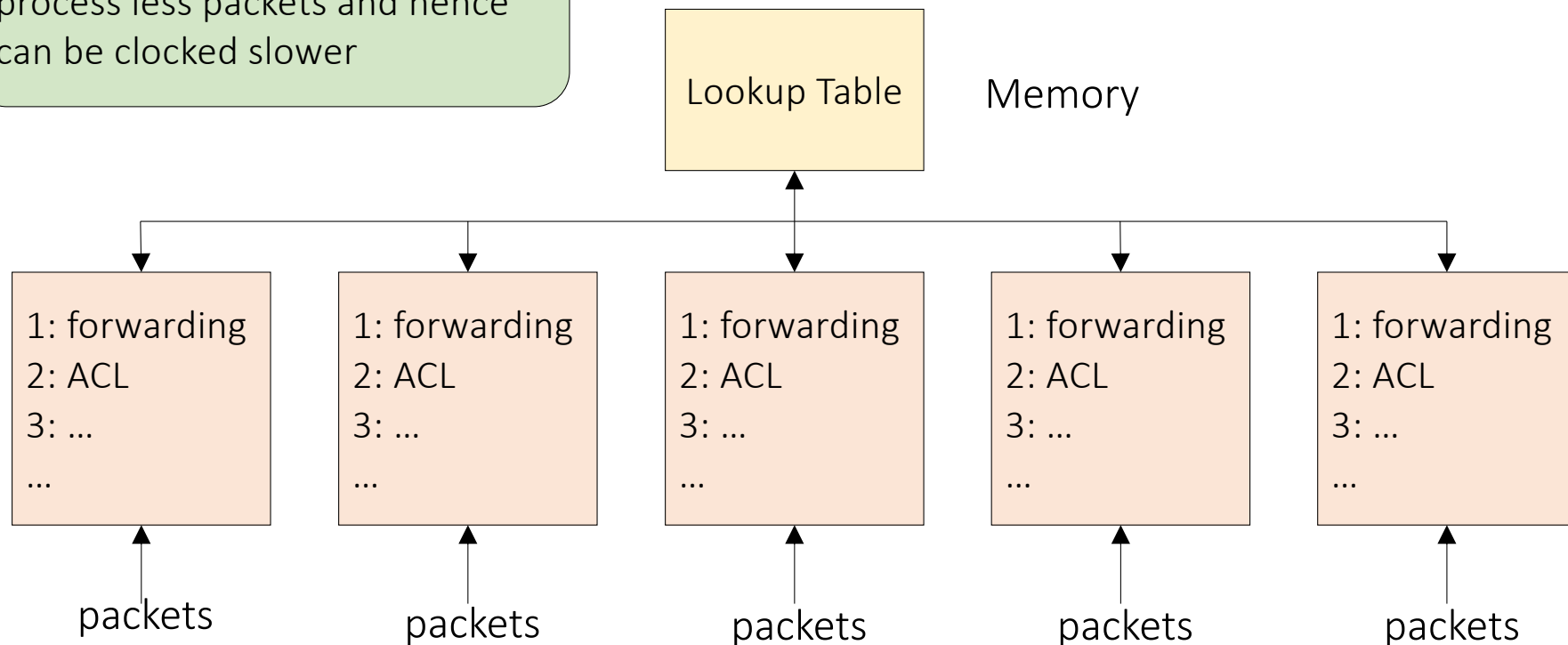
If we had a single processor..

To be able to process billions/sec packets, the frequency would need to be too high



What if we duplicate the processing units?

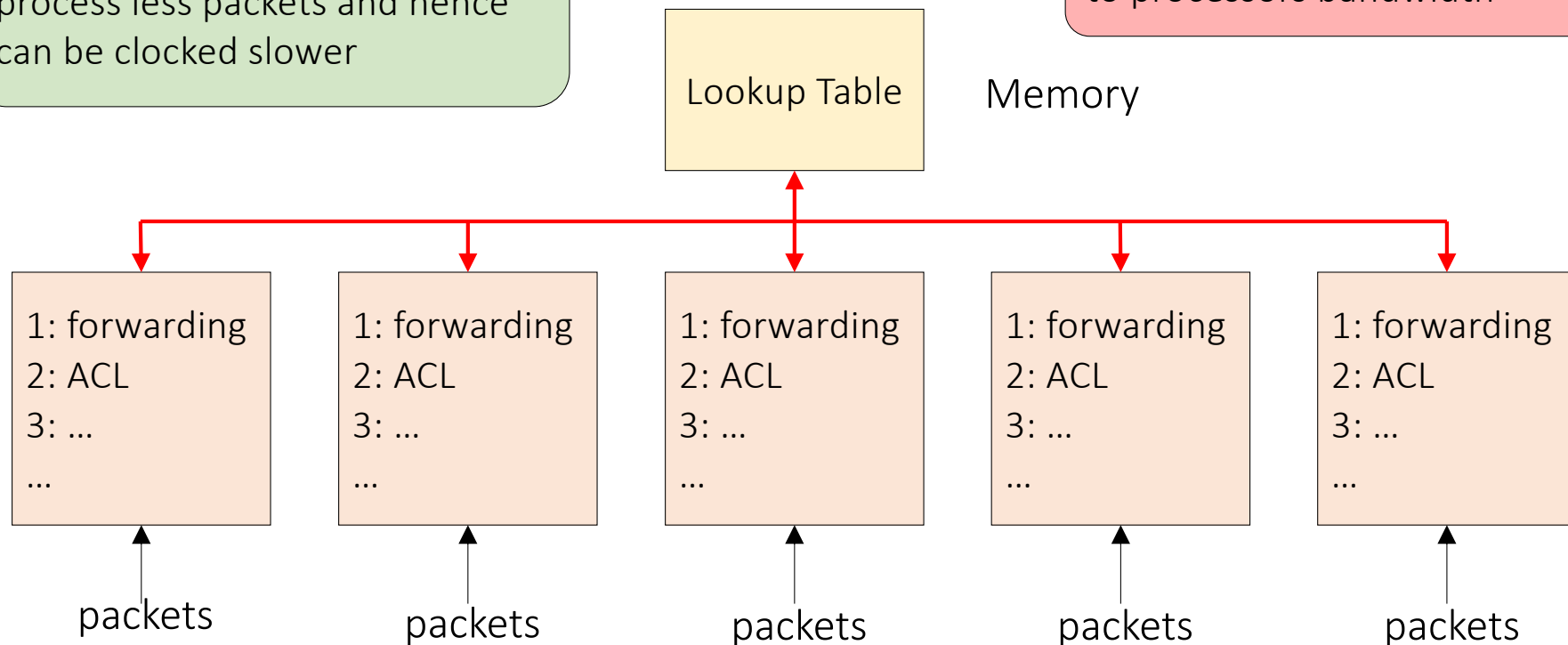
Each processing unit now must process less packets and hence can be clocked slower



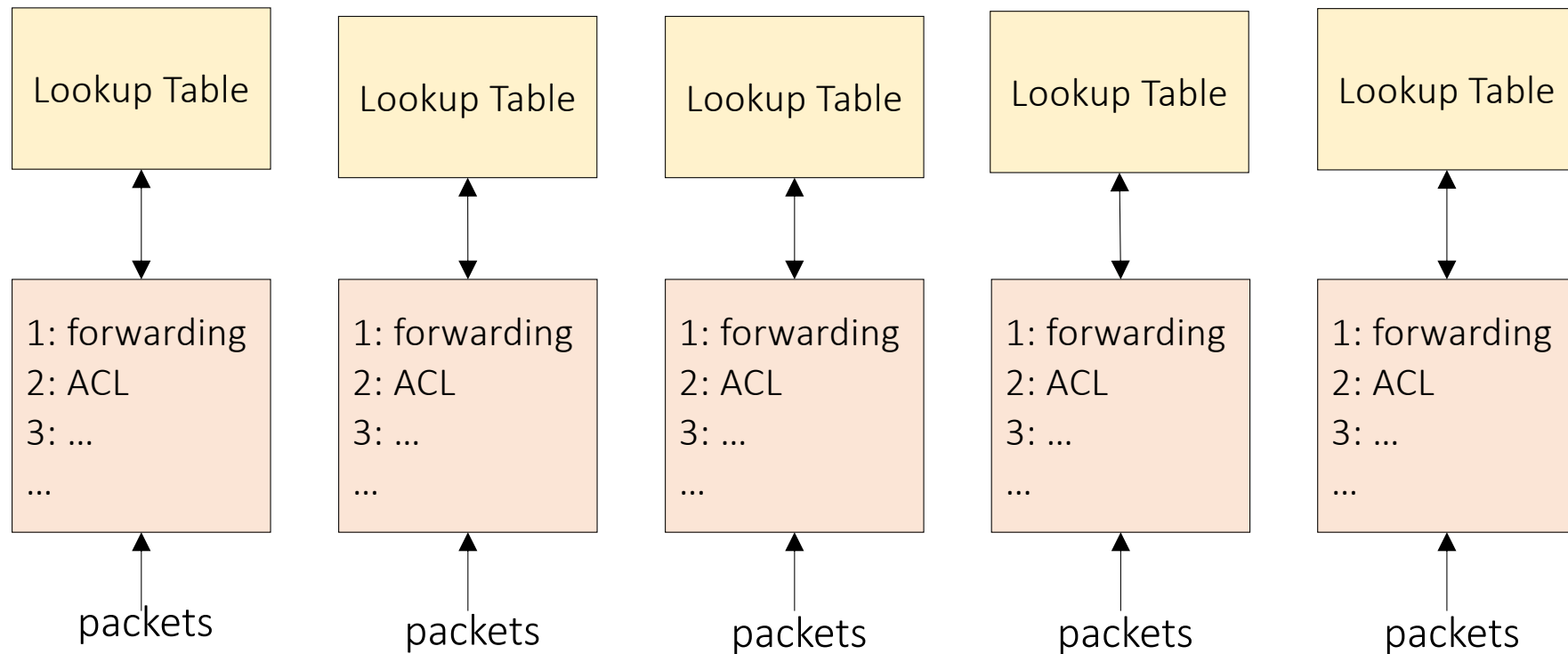
What if we duplicate the processing units?

Each processing unit now must process less packets and hence can be clocked slower

The new bottleneck is the memory to processors bandwidth

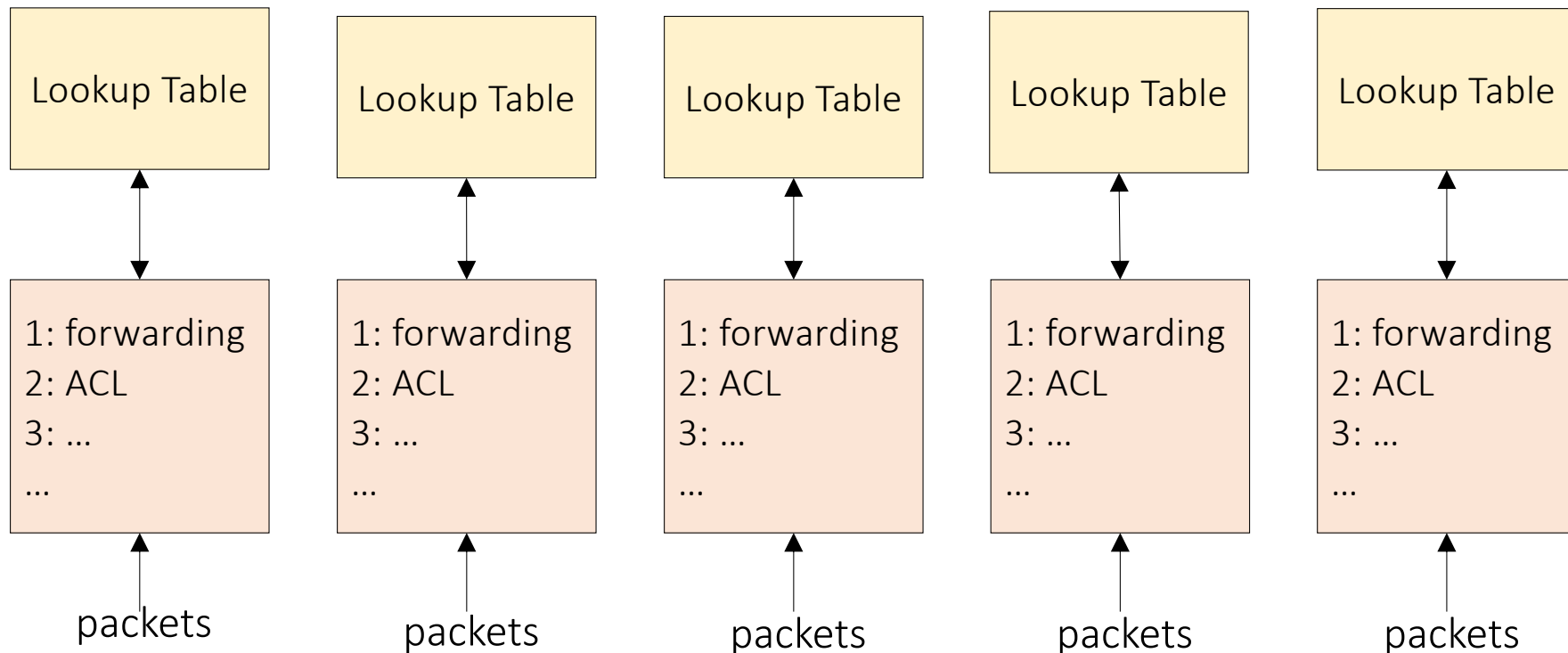


What if we replicate the memory?



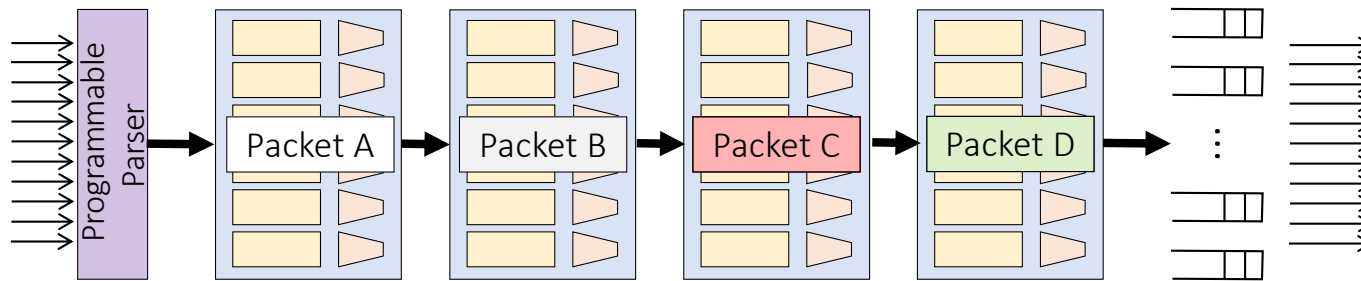
What if we replicate the memory?

This comes at a huge cost in terms of area and problems for memory coherency ☹



Why multiple stages?

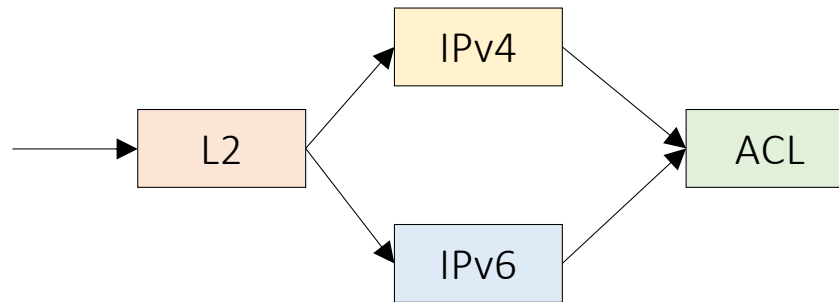
The ideal is to process packets in a pipelined fashion (this allows for high throughput)



- Each stage has fixed number of resources (in terms of memory and ALUs)
- It is possible to do multiple lookups on the same stage (operations on a packet), provided there is no dependency between them
- This is because you do not want to stall the pipeline!

So what does the compiler do?

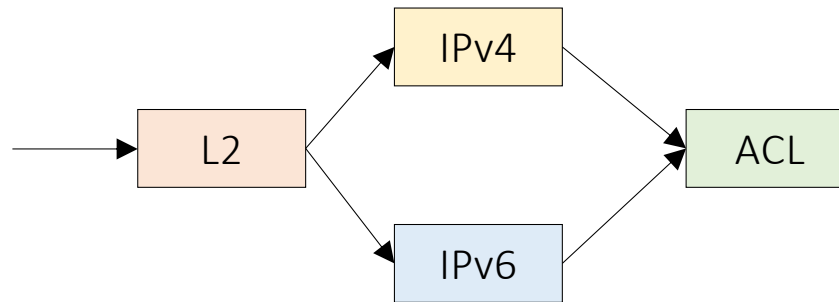
- The compiler maps your code into the pipeline. To do so, it needs to first perform a dependency analysis



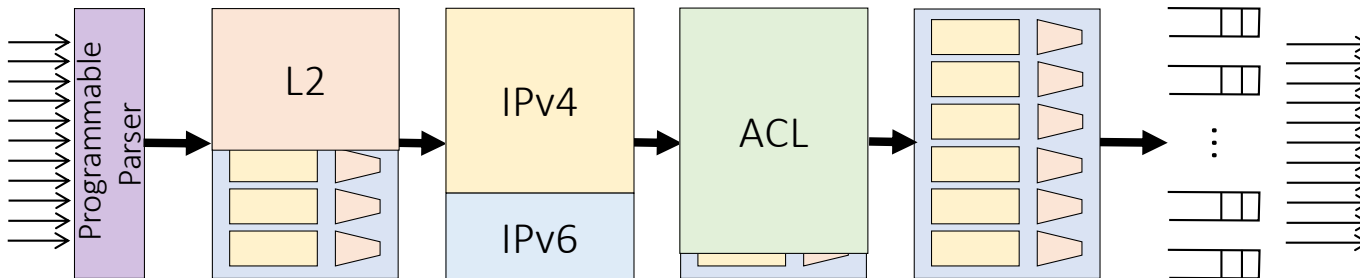
Let's assume this is a logical view of your data plane program written in **P4**

So what does the compiler do?

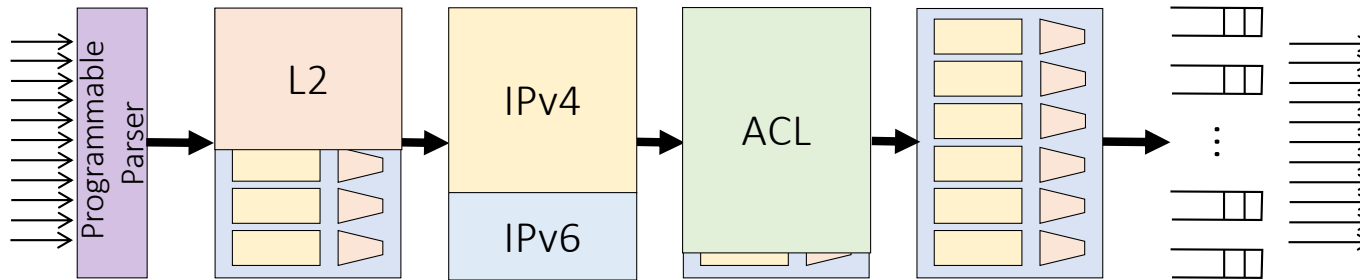
- The compiler maps your code into the pipeline. To do so, it needs to first perform a dependency analysis



Let's assume this is a logical view of your data plane program written in **P4**



So what does the compiler do?



- The placement depends on both match and action dependency

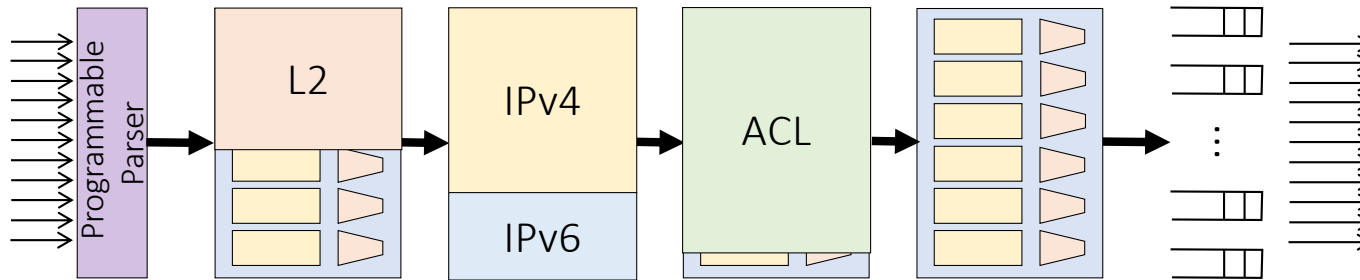
$M_1(pkt.header.field1) \rightarrow A_1(pkt.metadata.fieldA)$

$M_2(pkt.metadata.fieldA) \rightarrow A_2(pkt.header.field2)$



This is a match dependency

So what does the compiler do?



- The placement depends on both match and action dependency

$M_1(pkt.header.field1) \rightarrow A_1(pkt.metadata.fieldA)$

$M_2(pkt.metadata.fieldA) \rightarrow A_2(pkt.header.field2)$



This is a match dependency

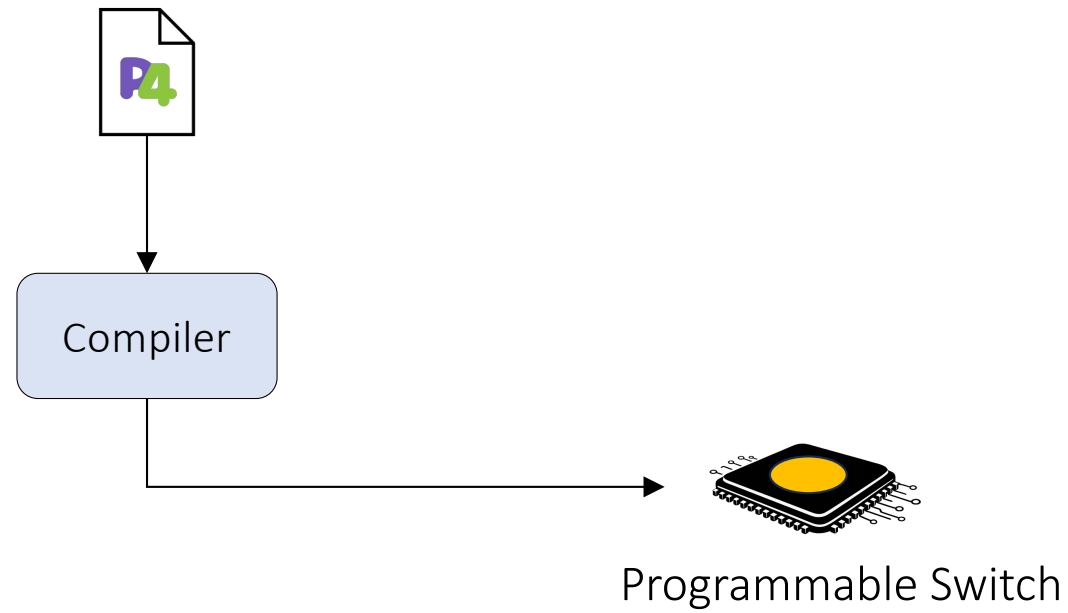
$M_1(pkt.header.field1) \rightarrow A_1(pkt.metadata.drop)$

$M_2(pkt.header.field2) \rightarrow A_2(pkt.metadata.drop)$

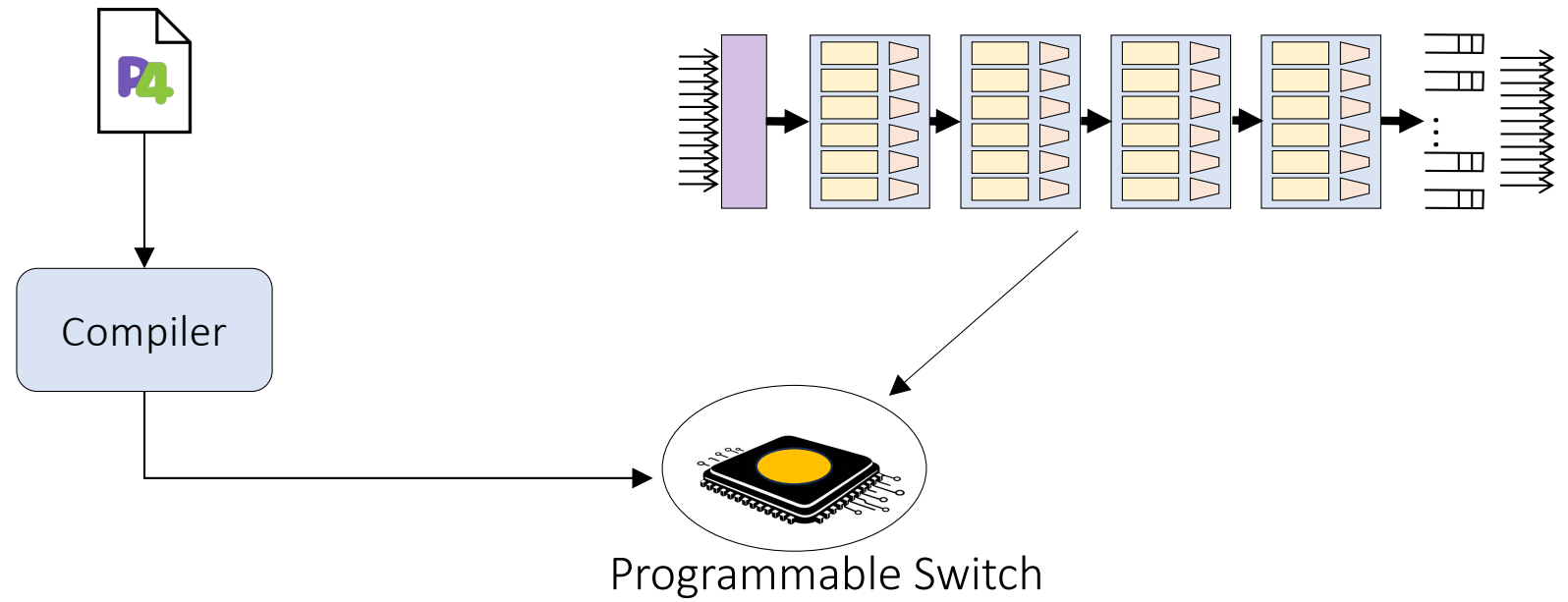


This is an action dependency

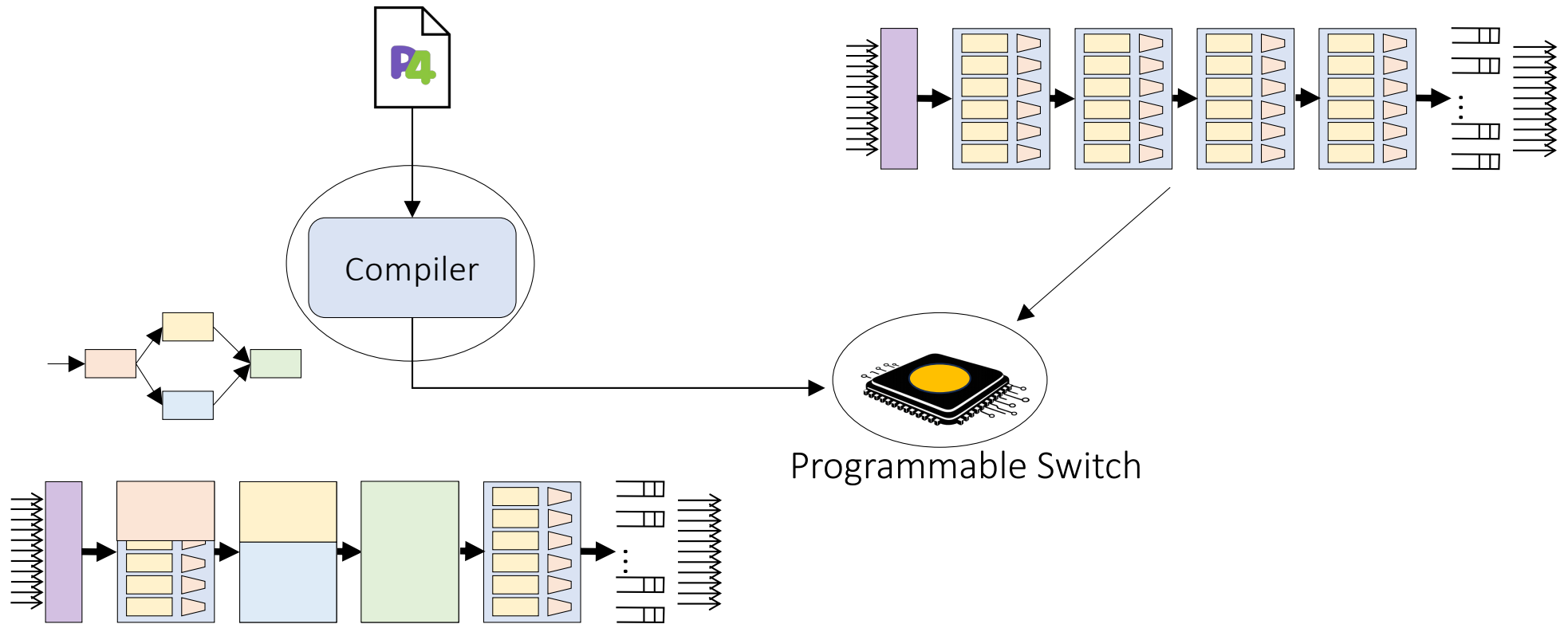
Quick recap



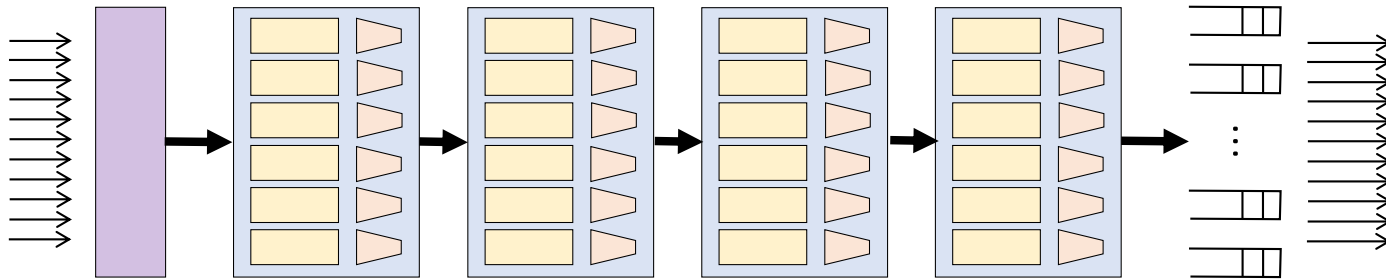
Quick recap



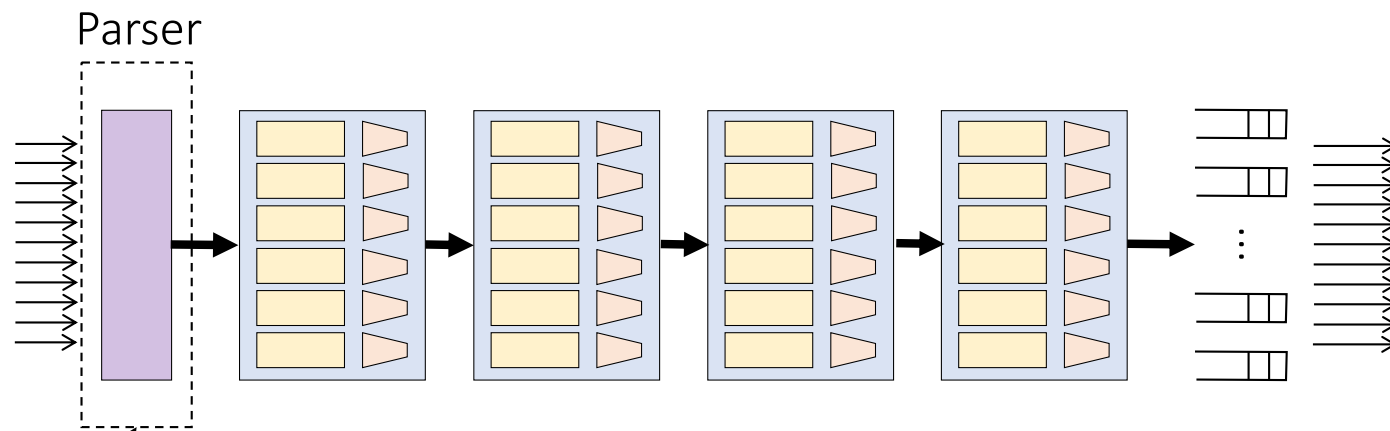
Quick recap



Recap on PISA

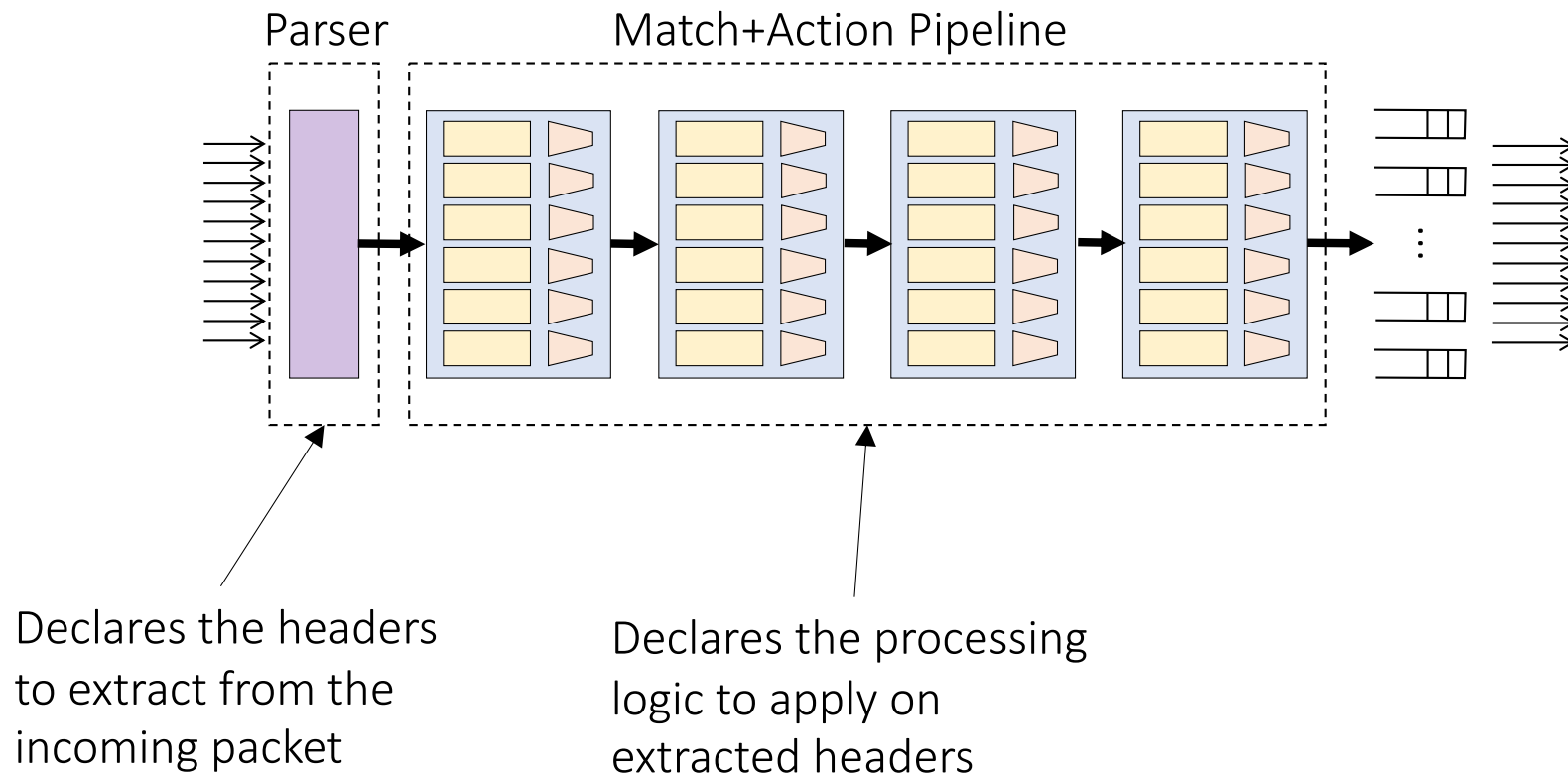


Recap on PISA

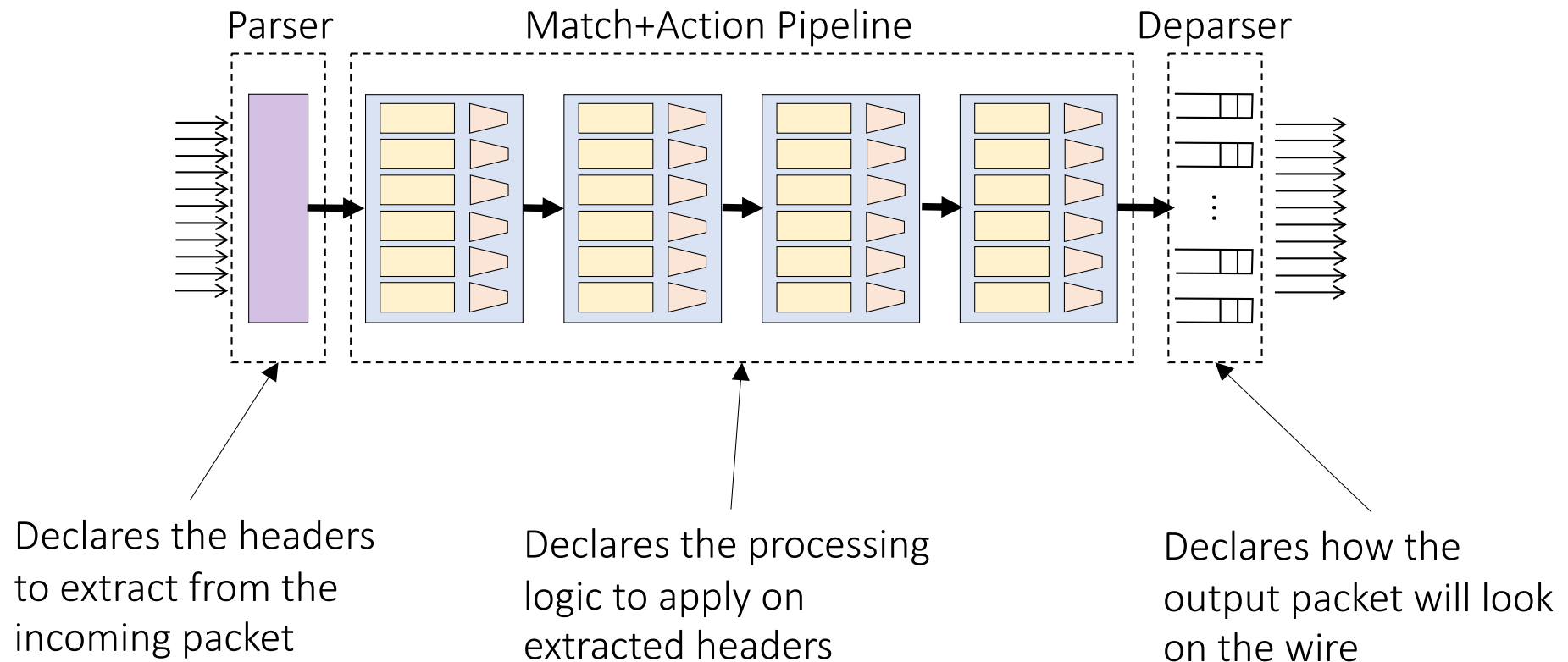


Declares the headers
to extract from the
incoming packet

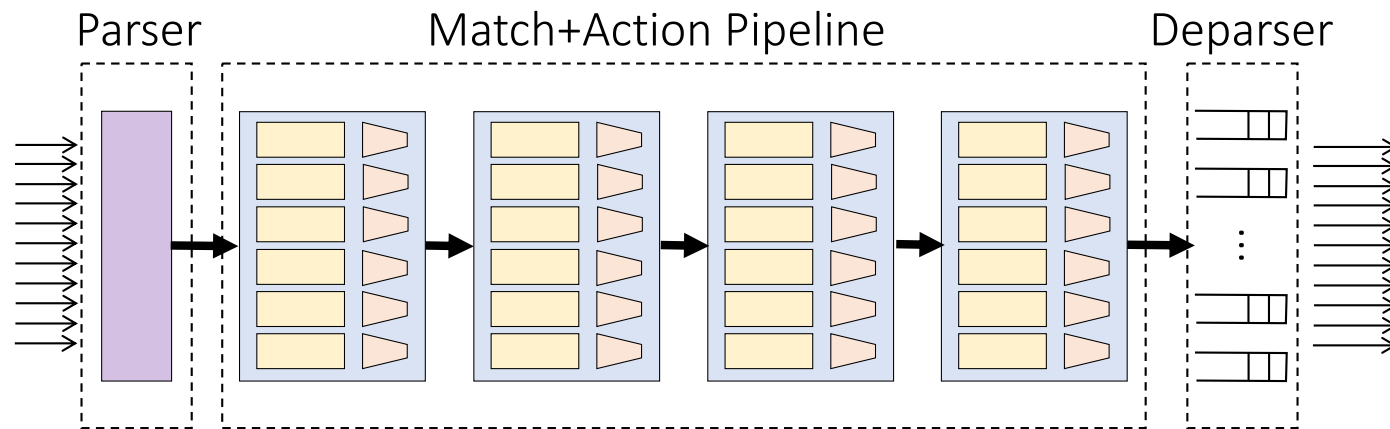
Recap on PISA



Recap on PISA

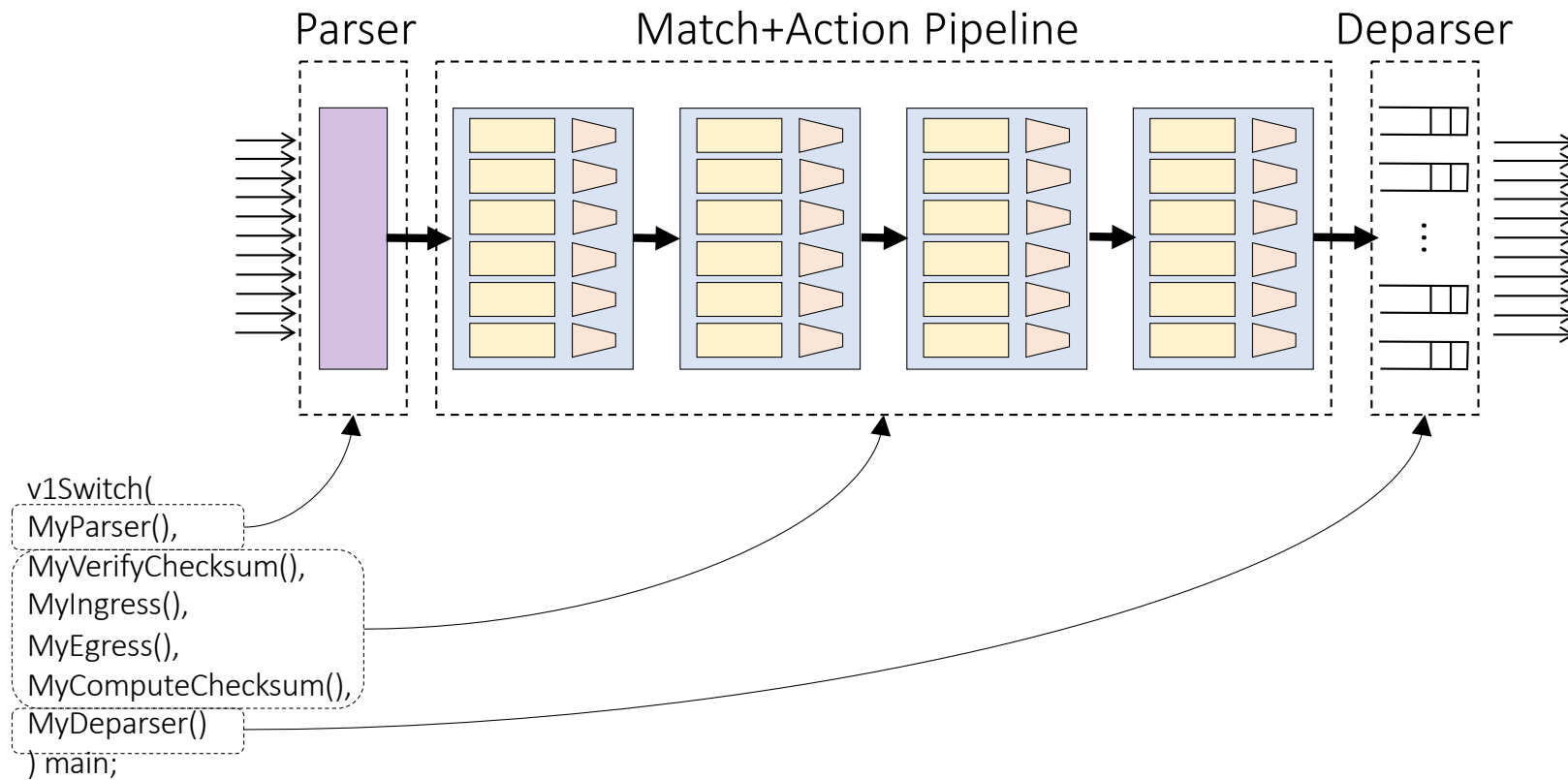


P4 primer



```
v1Switch(  
  MyParser(),  
  MyVerifyChecksum(),  
  MyIngress(),  
  MyEgress(),  
  MyComputeChecksum(),  
  MyDeparser()  
) main;
```

P4 primer



Packet reflector in P4

- Let's analyze a simple P4 code of a packet reflector
- The reflector simply send back the packets received to the same port they arrived and swap the MAC addresses

Packet reflector in P4

```
#include <core.p4>
#include <v1model.p4>

/*****
***** HEADERS *****/

typedef bit<48> macAddr_t;

header ethernet_t {
    macAddr_t dstAddr;
    macAddr_t srcAddr;
    bit<16>  etherType;
}

struct metadata {
    /* empty */
}

struct headers {
    ethernet_t  ethernet;
}
```

Librarians

Declarations

```
#include <core.p4>
#include <v1model.p4>

/*****
***** HEADERS *****/

typedef bit<48> macAddr_t;

header ethernet_t {
    macAddr_t dstAddr;
    macAddr_t srcAddr;
    bit<16>  etherType;
}

struct metadata {
    /* empty */
}

struct headers {
    ethernet_t  ethernet;
}

// ===== P A R S E R =====

parser MyParser(packet_in packet,
                out Headers hdr,
                inout metadata meta,
                inout standard_metadata_t standard_metadata) {

    state start {
        packet.extract(hdr.ethernet);
        transition accept;
    }

    // ===== CHECKSUM VERIFICATION =====

    control MyVerifyChecksum(inout headers hdr, inout metadata meta) {
        apply { }
    }

    // ===== INGRESS PROCESSING =====

    control MyIngress(inout headers hdr,
                     inout metadata meta,
                     inout standard_metadata_t standard_metadata) {

        action swap_mac {
            macAddr_t tmp;
            tmp = hdr.ethernet.srcAddr;
            hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;
            hdr.ethernet.dstAddr = tmp;
        }

        apply {
            // Swap MAC addresses.
            swap_mac();

            // Set output port on input port
            standard_metadata.egress_spec = standard_metadata.ingress_port;
        }
    }

    // ===== EGRESS PROCESSING =====

    control MyEgress(inout headers hdr,
                    inout metadata meta,
                    inout standard_metadata_t standard_metadata) {
        apply { }
    }

    // ===== CHECKSUM COMPUTATION =====

    control MyComputeChecksum(inout headers hdr, inout metadata meta) {
        apply { }
    }

    // ===== DISPATCH =====

    control MyDispatcher(packet_out packet, in headers hdr) {
        apply {
            // parsed headers have to be added again into the packet
            packet.emit(hdr.ethernet);
        }
    }

    // ===== SWITCH =====

    V1Switch(
        MyParser(),
        MyVerifyChecksum(),
        MyIngress(),
        MyEgress(),
        MyComputeChecksum(),
        MyDispatcher()
    );
}
```

93

<https://github.com/nsg-ethz/p4-learning/blob/master/examples/reflector/reflector.p4>

Packet reflector in P4

```
#include <core.p4>
#include <v1model.p4>

/*****
***** H E A D E R S *****/

typedef bit<48> macAddr_t;

header ethernet_t {
    macAddr_t dstAddr;
    macAddr_t srcAddr;
    bit<16> etherType;
}

struct metadata {
    /* empty */
}

struct headers {
    ethernet_t ethernet;
}
```

Here, define the fields of the headers indicated in the “*struct headers*”

```
#include <core.p4>
#include <v1model.p4>

/*****
***** H E A D E R S *****/

typedef bit<48> macAddr_t;

header ethernet_t {
    macAddr_t dstAddr;
    macAddr_t srcAddr;
    bit<16> etherType;
}

struct metadata {
    /* empty */
}

struct headers {
    ethernet_t ethernet;
}

// ===== P A R S E R =====
parser MyParser(packet_in packet,
                out Headers hdr,
                inout metadata meta,
                inout standard_metadata_t standard_metadata) {

    state start {
        packet.extract(hdr.ethernet);
        transition accept;
    }

    // ===== CHECKSUM VERIFICATION =====
    control MyVerifyChecksum(inout headers hdr, inout metadata meta) {
        apply { }
    }

    // ===== INGRESS PROCESSING =====
    control MyIngress(inout headers hdr,
                     inout metadata meta,
                     inout standard_metadata_t standard_metadata) {

        action swap_mac {
            macAddr_t tmp;
            tmp = hdr.ethernet.dstAddr;
            hdr.ethernet.dstAddr = hdr.ethernet.srcAddr;
            hdr.ethernet.srcAddr = tmp;
        }

        apply {
            // Swap MAC addresses.
            swap_mac();

            // Set output port on input port
            standard_metadata.egress_spec = standard_metadata.ingress_port;
        }

    }

    // ===== EGRESS PROCESSING =====
    control MyEgress(inout headers hdr,
                    inout metadata meta,
                    inout standard_metadata_t standard_metadata) {
        apply { }
    }

    // ===== CHECKSUM CORRECTION =====
    control MyComputeChecksum(inout headers hdr, inout metadata meta) {
        apply { }
    }

    // ===== DISPATCH =====
    control MyDispatcher(packet_out packet, in headers hdr) {
        apply {
            // parsed headers have to be added again into the packet
            packet.emit(hdr.ethernet);
        }
    }

    // ===== SWITCH =====
    V1Switch(
        MyParser(),
        MyVerifyChecksum(),
        MyIngress(),
        MyEgress(),
        MyComputeChecksum(),
        MyDispatcher()
    );
}

int main();
```

<https://github.com/nsg-ethz/p4-learning/blob/master/examples/reflector/reflector.p4>

Packet reflector in P4

```
#include <core.p4>
#include <v1model.p4>

/*****
***** HEADERS *****/

typedef bit<48> macAddr_t;

header ethernet_t {
    macAddr_t dstAddr;
    macAddr_t srcAddr;
    bit<16>  etherType;
}

struct metadata {
    /* empty */
}

struct headers {
    ethernet_t  ethernet;
}
```

Here, define the fields of the headers indicated in the “*struct headers*”

Here define the list of headers we care

```
#include <core.p4>
#include <v1model.p4>

/*****
***** HEADERS *****/

typedef bit<48> macAddr_t;

header ethernet_t {
    macAddr_t dstAddr;
    macAddr_t srcAddr;
    bit<16>  etherType;
}

struct metadata {
    /* empty */
}

struct headers {
    ethernet_t  ethernet;
}

// ===== P A R S E R =====
parser MyParser(packet_in packet,
                Out Headers hdr,
                Inout metadata meta,
                Inout standard_metadata_t standard_metadata) {

    state start {
        packet.extract(hdr.ethernet);
        transition accept;
    }

    // ===== CHECKSUM VERIFICATION =====
    control MyVerifyChecksum(Inout headers hdr, Inout metadata meta) {
        apply { }
    }

    // ===== INGRESS PROCESSING =====
    control MyIngress(Inout headers hdr,
                     Inout metadata meta,
                     Inout standard_metadata_t standard_metadata) {

        action swap_mac {
            macAddr_t tmp;
            tmp = hdr.ethernet.srcAddr;
            hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;
            hdr.ethernet.dstAddr = tmp;
        }

        apply {
            // Swap MAC addresses.
            swap_mac();

            // Set output port on input port
            standard_metadata.egress_spec = standard_metadata.ingress_port;
        }

    }

    // ===== EGRESS PROCESSING =====
    control MyEgress(Inout headers hdr,
                    Inout metadata meta,
                    Inout standard_metadata_t standard_metadata) {
        apply { }
    }

    // ===== CHECKSUM CORRECTION =====
    control MyComputeChecksum(Inout headers hdr, Inout metadata meta) {
        apply { }
    }

    // ===== DISPATCH =====
    control MyDispatcher(packet_out packet, In headers hdr) {
        apply {
            // parsed headers have to be added again into the packet
            packet.emit(hdr.ethernet);
        }
    }

    // ===== SWITCH =====
    V1Switch(
        MyParser(),
        MyVerifyChecksum(),
        MyIngress(),
        MyEgress(),
        MyComputeChecksum(),
        MyDispatcher()
    );
}
```

<https://github.com/nsg-ethz/p4-learning/blob/master/examples/reflector/reflector.p4>

Packet reflector in P4

```
/******  
***** P A R S E R *****  
*****/  
  
parser MyParser(packet_in packet,  
    out headers hdr,  
    inout metadata meta,  
    inout standard_metadata_t standard_metadata) {  
  
    state start{  
        packet.extract(hdr.ethernet);  
        transition accept;  
    }  
  
}  
  
/******  
***** C H E C K S U M   V E R I F I C A T I O N *****  
*****/  
  
control MyVerifyChecksum(inout headers hdr, inout metadata meta) {  
    apply { }  
}  
  
/******  
***** C H E C K S U M   V E R I F I C A T I O N *****  
*****/  
  
control MyVerifyChecksum(inout headers hdr, inout metadata meta) {  
    apply { }  
}
```

Extract packet
headers

```
#include <core.p4>  
#include <myModel.p4>  
  
//*****  
//***** P A R S E R *****  
//*****  
  
typedef bit[48] macAddr_t;  
  
header ethernet_t {  
    macAddr_t dstAddr;  
    macAddr_t srcAddr;  
    bit[16] etherType;  
}  
  
struct metadata_t {  
    // empty w/  
}  
  
struct headers_t {  
    ethernet_t ethernet;  
}  
  
//*****  
//***** P A R S E R *****  
//*****  
  
parser MyParser(packet_in packet,  
    out Headers hdr,  
    inout metadata meta,  
    inout standard_metadata_t standard_metadata) {  
  
    state start{  
        packet.extract(hdr.ethernet);  
        transition accept;  
    }  
  
//*****  
//***** C H E C K S U M   V E R I F I C A T I O N *****  
//*****  
  
control MyVerifyChecksum(inout headers hdr, inout metadata meta) {  
    apply { }  
}  
  
//*****  
//***** I N G R E S S   P R O C E S S I N G *****  
//*****  
  
control MyIngress(inout headers hdr,  
    inout metadata meta,  
    inout standard_metadata_t standard_metadata) {  
  
    action swap_mac{  
        macAddr_t tmp;  
        tmp = hdr.ethernet.srcAddr;  
        hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;  
        hdr.ethernet.dstAddr = tmp;  
    }  
  
    apply {  
        // Swap MAC addresses.  
        swap_mac();  
    }  
  
    //Set output port on input port  
    standard_metadata.egress_spec = standard_metadata.ingress_port;  
}  
  
//*****  
//***** E G R E S S   P R O C E S S I N G *****  
//*****  
  
control MyEgress(inout headers hdr,  
    inout metadata meta,  
    inout standard_metadata_t standard_metadata) {  
    apply { }  
}  
  
//*****  
//***** C H E C K S U M   C O M P A R I S O N *****  
//*****  
  
control MyCompareChecksum(inout headers hdr, inout metadata meta) {  
    apply { }  
}  
  
//*****  
//***** D E F A U L T *****  
//*****  
  
control MyDefault(packet_out packet, in headers hdr) {  
    apply {  
        // parsed headers have to be added again into the packet  
        packet.set(hdr.ethernet);  
    }  
}  
  
//*****  
//***** S W I T C H *****  
//*****  
  
#include <MyParser.p4>  
#include <MyVerifyChecksum.p4>  
#include <MyIngress.p4>  
#include <MyEgress.p4>  
#include <MyCompareChecksum.p4>  
#include <MyDefault.p4>  
  
int main()  
{  
    MyParser p;  
    MyVerifyChecksum v;  
    MyIngress i;  
    MyEgress e;  
    MyCompareChecksum c;  
    MyDefault d;  
}
```

<https://github.com/nsg-ethz/p4-learning/blob/master/examples/reflector/reflector.p4>

Packet reflector in P4

```
/******  
***** P A R S E R *****  
*****/  
  
parser MyParser(packet_in packet,  
    out headers hdr,  
    inout metadata meta,  
    inout standard_metadata_t standard_metadata) {  
  
    state start{  
        packet.extract(hdr.ethernet);  
        transition accept;  
    }  
  
}  
  
/******  
***** C H E C K S U M   V E R I F I C A T I O N *****  
*****/  
  
control MyVerifyChecksum(inout headers hdr, inout metadata meta) {  
    apply { }  
}  
  
/******  
***** C H E C K S U M   C O M P A R I S O N *****  
*****/  
  
control MyCompareChecksum(inout headers hdr, inout metadata meta) {  
    apply { }  
}  
  
/******  
***** S W I T C H *****  
*****/  
  
V1swith1  
    MyParser(),  
    MyVerifyChecksum(),  
    MyIngress(),  
    MyEgress(),  
    MyComputeChecksum(),  
    MyOutput();  
}
```

Extract packet
headers

This is the
final state

```
#include <core.p4>  
#include <my.p4>  
  
//*****  
//***** P A R S E R *****  
//*****/  
  
typedef bit[40] macAddr_t;  
  
header ethernet_t {  
    macAddr_t dstAddr;  
    macAddr_t srcAddr;  
    bit[12] etherType;  
}  
  
struct metadata_t {  
    // empty w/  
}  
  
struct headers_t {  
    ethernet_t ethernet;  
}  
  
//*****  
//***** P A R S E R *****  
//*****/  
  
parser MyParser(packet_in packet,  
    out headers hdr,  
    inout metadata meta,  
    inout standard_metadata_t standard_metadata) {  
  
    state start{  
        packet.extract(hdr.ethernet);  
        transition accept;  
    }  
  
//*****  
//***** C H E C K S U M   V E R I F I C A T I O N *****  
//*****/  
  
control MyVerifyChecksum(inout headers hdr, inout metadata meta) {  
    apply { }  
}  
  
//*****  
//***** I N G R E S S   P R O C E S S I N G *****  
//*****/  
  
control MyIngress(inout headers hdr,  
    inout metadata meta,  
    inout standard_metadata_t standard_metadata) {  
  
    action swap_mac{  
        macAddr_t tmp;  
        tmp = hdr.ethernet.srcAddr;  
        hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;  
        hdr.ethernet.dstAddr = tmp;  
    }  
  
    apply {  
        // Swap MAC addresses.  
        swap_mac();  
    }  
  
    //Set output port on input port  
    standard_metadata.egress_spec = standard_metadata.ingress_port;  
}  
  
//*****  
//***** E G R E S S   P R O C E S S I N G *****  
//*****/  
  
control MyEgress(inout headers hdr,  
    inout metadata meta,  
    inout standard_metadata_t standard_metadata) {  
    apply { }  
}  
  
//*****  
//***** C H E C K S U M   C O M P A R I S O N *****  
//*****/  
  
control MyCompareChecksum(inout headers hdr, inout metadata meta) {  
    apply { }  
}  
  
//*****  
//***** D E F A U L T *****  
//*****/  
  
control MyDefaultParser(packet_out packet, in headers hdr) {  
    apply {  
        // parsed headers have to be added again into the packet  
        packet.set(hdr.ethernet);  
    }  
}  
  
//*****  
//***** S W I T C H *****  
//*****/  
  
V1swith1  
    MyParser(),  
    MyVerifyChecksum(),  
    MyIngress(),  
    MyEgress(),  
    MyComputeChecksum(),  
    MyOutput();  
}
```

<https://github.com/nsg-ethz/p4-learning/blob/master/examples/reflector/reflector.p4>

Packet reflector in P4

```
/******  
***** P A R S E R *****  
*****/  
  
parser MyParser(packet_in packet,  
    out headers hdr,  
    inout metadata meta,  
    inout standard_metadata_t standard_metadata) {  
  
    state start{  
        packet.extract(hdr.ethernet);  
        transition accept;  
    }  
  
}  
  
/******  
***** C H E C K S U M   V E R I F I C A T I O N *****  
*****/  
  
control MyVerifyChecksum(inout headers hdr, inout metadata meta) {  
    apply { }  
}
```

Extract packet
headers

This is the
final state

Part of the control flow
to modify the packet

```
#include <uvm.h>  
#include <uvm/verilog.h>  
  
//*****  
//***** P A R S E R *****  
//*****/  
  
typedef bit[48] macAddr_t;  
  
header ethernet_t {  
    macAddr_t dstAddr;  
    macAddr_t srcAddr;  
    bit[16] etherType;  
};  
  
struct metadata {  
    // empty w/  
};  
  
struct headers {  
    ethernet_t ethernet;  
};  
  
//*****  
//***** P A R S E R *****  
//*****/  
  
parser MyParser(packet_in packet,  
    out Headers hdr,  
    inout metadata meta,  
    inout standard_metadata_t standard_metadata) {  
  
    state start{  
        packet.extract(hdr.ethernet);  
        transition accept;  
    }  
  
}  
  
//*****  
//***** C H E C K S U M   V E R I F I C A T I O N *****  
//*****/  
  
control MyVerifyChecksum(inout headers hdr, inout metadata meta) {  
    apply { }  
}  
  
//*****  
//***** I N G R E S S   P R O C E S S I N G *****  
//*****/  
  
control MyIngress(inout headers hdr,  
    inout metadata meta,  
    inout standard_metadata_t standard_metadata) {  
  
    action swap_mac{  
        macAddr_t tmp;  
        tmp = hdr.ethernet.srcAddr;  
        hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;  
        hdr.ethernet.dstAddr = tmp;  
    }  
  
    apply {  
        // Swap MAC addresses.  
        swap_mac();  
    }  
  
    //Set output port on input port  
    standard_metadata.egress_spec = standard_metadata.ingress_port;  
}  
  
//*****  
//***** E G R E S S   P R O C E S S I N G *****  
//*****/  
  
control MyEgress(inout headers hdr,  
    inout metadata meta,  
    inout standard_metadata_t standard_metadata) {  
    apply { }  
}  
  
//*****  
//***** C H E C K S U M   C O M P A R I S O N *****  
//*****/  
  
control MyCompareChecksum(inout headers hdr, inout metadata meta) {  
    apply { }  
}  
  
//*****  
//***** D E P A R S E R *****  
//*****/  
  
deparser MyDeparser(packet_out packet, in headers hdr) {  
    apply {  
        // parsed headers have to be added again into the packet  
        packet.emit(hdr.ethernet);  
    }  
}  
  
//*****  
//***** S W I T C H *****  
//*****/  
  
switch {  
    MyParser(),  
    MyVerifyChecksum(),  
    MyIngress(),  
    MyEgress(),  
    MyCompareChecksum(),  
    MyDeparser();  
};  
  
main {  
    uvm_test_top();  
}
```

<https://github.com/nsg-ethz/p4-learning/blob/master/examples/reflector/reflector.p4>

Packet reflector in P4

```
/******  
***** INGRESS PROCESSING *****  
*****  
control MyIngress(inout headers hdr,  
    inout metadata meta,  
    inout standard_metadata_t standard_metadata) {  
  
    action swap_mac(){  
        macAddr_t tmp;  
        tmp = hdr.ethernet.srcAddr;  
        hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;  
        hdr.ethernet.dstAddr = tmp;  
    }  
  
    apply {  
        // Swap MAC addresses.  
        swap_mac();  
  
        //Set Output port == Input port  
        standard_metadata.egress_spec = standard_metadata.ingress_port;  
    }  
}
```

Part of the control flow
to modify the packet

```
#include <core.p4>  
#include <myModel.p4>  
  
//*****  
//***** DEPENDENCIES *****  
//*****  
  
typedef bit<48> macAddr_t;  
  
header ethernet_t {  
    macAddr_t dstAddr;  
    macAddr_t srcAddr;  
    bit<2> etherType;  
}  
  
struct metadata_t {  
    // empty w/  
}  
  
struct headers {  
    ethernet_t ethernet;  
}  
  
//*****  
//***** PARSING *****  
//*****  
parser MyParser(packet_in packet,  
    Out Headers hdr,  
    Inout metadata meta,  
    Inout standard_metadata_t standard_metadata) {  
  
    state start{  
        packet.extract(hdr.ethernet);  
        Transition accept;  
    }  
}  
  
//*****  
//***** CHECKSUM VERIFICATION *****  
//*****  
control MyVerifyChecksum(inout headers hdr, inout metadata meta) {  
    apply { }  
}  
  
//*****  
//***** INGRESS PROCESSING *****  
//*****  
control MyIngress(inout headers hdr,  
    Inout metadata meta,  
    Inout standard_metadata_t standard_metadata) {  
  
    action swap_mac(){  
        macAddr_t tmp;  
        tmp = hdr.ethernet.srcAddr;  
        hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;  
        hdr.ethernet.dstAddr = tmp;  
    }  
  
    apply {  
        // Swap MAC addresses.  
        swap_mac();  
  
        //Set Output port == Input port  
        standard_metadata.egress_spec = standard_metadata.ingress_port;  
    }  
}  
  
//*****  
//***** EGRESS PROCESSING *****  
//*****  
control MyEgress(inout headers hdr,  
    Inout metadata meta,  
    Inout standard_metadata_t standard_metadata) {  
    apply { }  
}  
  
//*****  
//***** CHECKSUM COMPUTATION *****  
//*****  
control MyComputeChecksum(inout headers hdr, inout metadata meta) {  
    apply { }  
}  
  
//*****  
//***** DEPACKET *****  
//*****  
control MyDeParser(packet_out packet, In headers hdr) {  
    apply {  
        // packet headers have to be added again into the packet  
        packet.emit(hdr.ethernet);  
    }  
}  
  
//*****  
//***** SWITCH *****  
//*****  
#include  
MyParser(),  
MyVerifyChecksum(),  
MyIngress(),  
MyComputeChecksum(),  
MyDeParser()  
  
int main();
```

<https://github.com/nsg-ethz/p4-learning/blob/master/examples/reflector/reflector.p4>

Packet reflector in P4

```
/******
*****  INGRESS PROCESSING  *****
******/

control MyIngress(inout headers hdr,
                  inout metadata meta,
                  inout standard_metadata_t standard_metadata) {

    action swap_mac(){
        macAddr_t tmp;
        tmp = hdr.ethernet.srcAddr;
        hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;
        hdr.ethernet.dstAddr = tmp;
    }

    apply {
        // Swap MAC addresses.
        swap_mac();

        //Set Output port == Input port
        standard_metadata.egress_spec = standard_metadata.ingress_port;
    }
}
```

Describe how headers shall be processed

```
#include <core.p4>
#include <my.p4>

//*****
//*****  INGRESS PROCESSING  *****
//*****

typedef bit[48] macAddr_t;

struct ethernet_t {
    macAddr_t dstAddr;
    macAddr_t srcAddr;
    bit[16] etherType;
}

struct metadata {
    // empty
}

struct headers {
    ethernet_t ethernet;
}

//*****
//*****  CHECKSUM VERIFICATION  *****
//*****

parser MyParser(packet_in packet,
                out Headers hdr,
                inout metadata meta,
                inout standard_metadata_t standard_metadata) {

    state start {
        packet.extract(hdr.ethernet);
        transition accept;
    }

    //*****
    //*****  CHECKSUM VERIFICATION  *****
    //*****

    control MyVerifyChecksum(inout headers hdr, inout metadata meta) {
        apply { }
    }

    //*****
    //*****  INGRESS PROCESSING  *****
    //*****

    control MyIngress(inout headers hdr,
                     inout metadata meta,
                     inout standard_metadata_t standard_metadata) {

        action swap_mac(){
            macAddr_t tmp;
            tmp = hdr.ethernet.srcAddr;
            hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;
            hdr.ethernet.dstAddr = tmp;
        }

        apply {
            // Swap MAC addresses.
            swap_mac();

            //Set Output port == Input port
            standard_metadata.egress_spec = standard_metadata.ingress_port;
        }

        //*****
        //*****  INGRESS PROCESSING  *****
        //*****

        control MyEgress(inout headers hdr,
                         inout metadata meta,
                         inout standard_metadata_t standard_metadata) {
            apply { }
        }

        //*****
        //*****  CHECKSUM COMPARISON  *****
        //*****

        control MyComputeChecksum(inout headers hdr, inout metadata meta) {
            apply { }
        }

        //*****
        //*****  DEPARSER  *****
        //*****

        control MyDeparser(packet_out packet, in headers hdr) {
            apply {
                // parsed headers have to be added again into the packet
                packet.emit(hdr.ethernet);
            }
        }

        //*****
        //*****  SWITCH  *****
        //*****

        //*****
        //*****  MAIN  *****
        //*****

        MyParser(),
        MyVerifyChecksum(),
        MyIngress(),
        MyComputeChecksum(),
        MyDeparser()
    }
}
```

<https://github.com/nsg-ethz/p4-learning/blob/master/examples/reflector/reflector.p4>

Packet reflector in P4

```
/****** INGRESS PROCESSING *****/
control MyIngress(inout headers hdr,
                  inout metadata meta,
                  inout standard_metadata_t standard_metadata) {

    action swap_mac(){
        macAddr_t tmp;
        tmp = hdr.ethernet.srcAddr;
        hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;
        hdr.ethernet.dstAddr = tmp;
    }

    apply {
        // Swap MAC addresses.
        swap_mac();

        //Set Output port == Input port
        standard_metadata.egress_spec = standard_metadata.ingress_port;
    }
}
```

Describe how headers shall be processed

Fragments manipulating headers/metadata (similar to C functions)

```
#include <core.p4>
#include <my.p4>

//***** INGRESS PROCESSING *****/
typedef bit[48] macAddr_t;

struct ethernet_t {
    macAddr_t dstAddr;
    macAddr_t srcAddr;
    bit[16] etherType;
};

struct metadata_t {
    // empty
};

struct headers_t {
    ethernet_t ethernet;
};

//***** CHECKSUM VERIFICATION *****/
parser MyParser(packet_in packet,
                out Headers hdr,
                inout metadata meta,
                inout standard_metadata_t standard_metadata) {

    state start {
        packet.extract(hdr.ethernet);
        transition accept;
    }

    //***** CHECKSUM VERIFICATION *****/

    control MyVerifyChecksum(inout headers_t hdr, inout metadata meta) {
        apply {
            // Verify checksum
        }
    }

    //***** INGRESS PROCESSING *****/

    control MyIngress(inout headers_t hdr,
                     inout metadata meta,
                     inout standard_metadata_t standard_metadata) {

        action swap_mac(){
            macAddr_t tmp;
            tmp = hdr.ethernet.srcAddr;
            hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;
            hdr.ethernet.dstAddr = tmp;
        }

        apply {
            // Swap MAC addresses.
            swap_mac();

            //Set Output port == Input port
            standard_metadata.egress_spec = standard_metadata.ingress_port;
        }
    }

    //***** EGRESS PROCESSING *****/

    control MyEgress(inout headers_t hdr,
                    inout metadata meta,
                    inout standard_metadata_t standard_metadata) {
        apply {
            // Egress processing
        }
    }

    //***** CHECKSUM CORRECTION *****/

    control MyComputeChecksum(inout headers_t hdr, inout metadata meta) {
        apply {
            // Compute checksum
        }
    }

    //***** DEFRAGMENT *****/

    control MyDeFragment(packet_out packet, in headers_t hdr) {
        apply {
            // Defragment packet
            packet.emit(hdr.ethernet);
        }
    }

    //***** SWITCH *****/

    //***** MAIN *****/
    MyParser(),
    MyVerifyChecksum(),
    MyIngress(),
    MyComputeChecksum(),
    MyDeFragment()
}

int main() {
    // Main function
}
```

<https://github.com/nsg-ethz/p4-learning/blob/master/examples/reflector/reflector.p4>

Packet reflector in P4

```
/****** INGRESS PROCESSING *****/
control MyIngress(inout headers hdr,
                  inout metadata meta,
                  inout standard_metadata_t standard_metadata) {

    action swap_mac(){
        macAddr_t tmp;
        tmp = hdr.ethernet.srcAddr;
        hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;
        hdr.ethernet.dstAddr = tmp;
    }

    apply {
        // Swap MAC addresses.
        swap_mac();

        //Set Output port == Input port
        standard_metadata.egress_spec = standard_metadata.ingress_port;
    }
}
```

Describe how headers shall be processed

Fragments manipulating headers/metadata (similar to C functions)

The control flow

```
#include <core.p4>
#include <my.p4>

//***** INGRESS PROCESSING *****/
typedef bit[48] macAddr_t;

struct ethernet_t {
    macAddr_t dstAddr;
    macAddr_t srcAddr;
    bit[16] etherType;
};

struct metadata {
    // empty
};

struct headers {
    ethernet_t ethernet;
};

//***** CHECKSUM VERIFICATION *****/
parser MyParser(packet_in packet,
                out Headers hdr,
                inout metadata meta,
                inout standard_metadata_t standard_metadata) {

    state start {
        packet.extract(hdr.ethernet);
        transition accept;
    }

//***** CHECKSUM VERIFICATION *****/

control MyVerifyChecksum(inout headers hdr, inout metadata meta) {
    apply { }
}

//***** INGRESS PROCESSING *****/
control MyIngress(inout headers hdr,
                  inout metadata meta,
                  inout standard_metadata_t standard_metadata) {

    action swap_mac(){
        macAddr_t tmp;
        tmp = hdr.ethernet.srcAddr;
        hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;
        hdr.ethernet.dstAddr = tmp;
    }

    apply {
        // Swap MAC addresses.
        swap_mac();

        //Set Output port == Input port
        standard_metadata.egress_spec = standard_metadata.ingress_port;
    }
}

//***** EGRESS PROCESSING *****/
control MyEgress(inout headers hdr,
                  inout metadata meta,
                  inout standard_metadata_t standard_metadata) {
    apply { }
}

//***** CHECKSUM CORRECTION *****/
control MyComputeChecksum(inout headers hdr, inout metadata meta) {
    apply { }
}

//***** DEFRAGMENT *****/
control MyDeFragment(packet_out packet, in headers hdr) {
    apply {
        // packet headers have to be added again into the packet
        packet.emit(hdr.ethernet);
    }
}

//***** SWITCH *****/
//***** MAIN *****/
//***** INIT *****/
MyParser(),
MyVerifyChecksum(),
MyIngress(),
MyComputeChecksum(),
MyDeFragment();

main;
```

<https://github.com/nsg-ethz/p4-learning/blob/master/examples/reflector/reflector.p4>

Packet reflector in P4

```
/* ***** INGRESS PROCESSING ***** */
/* ***** */

control MyIngress(inout headers hdr,
                  inout metadata meta,
                  inout standard_metadata_t standard_metadata) {

    action swap_mac() {
        macAddr_t tmp;
        tmp = hdr.ethernet.srcAddr;
        hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;
        hdr.ethernet.dstAddr = tmp;
    }

    apply {
        // Swap MAC addresses.
        swap_mac();

        // Set Output port == Input port
        standard_metadata.egress_spec = standard_metadata.ingress_port;
    }
}
```

Describe how headers shall be processed

Fragments manipulating headers/metadata (similar to C functions)

The control flow

Target dependent

```
#include <core.p4>
#include <model.p4>

// ***** DEPENDENCIES *****
// ***** */

typedef bit<48> macAddr_t;

struct ethernet_t {
    macAddr_t dstAddr;
    macAddr_t srcAddr;
    bit<16> etherType;
};

struct metadata {
    // empty
};

struct headers {
    ethernet_t ethernet;
};

// ***** PARSING *****
// ***** */

parser MyParser(packet_in packet,
                out Headers hdr,
                inout metadata meta,
                inout standard_metadata_t standard_metadata) {

    state start {
        packet.extract(hdr.ethernet);
        transition accept;
    }

    // ***** CHECKSUM VERIFICATION *****
    // ***** */

    control MyVerifyChecksum(inout headers hdr, inout metadata meta) {
        apply { }
    }

    // ***** INGRESS PROCESSING *****
    // ***** */

    control MyIngress(inout headers hdr,
                     inout metadata meta,
                     inout standard_metadata_t standard_metadata) {

        action swap_mac() {
            macAddr_t tmp;
            tmp = hdr.ethernet.srcAddr;
            hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;
            hdr.ethernet.dstAddr = tmp;
        }

        apply {
            // Swap MAC addresses.
            swap_mac();

            // Set Output port == Input port
            standard_metadata.egress_spec = standard_metadata.ingress_port;
        }
    }

    // ***** EGRESS PROCESSING *****
    // ***** */

    control MyEgress(inout headers hdr,
                    inout metadata meta,
                    inout standard_metadata_t standard_metadata) {
        apply { }
    }

    // ***** CHECKSUM COMPUTATION *****
    // ***** */

    control MyComputeChecksum(inout headers hdr, inout metadata meta) {
        apply { }
    }

    // ***** DEPARSER *****
    // ***** */

    control MyDeparser(packet_out packet, in headers hdr) {
        apply {
            // parsed headers have to be added again into the packet
            packet.emit(hdr.ethernet);
        }
    }

    // ***** SWITCH *****
    // ***** */

    MyParser(),
    MyVerifyChecksum(),
    MyIngress(),
    MyEgress(),
    MyComputeChecksum(),
    MyDeparser()
}

main {
}
```

<https://github.com/nsg-ethz/p4-learning/blob/master/examples/reflector/reflector.p4>

Packet reflector in P4

```
/******
*****  EGRESS PROCESSING  *****
*****/

control MyEgress(inout headers hdr,
                 inout metadata meta,
                 inout standard_metadata_t standard_metadata) {
    apply { }
}

/******
*****  CHECKSUM COMPUTATION  *****
*****/

control MyComputeChecksum(inout headers hdr, inout metadata meta) {
    apply { }
}
```

Part of the control flow
to modify the packet

Part of the control flow
to modify the packet

```
#include <core.p4>
#include <model.p4>

//*****
//*****  PACKET PARSING  *****
//*****

typedef bit[48] macAddr_t;

header ethernet_t {
    macAddr_t dstAddr;
    macAddr_t srcAddr;
    bit[12] etherType;
}

struct metadata_t {
    // empty w/
}

struct headers_t {
    ethernet_t ethernet;
}

//*****
//*****  PACKET PARSING  *****
//*****

parser MyParser(packet_in packet,
               out Headers hdr,
               inout metadata meta,
               inout standard_metadata_t standard_metadata) {
    state start {
        packet.extract(hdr.ethernet);
        transition accept;
    }
}

//*****
//*****  CHECKSUM VERIFICATION  *****
//*****

control MyVerifyChecksum(inout headers hdr, inout metadata meta) {
    apply { }
}

//*****
//*****  INGRESS PROCESSING  *****
//*****

control MyIngress(inout headers hdr,
                  inout metadata meta,
                  inout standard_metadata_t standard_metadata) {
    action swap_mac {
        macAddr_t tmp;
        tmp = hdr.ethernet.srcAddr;
        hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;
        hdr.ethernet.dstAddr = tmp;
    }

    apply {
        // Swap MAC addresses.
        swap_mac();
    }

    // Set output port on input port
    standard_metadata.egress_spec = standard_metadata.ingress_port;
}

//*****
//*****  EGRESS PROCESSING  *****
//*****

control MyEgress(inout headers hdr,
                 inout metadata meta,
                 inout standard_metadata_t standard_metadata) {
    apply { }
}

//*****
//*****  CHECKSUM COMPUTATION  *****
//*****

control MyComputeChecksum(inout headers hdr, inout metadata meta) {
    apply { }
}

//*****
//*****  PACKET PARSING  *****
//*****

control MyParser(packet_out packet, in headers hdr) {
    apply {
        // parsed headers have to be added again into the packet
        packet.emit(hdr.ethernet);
    }
}

//*****
//*****  SWITCH *****
//*****

V1Main(
    MyParser(),
    MyVerifyChecksum(),
    MyIngress(),
    MyEgress(),
    MyComputeChecksum(),
    MyParser()
);

int main();
```

<https://github.com/nsg-ethz/p4-learning/blob/master/examples/reflector/reflector.p4>

Packet reflector in P4

```
/******  
***** D E P A R S E R *****  
*****/  
control MyDeparser(packet_out packet, in headers hdr) {  
    apply {  
        // parsed headers have to be added again into the packet  
        packet.emit(hdr.ethernet);  
    }  
}  
  
/******  
***** S W I T C H *****  
*****/  
V1Switch(  
    MyParser(),  
    MyVerifyChecksum(),  
    MyIngress(),  
    MyEgress(),  
    MyComputeChecksum(),  
    MyDeparser()  
) main;
```

Assemble output
packet

“main()”

```
#include <core.p4>  
#include <model.p4>  
  
/*****  
***** D E P A R S E R *****  
*****/  
typedef bit[48] macAddr_t;  
  
header ethernet_t {  
    macAddr_t dstAddr;  
    macAddr_t srcAddr;  
    bit[12] etherType;  
}  
  
struct metadata_t {  
    // empty w/  
}  
  
struct headers_t {  
    ethernet_t ethernet;  
}  
  
/*****  
***** P A R S E R *****  
*****/  
parser MyParser(packet_in packet,  
    out Headers hdr,  
    inout metadata meta,  
    inout standard_metadata_t standard_metadata) {  
  
    state start {  
        packet.extract(hdr.ethernet);  
        transition accept;  
    }  
}  
  
/*****  
***** C H E C K S U M   V E R I F I C A T I O N *****  
*****/  
control MyVerifyChecksum(inout Headers hdr, inout metadata meta) {  
    apply { }  
}  
  
/*****  
***** I N G R E S S   P R O C E S S I N G *****  
*****/  
control MyIngress(inout Headers hdr,  
    inout metadata meta,  
    inout standard_metadata_t standard_metadata) {  
  
    action swap_mac {  
        macAddr_t tmp;  
        tmp = hdr.ethernet.srcAddr;  
        hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;  
        hdr.ethernet.dstAddr = tmp;  
    }  
  
    apply {  
        // Swap MAC addresses.  
        swap_mac();  
    }  
  
    // Set output port on input port  
    standard_metadata.egress_spec = standard_metadata.ingress_port;  
}  
  
/*****  
***** E G R E S S   P R O C E S S I N G *****  
*****/  
control MyEgress(inout Headers hdr,  
    inout metadata meta,  
    inout standard_metadata_t standard_metadata) {  
    apply { }  
}  
  
/*****  
***** C H E C K S U M   C O M P U T A T I O N *****  
*****/  
control MyComputeChecksum(inout Headers hdr, inout metadata meta) {  
    apply { }  
}  
  
/*****  
***** D E P A R S E R *****  
*****/  
control MyDeparser(packet_out packet, in Headers hdr) {  
    apply {  
        // parsed headers have to be added again into the packet  
        packet.emit(hdr.ethernet);  
    }  
}  
  
/*****  
***** S W I T C H *****  
*****/  
V1Switch(  
    MyParser(),  
    MyVerifyChecksum(),  
    MyIngress(),  
    MyEgress(),  
    MyComputeChecksum(),  
    MyDeparser()  
) main;
```